

Principia

Implementation Specification

Browser-based three-body IC manifold explorer

Malachy Doherty

March 2026 — Living Document

Abstract

The planar three-body problem has an 8-dimensional symmetry-reduced initial-condition manifold whose structure—fractal basin boundaries, symbolic encounter dynamics, topological persistence across parameter families—remains largely unexplored because no tool exists to navigate it interactively at the resolution the dynamics demand.

Principia is a browser-based WebGPU instrument built to fill that gap: a real-time visualiser that treats the IC manifold as a zoomable map, allocating simulation budget adaptively where the dynamics are complex and leaving smooth regions coarse.

This document is the canonical implementation specification. It defines: the chart-based IC parameterisation and decoder pipeline; the Burrau–Pythagorean family as a structured traversal of the manifold; symplectic integration with COM projection and invariant monitoring; stability metrics from frequency diffusion through FTLE to free-group symbolic dynamics; a composable render graph with 20+ physics visualisation modes; the projective microscope interaction model; and a tiled quadtree architecture built in three layers—flat grid, cached slippy map, adaptive refinement—with explicit data flow contracts between GPU simulation, GPU rendering, and CPU scheduling.

Contents

I	The IC Manifold	6
0.1	Goal	7
0.2	Problem setup: symmetries and gauges	7
0.2.1	Injectivity caveats	8
0.3	General chart-based construction	8
0.3.1	View = coordinates + decoder + canonicaliser	8
0.3.2	Factorisation: masses $\times$ configuration $\times$ momentum	9
0.4	The 8D latent chart	10
0.5	Chart maps Φ	10
0.5.1	Affine slicing	10
0.6	Decoder and canonicaliser	11
0.6.1	Masses	11
0.6.2	Configuration: hyperspherical mass-weighted Jacobi	12
0.6.3	Canonicalise configuration	12
0.6.4	Momentum decode	13
0.6.5	Scale gauge	13
0.6.6	Energy normalisation (optional)	13
0.6.7	No-holes: degeneracy and $t = 0$ collision	13
0.7	Generic chart instantiations	14
0.7.1	Geometry + momentum in a fundamental domain	14
0.7.2	Invariant coordinates (L_z, E)	14
0.7.3	Invariant coordinates (L_z, K)	15
0.7.4	Shape-sphere projection (θ, φ)	15
0.7.5	Mixed-axis charts	17
0.8	Measure and sampling density	18

II	The Burrau–Pythagorean Family	19
0.9	Motivation	20
0.10	The discrete family	20
0.10.1	Euclid’s parametrisation	20
0.10.2	Distinct shapes	20
0.10.3	Jacobi coordinates for Burrau configurations	20
0.10.4	Continuous interpolation	20
0.11	Relaxing constraints	21
0.11.1	Mass simplex	21
0.11.2	Momentum	21
0.11.3	Progressive escalation	22
0.12	Burrau-specific charts	22
0.12.1	The Euclid plane	22
0.12.2	The acute-angle axis	22
0.12.3	Ternary mass plot	23
0.12.4	Bifurcation strips	23
0.13	Central hypothesis	23
III	Interaction Model	24
0.14	The projective microscope	25
0.14.1	Existing controls (unchanged)	25
0.14.2	Tilt: Givens rotation into hidden dimensions	25
0.15	Lock gesture	26
0.16	Locked-pixel inspector	27
0.16.1	Compute model	27
0.16.2	Why CPU and why high precision	27
0.16.3	Inspector overlay contents	27
0.16.4	Relationship to the tiled pipeline	28
0.17	Lookup: entering ICs directly	28
0.17.1	Inverse encode paths	28
0.17.2	Chart-aware validation	29
0.18	Named compound directions	30
0.19	Implementation summary	30

IV	Simulation and Analysis	31
0.20	Equations of motion and integrators	32
0.20.1	KDK leapfrog (default, 2nd order)	32
0.20.2	Yoshida composition (4th and 6th order)	33
0.20.3	RK4 (validation reference)	34
0.20.4	Future direction: KS regularisation	34
0.20.5	Failure handling	34
0.20.6	COM projection and invariant monitoring	34
0.21	Event detection	35
0.21.1	Collision	35
0.21.2	Escape	35
0.22	Stability metrics	36
0.22.1	Shape-sphere observable	36
0.22.2	Frequency diffusion	37
0.22.3	Finite-time Lyapunov exponent (FTLE)	38
0.22.4	Finite-resolution ensemble spread	39
0.22.5	Why FTLE and ensemble spread correlate but are not equivalent	40
0.22.6	Trajectory coherence on the shape sphere	42
0.22.7	Free-group word of the encounter sequence	42
0.22.8	Alternative indicators	44
0.22.9	Stability metric hierarchy	44
0.22.10	Canonical per-sample summary	45
0.22.11	Time-coherent shape representation and animation policy	46
0.22.12	Representative sample choice for coarse tiles	46
V	Rendering Pipeline	47
0.23	Render graph architecture	48
0.24	Shader helper library	49
0.25	Physics visualisation modes	49
0.25.1	Group 1: Conserved quantities and invariants	50
0.25.2	Group 2: Mass and configuration geometry	50
0.25.3	Group 3: Trajectory-derived quantities	50
0.25.4	Group 4: Numerical diagnostics	51

0.25.5	Compositing model	51
0.25.6	GUI colour mode selector	51
0.26	Spherical colour map system	52
0.26.1	Colour spaces	52
0.26.2	Von Mises–Fisher blending engine	52
0.26.3	Seamless LUT sphere	52
0.26.4	CVD simulation	52
0.27	Render modes	53
0.27.1	State taxonomy	53
0.27.2	Event classification mode	53
0.27.3	Stability evolution mode (OKLab/OKLCH)	53
0.27.4	Shape sphere physics overlay	54
0.27.5	Pattern and special modes	55

VI WebGPU Systems Layer 57

0.28	Layered build-up	58
0.28.1	Layer 0: flat grid, no cache	58
0.28.2	Layer 1: tile cache and ancestor fallback	59
0.28.3	Layer 2: adaptive refinement	59
0.29	Tiled quadtree architecture	60
0.29.1	Tile pyramid	60
0.29.2	Camera-to-zoom mapping	61
0.29.3	Parent fallback	61
0.29.4	The quadtree as adaptive scientific sampling	61
0.30	Two-stage tile pipeline	62
0.30.1	Per-pixel compute workflow	62
0.30.2	Ensemble sampling (optional)	62
0.30.3	Rendering from the same canonical data	62
0.30.4	Quality tiers	63
0.30.5	Full-retention mode	63
0.30.6	Export modes	64
0.31	WebGPU execution model	71
0.31.1	Dispatch	71

0.31.2	Deep-zoom precision: tile-local coordinates	71
0.31.3	Linearised decoder for deep zoom	72
0.31.4	Workgroup shape	74
0.31.5	Reduction strategy	74
0.31.6	WGSL constraints	74
0.32	Tile diagnostic payload	74
0.33	Tile-level summary and coherence score	75
0.33.1	Explicit split / keep / merge criteria	81
0.34	Scheduling and progressive refinement	81
0.34.1	Why CPU-managed	81
0.34.2	Tile request	81
0.34.3	Priority	83
0.34.4	Memory strategy and eviction	84
0.34.5	Per-frame update loop	84
0.34.6	Split policy	85
0.34.7	Controls	85
0.35	Validation and failure modes	86
0.36	Cache and invalidation	86
.1	Conventions and numerical defaults	88
.2	Cartesian Jacobi decode (reference only)	89
.3	Colour map quick reference	89

Part I

The IC Manifold

Chart contract. Every view in Principia must define:

1. **Chart coordinates:** a map $\Phi : [0, 1]^2 \rightarrow Y$ producing decode-ready parameters.
2. **Decoder:** $D : Y \rightarrow (\mathbf{r}_i, \mathbf{p}_i, m_i)$ via the canonical pipeline.
3. **Canonicaliser:** C enforcing gauge, scale, and no-holes guarantees.

Lookup must either invert into the current chart or fall back to latent $\mathbf{z}$. Lock and tilt operate on chart coordinates, not raw physical coordinates. The GPU shader is chart-agnostic: it reads $(m_i, \mathbf{r}_i, \mathbf{p}_i)$ and never knows which chart produced them.

0.1 Goal

We want a deterministic map from an exploration domain $U = [0, 1]^d$ to planar three-body initial conditions (ICs), together with a terminal label that is always defined. For every exploration coordinate $\mathbf{s} \in U$ (rendering alias: UV/UVW), we produce

$$\mathbf{s} \mapsto (m_i, \mathbf{r}_i, \mathbf{p}_i)_{i=0}^2$$

in a fixed, dimensionless unit system, and we attach a label describing what happens under simulation (collision/escape/bounded/etc.) *or* why the sample is terminal at decode time.

Totality: the pipeline must never reject a parameter point. The output texture is a total function of UV/UVW.

Two practical constraints shape the design: (i) quotient translation, rotation, reflection, and similarity scaling (default $I = 1$); (ii) ensure continuity of charts/decoders away from unavoidable seams and measure-zero degeneracies.

Momentum is handled in two complementary ways: *free-momentum* decodes full COM-frame momentum (4 DOF), *invariant-momentum* constructs momenta from coordinates like (L_z, E) or (L_z, K) , enforcing feasibility by domain warps.

No holes. Totality is enforced in two layers. Most numerical pathologies are avoided by choosing a stable configuration chart (recommended: hyperspherical mass-weighted Jacobi, §0.6.2). Remaining issues are explicit: decode-time failures emit `DEGENERATE(reason)`, and $t = 0$ near-collisions emit `COLLISION.T0`.

Views are charts, not branches. A view mode is a coordinate choice plus deterministic decode/canonicalise. Downstream simulation and rendering are shared (§0.3).

0.2 Problem setup: symmetries and gauges

An initial condition is $(m_i, \mathbf{r}_i, \mathbf{p}_i)_{i=0}^2$ with $m_i > 0$, evolving under Newton’s law with pairwise $1/r$ potential. We quotient the following symmetries.

Translation. Work in the COM frame: $\sum_i m_i \mathbf{r}_i = \mathbf{0}$ and $\sum_i \mathbf{p}_i = \mathbf{0}$.

Rotation and reflection. Decode configurations directly in a canonical frame (recommended), or rotate so $\boldsymbol{\rho}$ lies on $+x$ and enforce $\lambda_y \geq 0$ with a deadbanded mirror rule (§0.6.3).

Scale. Fix the mass-weighted moment of inertia: $I = \sum_i m_i \|\mathbf{r}_i\|^2 = 1$, exploiting the similarity symmetry $\mathbf{r} \rightarrow \lambda \mathbf{r}, t \rightarrow \lambda^{3/2} t$.

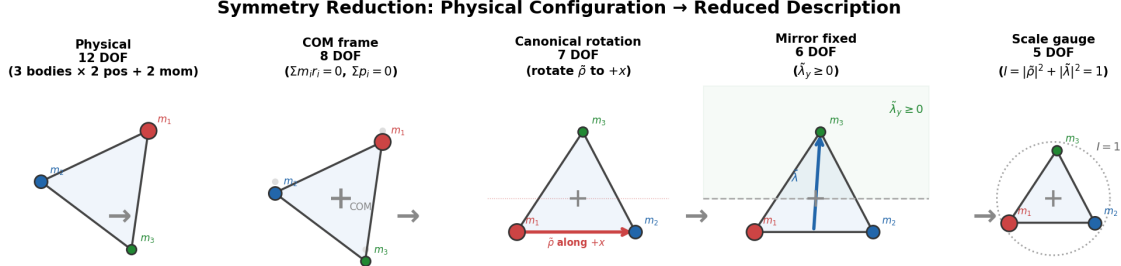


Figure 1: Symmetry reduction cascade for the planar three-body problem. Each step quotients a symmetry and removes degrees of freedom: translation (COM frame), rotation ($\tilde{\rho}$ to $+x$), reflection ($\tilde{\lambda}_y \geq 0$), and scale ($I = 1$). The final 5-DOF reduced space (2 mass + 2 shape + 1 momentum orientation) is what the decoder parameterises.

0.2.1 Injectivity caveats

Global injectivity is impossible with a single smooth chart. The main caveats are: *seams* (discrete symmetry boundaries, stabilised by deadbanding); *degenerate configurations* (tagged explicitly, not rejected); *redundant gauge directions* (avoided by canonical-frame decode); and *permutation symmetry* (S_3 for equal masses, not quotiented by default).

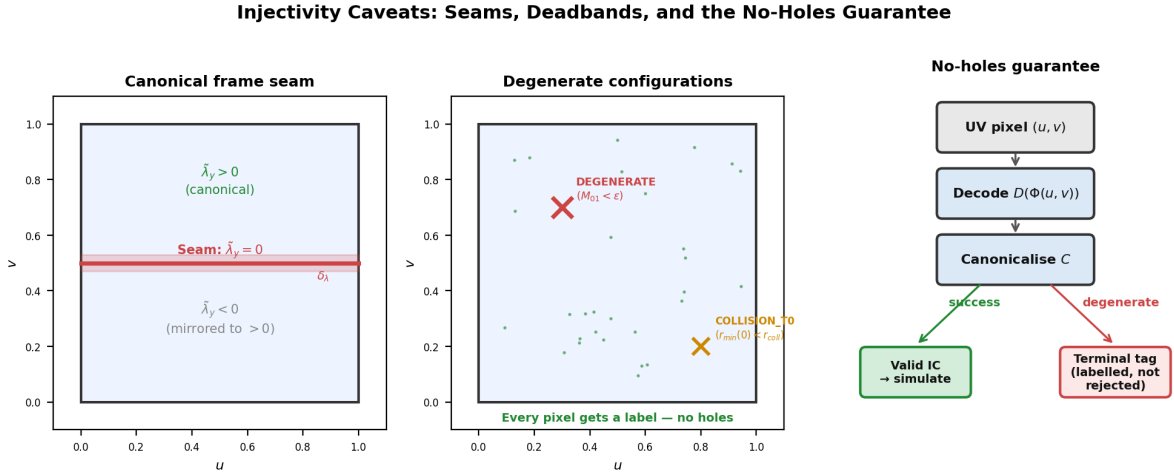


Figure 2: Injectivity caveats. Left: the canonical frame seam where $\tilde{\lambda}_y = 0$, with the deadband δ_λ stabilising the mirror rule. Centre: degenerate and collision-at- $t = 0$ configurations are tagged with terminal labels, not rejected—every pixel gets a well-defined output. Right: the no-holes guarantee as a branching flow: every UV pixel either produces a valid IC for simulation or receives a labelled terminal tag.

0.3 General chart-based construction

0.3.1 View = coordinates + decoder + canonicaliser

Let $U = [0, 1]^d$. A *view* is specified by a coordinate space Y and a map $\Phi : U \rightarrow Y$. A decoder $D : Y \rightarrow \mathcal{S}$ produces a raw state; a canonicaliser $C : \mathcal{S} \rightarrow \mathcal{S}_{\text{can}} \times \mathcal{T}$ fixes remaining gauges and

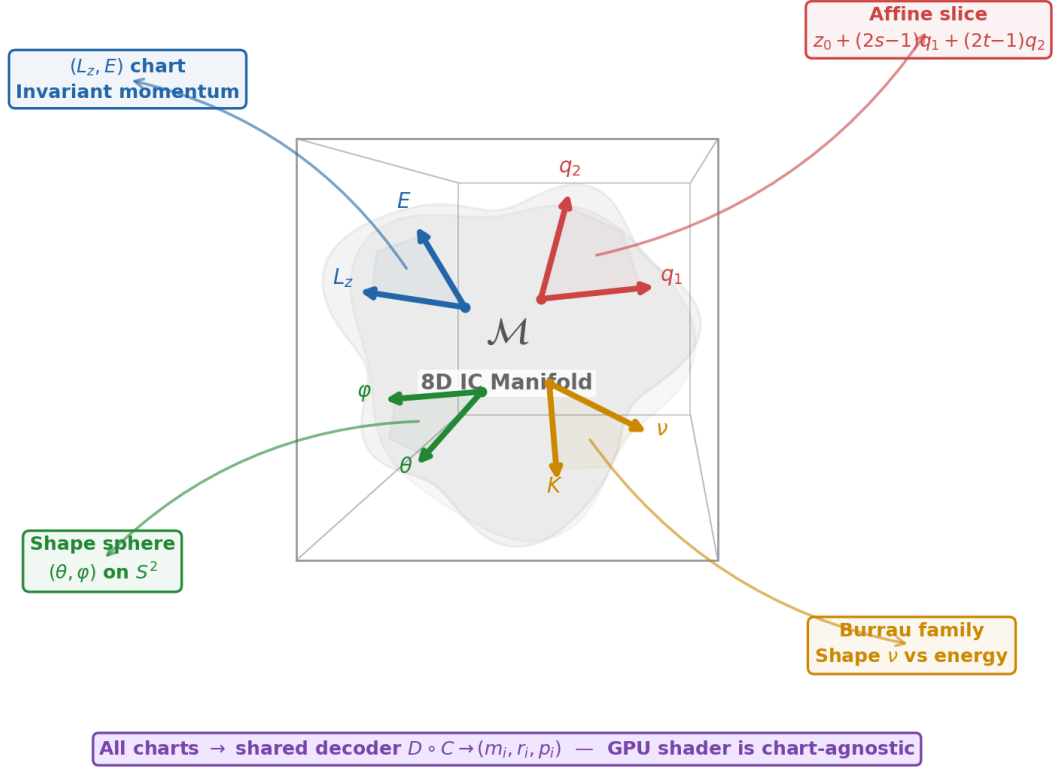


Figure 3: The chart atlas. Four different coordinate frames drawn on the same manifold $\mathcal{M}$ (shown inside a perspective wireframe cube representing the ambient space). Each pair of basis vectors—red (q_1, q_2) for affine slicing, blue (L_z, E) for invariant momentum, green (θ, φ) for the shape sphere, orange (ν, K) for the Burrau family—defines a 2D chart through $\mathcal{M}$ at a different orientation. All charts feed the same shared decoder $D \circ C$; the GPU shader is chart-agnostic.

emits terminal tags. The complete map is

$$F(\mathbf{s}) = C(D(\Phi(\mathbf{s}))).$$

Adding a new view means defining a new (Y, Φ) while leaving canonicalisation, simulation, and rendering unchanged.

0.3.2 Factorisation: masses $\times$ configuration $\times$ momentum

$Y \cong Y_{\text{mass}} \times Y_{\text{cfg}} \times Y_{\text{mom}}$. Mass decode produces $m_i > 0$ with $\sum m_i = 1$; configuration decode produces a COM-frame shape via Jacobi variables; momentum decode produces COM-frame momenta either freely (4 DOF) or subject to invariants. The join assembles $(m_i, \mathbf{r}_i, \mathbf{p}_i)$.

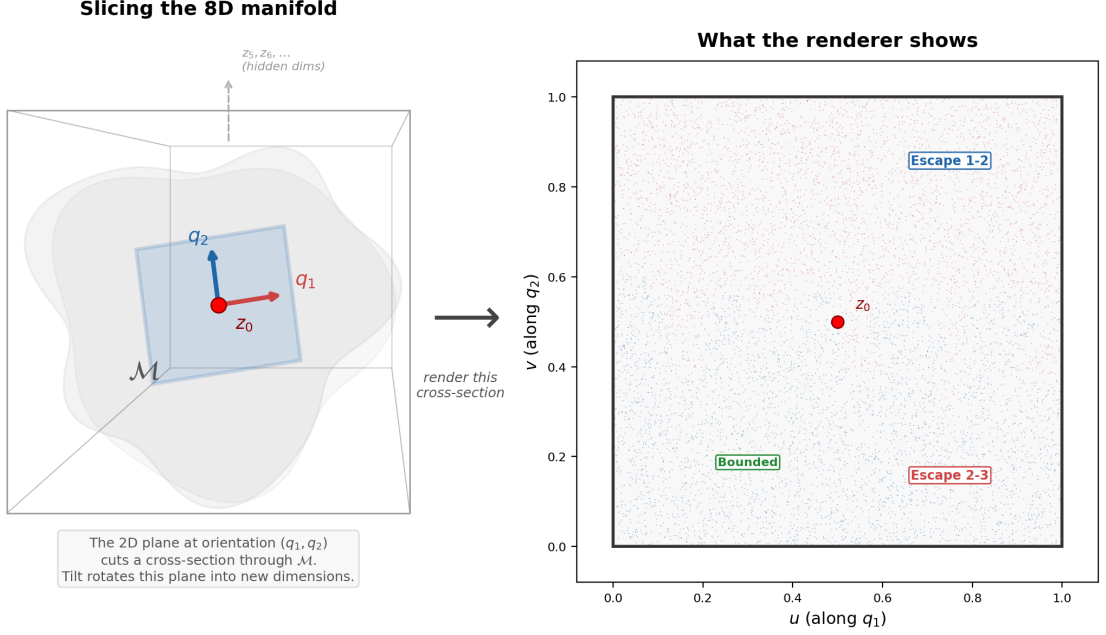


Figure 4: Slicing the manifold. Left: a 2D viewing plane (blue parallelogram) defined by basis vectors q_1 (red) and q_2 (blue) cuts through the manifold $\mathcal{M}$ at the lock point z_0 . The dashed arrow indicates hidden dimensions that the tilt gesture rotates into. Right: the resulting $[0, 1]^2$ cross-section as rendered—each pixel is one simulated trajectory, coloured by outcome (blue: escape 1–2, red: escape 2–3, green: bounded). The fractal basin boundary is visible where the basins meet. Changing the slice orientation (tilt) reveals different cross-sections of the same high-dimensional structure.

0.4 The 8D latent chart

After COM constraints, planar three-body ICs have 2 mass + 4 configuration + 4 momentum degrees of freedom before gauge fixing. The canonical-frame decode (§0.6.2) fixes rotation (pinning $\tilde{\rho}$ to $+x$) and scale ($\tilde{R} = 1$), consuming 2 of the 4 configuration DOF. The true symmetry-reduced manifold is therefore 8-dimensional: 2 configuration (α, β) + 4 momentum + 2 mass. We define

$$\mathbf{z} = (\underbrace{z_\alpha, z_\beta}_{\text{config}}, \underbrace{z_{q0}, z_{q1}, z_{q2}, z_{q3}}_{\text{momentum}}, \underbrace{z_{\mu1}, z_{\mu2}}_{\text{mass}}) \in \mathbb{R}^8.$$

Components are *controls* passed through smooth decoders (sigmoid, softmax, chart warps) to produce physical quantities. The role of $\mathbf{z}$ is to provide a single coordinate system easy to slice and interpolate in. No latent coordinate is wasted on gauge directions.

0.5 Chart maps Φ

0.5.1 Affine slicing

For the latent view, $Y = \mathbb{R}^8$. Choose a slice centre $\mathbf{z}_0$ and d direction vectors $\mathbf{q}_a$:

$$\Phi_{\text{slice}}(\mathbf{s}) = \mathbf{z}_0 + \sum_{a=1}^d (2s_a - 1) \mathbf{q}_a.$$

Decode Pipeline: Chart Coordinates $\rightarrow$ Physical IC

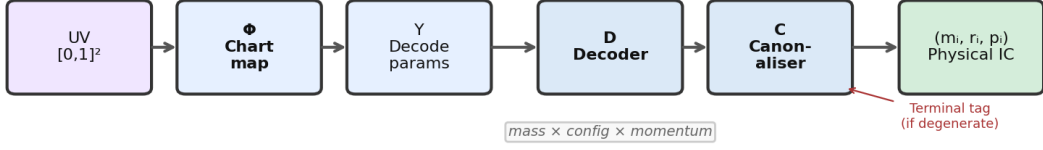


Figure 5: The decode pipeline. Chart coordinates in $[0,1]^2$ are mapped through the chart map Φ , decoder D (factorised as mass $\times$ configuration $\times$ momentum), and canonicaliser C to produce a physical IC. Degenerate configurations are tagged rather than rejected.

IC Manifold Factorisation: $Y \cong Y_{mass} \times Y_{cfg} \times Y_{mom}$

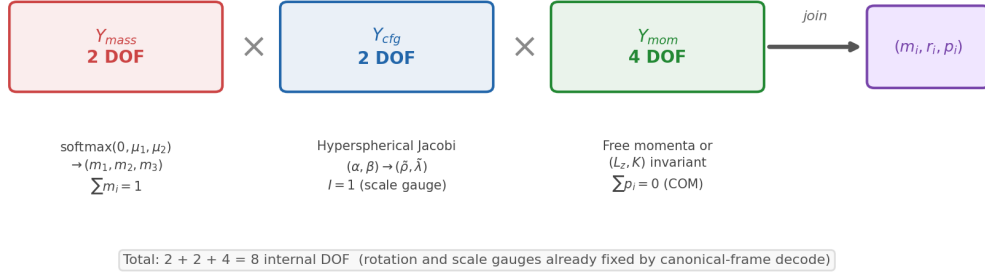


Figure 6: The IC manifold factorisation. The decode parameter space Y decomposes as mass (2 DOF) $\times$ configuration (2 DOF, after rotation and scale gauges are fixed) $\times$ momentum (4 DOF) = 8 DOF total. Each subspace has its own decode stage; the join assembles the physical IC.

For 2D textures: $\mathbf{z}(s, t) = \mathbf{z}_0 + (2s - 1)\mathbf{q}_1 + (2t - 1)\mathbf{q}_2$. For 3D volumes, add $(2r - 1)\mathbf{q}_3$.

Avoid gauge directions. Decode in a symmetry-reduced chart so that affine slice directions correspond to meaningful physical variations rather than wasted gauge motion.

0.6 Decoder and canonicaliser

0.6.1 Masses

$(m_0, m_1, m_2) = \text{softmax}(0, \mu_1, \mu_2)$ with logit saturation $\mu_k \leftarrow \mu_{\max} \tanh(z_{\mu k})$. Define $M_{01} = m_0 + m_1$; if $M_{01} < \varepsilon$, emit DEGENERATE(M01_TINY).

0.6.2 Configuration: hyperspherical mass-weighted Jacobi

Mass-weighted Jacobi vectors:

$$\tilde{\boldsymbol{\rho}} = \sqrt{\mu_\rho} (\mathbf{r}_1 - \mathbf{r}_0), \quad \tilde{\boldsymbol{\lambda}} = \sqrt{\mu_\lambda} \left(\mathbf{r}_2 - \frac{m_0 \mathbf{r}_0 + m_1 \mathbf{r}_1}{M_{01}} \right),$$

with $\mu_\rho = m_0 m_1 / M_{01}$, $\mu_\lambda = m_2 M_{01}$, giving $I = \|\tilde{\boldsymbol{\rho}}\|^2 + \|\tilde{\boldsymbol{\lambda}}\|^2$.

Canonical-frame decode (recommended).

$$\tilde{\boldsymbol{\rho}} = \tilde{R} \cos \alpha (1, 0), \quad \tilde{\boldsymbol{\lambda}} = \tilde{R} \sin \alpha (\cos \beta, \sin \beta),$$

with $\beta \in [0, \pi]$ (mirror fixed by construction), $\tilde{R} = 1$ (scale gauge), and $\alpha_{\min}$ buffer to keep $\|\boldsymbol{\rho}\|$ bounded away from zero. Decode controls:

$$\alpha = \alpha_{\min} + \left(\frac{\pi}{2} - 2\alpha_{\min} \right) \sigma(z_\alpha), \quad \beta = \pi \sigma(z_\beta).$$

Unweight and reconstruct. $\boldsymbol{\rho} = \tilde{\boldsymbol{\rho}} / \sqrt{\mu_\rho}$, $\boldsymbol{\lambda} = \tilde{\boldsymbol{\lambda}} / \sqrt{\mu_\lambda}$. Particle positions via Jacobi reconstruction:

$$\mathbf{r}_{01} = -m_2 \boldsymbol{\lambda}, \quad \mathbf{r}_2 = M_{01} \boldsymbol{\lambda}, \quad \mathbf{r}_0 = \mathbf{r}_{01} - \frac{m_1}{M_{01}} \boldsymbol{\rho}, \quad \mathbf{r}_1 = \mathbf{r}_{01} + \frac{m_0}{M_{01}} \boldsymbol{\rho}.$$

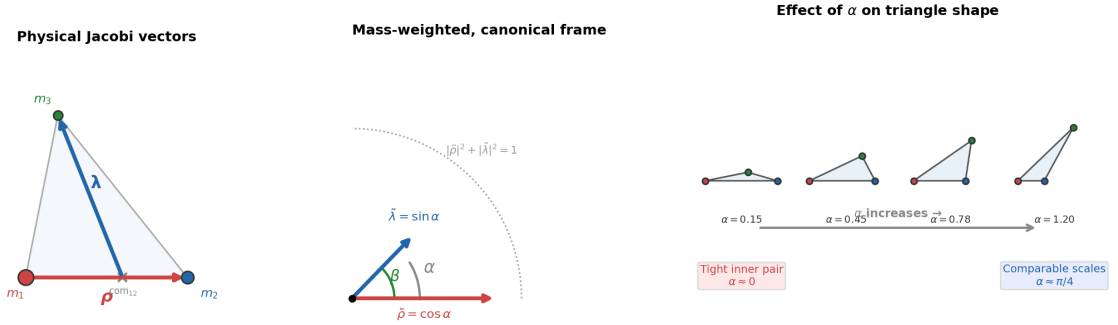


Figure 7: Hyperspherical mass-weighted Jacobi coordinates. Left: the physical Jacobi vectors $\boldsymbol{\rho}$ (inner pair, red) and $\boldsymbol{\lambda}$ (outer body relative to inner COM, blue) drawn on an actual three-body triangle. Centre: the canonical-frame parameterisation with α controlling the $|\tilde{\boldsymbol{\rho}}|/|\tilde{\boldsymbol{\lambda}}|$ ratio, $\beta \in [0, \pi]$ the angle between them, and $I = 1$ as the scale gauge. Right: how increasing α transforms the triangle from a tight inner pair (hierarchical) to comparable scales (compact).

0.6.3 Canonicalise configuration

If not using the canonical-frame decode: rotate all $\mathbf{r}_i$ by $R(-\phi)$ with $\phi = \text{atan2}(\rho_y, \rho_x)$; apply deadbanded mirror rule ($\delta_\lambda = 10^{-12}$) if $\lambda_y < 0$. With canonical-frame decode, both are no-ops away from the seam.

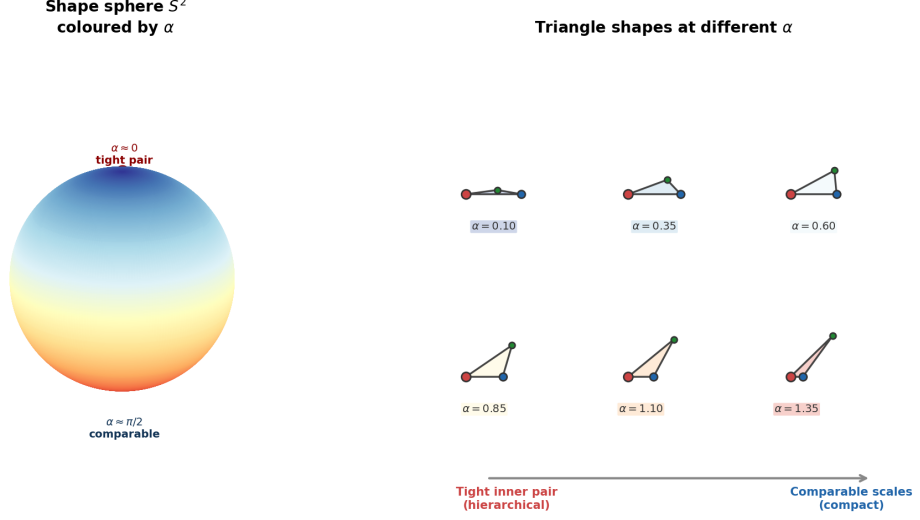


Figure 8: The shape sphere S^2 coloured by the Jacobi angle α . Red (north pole): $\alpha \approx 0$, tight inner pair. Blue (south pole): $\alpha \approx \pi/2$, comparable scales. White equator: $\alpha = \pi/4$, equal $|\tilde{\rho}|$ and $|\tilde{\lambda}|$. Right panel: example triangles at six α values, colour-matched to their position on the sphere. The parameterisation continuously interpolates between hierarchical and compact configurations.

0.6.4 Momentum decode

Free Jacobi momenta. $q_k = q_{\max}(2\sigma(z_{qk}) - 1)$. Jacobi-to-particle map:

$$\mathbf{p}_0 = -\mathbf{p}_\rho - \frac{m_0}{M_{01}}\mathbf{p}_\lambda, \quad \mathbf{p}_1 = \mathbf{p}_\rho - \frac{m_1}{M_{01}}\mathbf{p}_\lambda, \quad \mathbf{p}_2 = \mathbf{p}_\lambda.$$

Apply same rotation/mirror as positions. Optional per-body cap with COM re-enforcement.

0.6.5 Scale gauge

$\mathbf{r}_i \leftarrow \mathbf{r}_i/\ell$, $\mathbf{p}_i \leftarrow \sqrt{\ell}\mathbf{p}_i$ with $\ell = \sqrt{I}$. No-op if $\tilde{R} = 1$ was enforced.

0.6.6 Energy normalisation (optional)

To enforce target energy E^* : compute K_0 , check feasibility ($E^* \geq U$), rescale $\mathbf{p}_i \leftarrow \eta_E \mathbf{p}_i$ with $\eta_E = \sqrt{(E^* - U)/K_0}$.

0.6.7 No-holes: degeneracy and $t = 0$ collision

DEGENERATE(reason) for rare numerical failures. COLLISION_TO(pair) if $r_{\min}(0) < r_{\text{coll}}$, with $t_{\text{event}} = 0$. Every pixel receives a well-defined label.

0.7 Generic chart instantiations

0.7.1 Geometry + momentum in a fundamental domain

Pin $\boldsymbol{\rho} = (\rho_0, 0)$, sweep $\beta(s) = \pi s$, fix $\boldsymbol{\lambda} = \lambda_0(\cos \beta, \sin \beta)$. Momentum: vary direction of $\mathbf{p}_\lambda$, fix $\mathbf{p}_\rho$.

0.7.2 Invariant coordinates (L_z, E)

Feasibility. For any COM-frame momenta, $E = U + K$ with $K \geq 0$, so $E \geq U$. For fixed K the angular momentum satisfies

$$|L_z| \leq \sqrt{2IK} = \sqrt{2I(E - U)}.$$

Domain warp (always-feasible). Choose $K_{\max} > 0$, exponent $\gamma_K \geq 1$:

$$K(t) = K_{\max} t^{\gamma_K}, \quad E(t) = U + K(t), \quad L_{\max}(t) = \sqrt{2IK(t)}, \quad L_z(s, t) = (2s - 1)L_{\max}(t).$$

Deterministic momentum construction. We construct momenta satisfying $\sum \mathbf{p}_i = 0$, $K = E - U$, and $L_z = \sum (\mathbf{r}_i \times \mathbf{p}_i)_z$. Work with velocities $\mathbf{v}_i = \mathbf{p}_i/m_i$ and the mass-weighted inner product $\langle \mathbf{a}, \mathbf{b} \rangle_m = \sum_i m_i \mathbf{a}_i \cdot \mathbf{b}_i$. Let $J(x, y) = (-y, x)$ be the 90° rotation.

(i) *Minimal-energy rigid rotation realising L_z .*

$$\mathbf{v}_i^{(L)} = \omega J \mathbf{r}_i, \quad \omega = \frac{L_z}{I}, \quad K_{\min} = \frac{L_z^2}{2I}.$$

(ii) *Projected direction field adding energy without changing L_z .* Use a deterministic fallback family of seeds. Primary: $\dot{\boldsymbol{\rho}}^{(A)} = \boldsymbol{\rho}$, $\dot{\boldsymbol{\lambda}}^{(A)} = \mathbf{0}$. Fallbacks: $(\mathbf{0}, \boldsymbol{\lambda})$; $(J\boldsymbol{\rho}, \mathbf{0})$; $(\mathbf{0}, J\boldsymbol{\lambda})$. For each, convert to particle velocities:

$$\mathbf{v}_0 = -\frac{m_2}{M} \dot{\boldsymbol{\lambda}} - \frac{m_1}{M_{01}} \dot{\boldsymbol{\rho}}, \quad \mathbf{v}_1 = -\frac{m_2}{M} \dot{\boldsymbol{\lambda}} + \frac{m_0}{M_{01}} \dot{\boldsymbol{\rho}}, \quad \mathbf{v}_2 = \frac{M_{01}}{M} \dot{\boldsymbol{\lambda}}.$$

Set $\mathbf{w}_i^{(0)} := \mathbf{v}_i$. Remove COM drift:

$$\mathbf{c} = \sum_i m_i \mathbf{w}_i^{(0)}, \quad \mathbf{w}_i^{(1)} = \mathbf{w}_i^{(0)} - \frac{\mathbf{c}}{M}.$$

Remove angular momentum component (writing $\mathcal{L}(\mathbf{w}) = \sum_i m_i (\mathbf{r}_i \times \mathbf{w}_i)_z$):

$$\beta_{\mathcal{L}} = \frac{\mathcal{L}(\mathbf{w}^{(1)})}{I}, \quad \mathbf{w}_i^{(2)} = \mathbf{w}_i^{(1)} - \beta_{\mathcal{L}} J \mathbf{r}_i.$$

Normalise: $\mathbf{w}_i = \mathbf{w}_i^{(2)} / \sqrt{\|\mathbf{w}^{(2)}\|_m^2}$. Select the first seed satisfying $\|\mathbf{w}^{(2)}\|_m^2 > \varepsilon_w$ (default $\varepsilon_w = 10^{-10}$); if multiple are valid, choose the one with largest $\|\mathbf{w}^{(2)}\|_m$ for best conditioning. Only emit **DEGENERATE** if all seeds fail.

(iii) *Mix to hit target kinetic energy.* Let $K^* = E - U$. If $K^* < K_{\min}$, emit terminal. Otherwise:

$$a = \sqrt{2 \left(K^* - \frac{L_z^2}{2I} \right)}, \quad \mathbf{v}_i = \mathbf{v}_i^{(L)} + a \mathbf{w}_i, \quad \mathbf{p}_i = m_i \mathbf{v}_i.$$

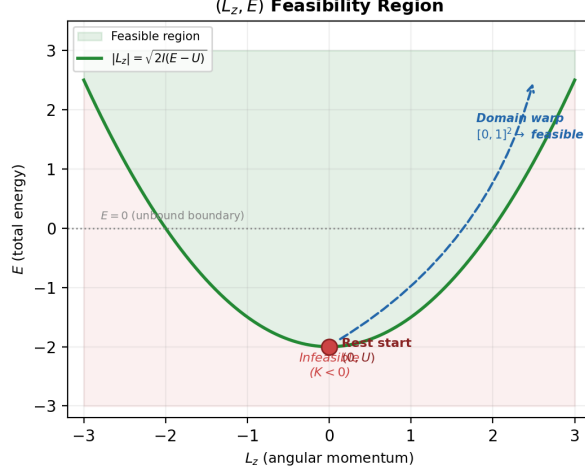


Figure 9: The (L_z, E) feasibility region. The parabolic boundary $|L_z| \leq \sqrt{2I(E - U)}$ separates the feasible region (green, $K \geq 0$) from the infeasible region (red, $K < 0$). The rest-start point $(0, U)$ sits at the parabola’s apex. The domain warp maps the $[0, 1]^2$ UV square into the feasible interior, guaranteeing every pixel produces a valid IC.

0.7.3 Invariant coordinates (L_z, K)

Identical construction with $K^* = K(t)$ directly. Simpler warp since $K \geq 0$ is the natural constraint.

0.7.4 Shape-sphere projection (θ, φ)

The shape sphere S^2 is the quotient of three-body configuration space by translations, rotations, and scale. Its coordinates (θ, φ) parameterise the *shape* of the triangle formed by the three bodies, independent of its size, orientation, and centre of mass. Rendering this directly gives a flat map of shape space with all physics overlay landmarks (collision singularities, Euler configurations, Lagrange poles, BC octant boundaries) at known fixed coordinates.

Chart map.

$$\Phi_{S^2}(u, v) = (\mathbf{n}(\theta(u), \varphi(v)), \mathbf{m}_{\text{fixed}}, \mathbf{p}_{\text{fixed}}),$$

with $\theta(u) = \varepsilon + (\pi - 2\varepsilon)u$, $\varphi(v) = 2\pi v$, and buffer $\varepsilon > 0$ to avoid the collision-singularity poles. Masses and momenta are held fixed; only the triangle shape varies. Each pixel is a different three-body shape at the same mass ratio and the same momentum.

Projection choice. The default is equirectangular (θ and φ are linearly mapped to the rendering axes), which distorts area near the poles but keeps coordinates directly readable. For quantitative area comparisons, an equal-area alternative (Mollweide or Hammer–Aitoff) can be offered, which is important because the Lagrange equilateral-triangle configurations sit at the poles and may be compressed to invisibility in equirectangular.

What lives on this map.

- Collision singularities $\hat{\mathbf{b}}_1, \hat{\mathbf{b}}_2, \hat{\mathbf{b}}_3$: fixed (θ, φ) points, depending on masses.

- Euler collinear configurations: on the equator.
- Lagrange equilateral triangles: at the poles $\hat{\mathbf{I}}^+, \hat{\mathbf{I}}^-$.
- BC octant boundaries, Voronoi cells: fixed curves, rendered as overlay annotations.
- Free-group branch cuts: great-circle arcs from collision points to poles.

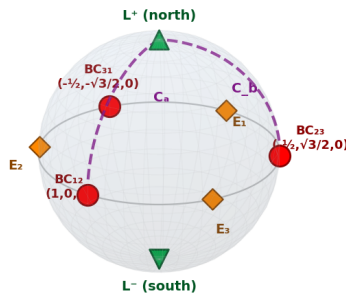


Figure 10: The shape sphere S^2 with all distinguished configurations labelled (equal-mass conventions). Red circles: binary collision singularities BC_{12} , BC_{23} , BC_{31} on the equator at 120° intervals. Orange diamonds: Euler collinear configurations E_1 , E_2 , E_3 (antipodal to BCs). Green triangles: Lagrange equilateral poles L^+ (north), L^- (south). Dashed purple arcs: branch cuts C_a , C_b for the free-group word encoding.

Burrau application. Fix Burrau masses and rest start, sweep (θ, φ) over the full sphere. The right-triangle Burrau configurations appear as a 1D curve on the 2D map (the locus where one interior angle is $\pi/2$). The rest of the map shows what happens to non-right-triangle shapes at the same masses—directly answering whether the right-angle constraint is dynamically special or just geometrically convenient.

The shape sphere as phase portrait. This chart is fundamentally different from all others because the rendering surface and the dynamical state space are the same object. Each pixel is a starting shape $\mathbf{n}(0) \in S^2$; the trajectory $\mathbf{n}(t)$ from that IC traces a path *across the very surface being rendered*. The collision singularities $\hat{\mathbf{b}}_j$ are singular points of the flow. The Euler collinear configurations are equilibria on the equator. The Lagrange equilateral triangles are the poles. The free-group word records which singularities the trajectory wound around—and those windings are visible as loops on the map.

This is the three-body analogue of a Lorenz attractor plot: state space = display space, trajectories are visible curves in the rendered coordinates, and the fractal basin boundaries painted onto the map *are* the separatrices of the flow. When a locked pixel’s trajectory is animated in the inspector overlay, it draws on top of the map it came from: the starting point is the locked pixel, and the trajectory wanders across the same (θ, φ) surface, threading between the collision points.

Formally, the full reduced phase space is T^*S^2 (shape sphere plus shape momenta), so S^2 is the configuration-space projection. Different momenta at the same starting shape produce crossing trajectories on the map—same pixel, different velocity, different fate. This is visible as outcome

class changing when momentum is varied at a fixed shape: the pixel colour changes, but the trajectory paths through the same region of S^2 diverge. A mixed-axis chart pairing θ or φ with a momentum coordinate (as in §0.7.5) lifts this degeneracy by showing the full phase-space cross-section.

Hover-streamline interaction. Because rendering surface = state space, a unique interaction is possible in this chart: as the cursor moves across the map, a trajectory streamline is drawn in real time from the hovered pixel. The implementation is lightweight:

1. Decode the IC from the hovered (u, v) via the chart map.
2. Integrate one trajectory on the CPU at **f64** with fine time resolution (microseconds for a single trajectory—well within frame budget even on every mouse-move event).
3. Project the resulting $\mathbf{n}(t)$ path back to (θ, φ) and draw as a smooth curve on a canvas overlay.

No GPU dispatch, no tile computation, no readback from the **SimResult** buffer. The streamline is a CPU-side overlay that composites on top of the GPU-rendered basin map.

What the user sees. Move the cursor across the shape sphere: a smooth trajectory streams out from the cursor position, threading between the collision singularities $\hat{\mathbf{b}}_j$, visibly changing topology as the cursor crosses basin boundaries. Each loop around a collision point corresponds to a symbol in the free-group word. The word **abBA** is visible as: loop around $\hat{\mathbf{b}}_1$ anticlockwise, loop around $\hat{\mathbf{b}}_2$ anticlockwise, loop around $\hat{\mathbf{b}}_2$ clockwise, loop around $\hat{\mathbf{b}}_1$ clockwise. The symbolic dynamics become directly visible as physical motion on the map.

Relationship to the locked-pixel inspector. The hover streamline is a lightweight preview; the locked-pixel inspector (§0.16) is the full analysis. The hover streamline uses the same CPU integration path but draws only the shape-sphere trajectory. The inspector adds real-space animation, diagnostics, the full free-group word, and energy conservation checks. On lock (click), the hover streamline freezes and the inspector overlay opens on top of it.

0.7.5 Mixed-axis charts

The two rendering axes need not come from the same subspace of the 8D IC manifold. Any pair of coordinates can be the horizontal and vertical axes, with the remaining degrees of freedom frozen at specified values. This enables direct visualisation of phase-space cross-sections that are impossible in any single-subspace chart.

Examples.

Horizontal axis	Vertical axis	What it shows
Shape θ	L_z	how basin structure varies with shape and angular momentum
Shape φ	K	azimuthal orientation vs kinetic energy
ν (Burrau)	K	triangle shape vs energy (bifurcation strip)
Mass m_1	Shape θ	mass–shape coupling at fixed momentum
L_z	θ (shape)	angular momentum vs triangle elongation
α (Jacobi)	p_ρ (radial momentum)	classical phase-space portrait

Implementation. Each axis specifies: which latent coordinate or derived quantity it maps, and what warp function maps $[0, 1]$ to the physical range. The remaining 6 coordinates are set by the chart convention (e.g. “Burrau masses, rest start” fixes 6 of 8). The chart map is then:

$$\Phi_{\text{mixed}}(u, v) = D(\mathbf{z}(u, v)),$$

where $\mathbf{z}(u, v)$ fills coordinate i from u , coordinate j from v , and the rest from the frozen values. This composes with the existing decoder D and requires no new GPU logic—the shader reads $(m_i, \mathbf{r}_i, \mathbf{p}_i)$ as always.

Phase-space plots. When one axis is a configuration coordinate and the other is its conjugate momentum, the rendering is literally a Poincaré section of the three-body phase space. The basin structure in this view shows how configuration-space and momentum-space features interact—information invisible in any configuration-only or momentum-only chart.

0.8 Measure and sampling density

The decoding transforms (sigmoid, softmax, domain warps) induce a non-uniform sampling density in physically meaningful spaces (mass fractions, triangle shapes, energy, angular momentum). This is acceptable for exploratory rendering, but must be documented when quantitative statements are intended.

Examples of non-uniformity. Softmax with saturated logits is smooth but not uniform over the simplex. Uniform in internal angles is not the same as uniform over triangle shapes (the shape sphere has its own area element). Power-law warps $K(t) = K_{\text{max}} t^{\gamma_K}$ deliberately bias density toward low K .

Energy drift as a quality diagnostic. For bounded trajectories, the relative energy drift $|\Delta E/E|$ over the integration horizon serves as a per-pixel integration quality indicator. This can be stored as an additional diagnostic channel and used to flag pixels where numerical accuracy is suspect.

Unless otherwise stated, the default goal is smooth, controllable exploration, not strict uniform sampling.

Three distinct notions of density. *Exploration density:* what makes interaction smooth and useful. *Refinement density:* where the renderer spends compute because the image/dynamics are complicated. *Physical/statistical density:* what would matter for statements like “fraction of IC space showing X.” These are not the same thing. In particular: **refinement density is not physical probability density.** The quadtree concentrates compute at basin boundaries because they are visually and dynamically interesting, not because those ICs are more probable.

Part II

The Burrau–Pythagorean Family

0.9 Motivation

Burrau’s problem (1913): three bodies at the vertices of a 3–4–5 right triangle, each mass equal to its opposite side. This is the simplest member of an infinite family. Every primitive Pythagorean triple (a, b, c) defines a distinct right-triangle Burrau configuration.

0.10 The discrete family

0.10.1 Euclid’s parametrisation

For coprime $m > n > 0$ with $m - n$ odd:

$$a = m^2 - n^2, \quad b = 2mn, \quad c = m^2 + n^2. \quad (1)$$

Every primitive Pythagorean triple arises exactly once (up to leg swap). Non-primitive triples (ka, kb, kc) are dynamically equivalent under the similarity symmetry.

0.10.2 Distinct shapes

Different primitive triples give genuinely non-similar triangles:

(m, n)	Triple	Angles	a/b
(2, 1)	(3, 4, 5)	36.9°, 53.1°	0.75
(3, 2)	(5, 12, 13)	22.6°, 67.4°	0.42
(4, 1)	(15, 8, 17)	61.9°, 28.1°	1.88
(4, 3)	(7, 24, 25)	16.3°, 73.7°	0.29

0.10.3 Jacobi coordinates for Burrau configurations

Place right angle at origin, normalise by hypotenuse, assign Burrau masses:

$$\mathbf{r}_1 = (0, 0), \quad m_1 = \frac{c}{a + b + c}, \quad \mathbf{r}_2 = \left(\frac{a}{c}, 0\right), \quad m_2 = \frac{b}{a + b + c}, \quad \mathbf{r}_3 = \left(0, \frac{b}{c}\right), \quad m_3 = \frac{a}{a + b + c}. \quad (2)$$

Jacobi vectors (inner pair 1–2):

$$\boldsymbol{\rho} = \left(\frac{a}{c}, 0\right), \quad \boldsymbol{\lambda} = \left(-\frac{ab}{c(b + c)}, \frac{b}{c}\right). \quad (3)$$

0.10.4 Continuous interpolation

The Euclid formulae hold for all real $m > n > 0$. The single ratio $\nu := n/m \in (0, 1)$ determines the triangle shape; the acute angle is $\theta(\nu) = \arctan((1 - \nu^2)/(2\nu))$. Primitive triples sit at a countable set of ν values. Everything—positions, Jacobi vectors, Burrau masses—varies smoothly with ν , feeding directly into the Principia decode pipeline.

Burrau-Pythagorean Family: Primitive Triples at Unit Hypotenuse

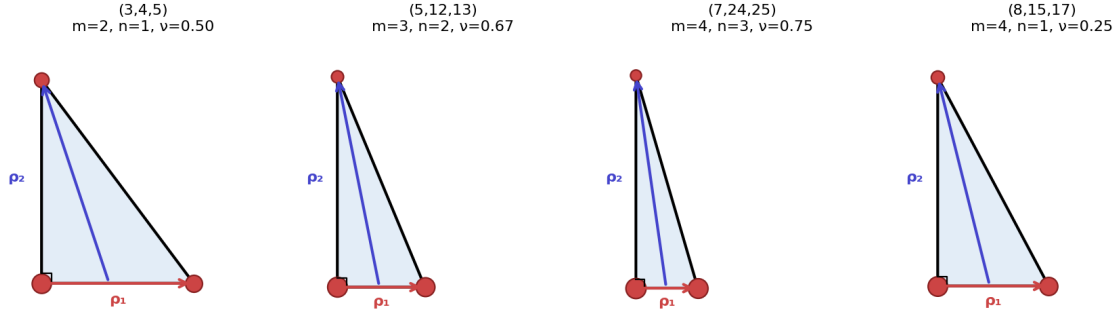


Figure 11: Four primitive Pythagorean triples at unit hypotenuse. Mass at each vertex is proportional to the opposite side (Burrau convention). Red arrows: inner Jacobi vector ρ_1 . Blue arrows: outer Jacobi vector ρ_2 . Different $\nu = n/m$ values produce genuinely distinct triangle shapes with different Jacobi geometry.

0.11 Relaxing constraints

0.11.1 Mass simplex

Decouple masses from geometry: for a fixed triangle, sweep the 2-simplex $\Delta_2 = \{(m_1, m_2, m_3) \in \mathbb{R}_{\geq 0}^3 : \sum m_i = 1\}$. The Burrau point $(c, b, a)/(a + b + c)$ is one distinguished location.

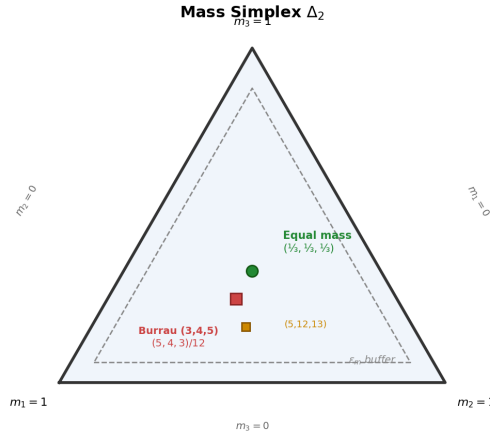


Figure 12: The mass simplex Δ_2 . The Burrau point for the $(3, 4, 5)$ triple and the equal-mass barycentre $(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ are marked. The dashed inner triangle shows the ε_m buffer that keeps masses bounded away from zero. Edge labels indicate which mass vanishes along each boundary.

0.11.2 Momentum

The classical Burrau problem starts from rest. The rest start in the (L_z, E) chart sits at $(0, U)$ —the apex of the feasibility parabola. Sweeping away from it reveals how basins nucleate as the energy budget grows. Alternatively, use free Jacobi momenta (4D) or geometry–momentum mixed slices.

0.11.3 Progressive escalation

Stage 1a (discrete survey): strict Burrau family at rest across primitive triples.

Stage 1b (continuous geometry, 1D): sweep $\nu \in (0, 1)$ continuously; Pythagorean triples as landmarks.

Stage 2a (continuous masses, 2D): fix triangle, sweep Δ_2 at zero momentum.

Stage 2b (continuous momentum, 2D): fix triangle and Burrau masses, sweep (L_z, E) or (L_z, K) .

Stage 3 (combined, 4D+): masses and momenta vary simultaneously.

Stage 4 (full, 8D): remove all constraints; Burrau family becomes cross-sections of the full manifold.

0.12 Burrau-specific charts

All Burrau-family views are standard chart maps $\Phi : [0, 1]^2 \rightarrow Y_{\text{Burrau}}$ feeding the existing decoder. Use the factorisation $Y_{\text{Burrau}} \cong Y_{\text{shape}} \times Y_{\text{mass}} \times Y_{\text{mom}}$. The shape coordinate is $\nu := n/m \in (0, 1)$, with normalised Euclid formulae (divided by m^2):

$$a(\nu) = 1 - \nu^2, \quad b(\nu) = 2\nu, \quad c(\nu) = 1 + \nu^2.$$

Decoder contract. Each Burrau chart produces $(\nu, \mathbf{m}, \text{mom}) \mapsto (\boldsymbol{\rho}, \boldsymbol{\lambda}, \mathbf{m}, \text{mom}) \mapsto (\mathbf{r}_i, \mathbf{p}_i, m_i)$ via the standard downstream canonicaliser. There is no Burrau-specific integrator or renderer.

0.12.1 The Euclid plane

Let $(u, v) \in [0, 1]^2$. Choose $m \in [m_{\min}, m_{\max}]$ (defaults 1, 32), $\nu \in [\nu_{\min}, \nu_{\max}]$ (defaults 1/32, 31/32):

$$m(u) = m_{\min} \left(\frac{m_{\max}}{m_{\min}} \right)^u, \quad \nu(v) = \nu_{\min} + (\nu_{\max} - \nu_{\min})v.$$

$$\boxed{\Phi_{\text{Euclid}}(u, v) = (\nu(v), \mathbf{m}_{\text{Burrau}}(\nu(v)), \text{rest}).}$$

The decoder uses only ν ; m is a display/landmark coordinate for highlighting integer lattice points and primitive-triple overlays. Primitive triples occur where $m, n \in \mathbb{Z}$, $m > n$, $\gcd(m, n) = 1$, $m - n$ odd.

0.12.2 The acute-angle axis

The acute angle $\theta \in (0, \pi/4]$ parameterises right-triangle shape directly. Inversion from θ :

$$\nu(\theta) = \sec \theta - \tan \theta.$$

For (θ, K) : $\theta(u) = \theta_{\min} + (\theta_{\max} - \theta_{\min})u$, $K(v) = K_{\max}v^{\gamma_K}$:

$$\boxed{\Phi_{\theta, K}(u, v) = (\nu(\theta(u)), \mathbf{m}_{\text{Burrau}}(\nu(\theta(u))), (L_z=0, K(v))).}$$

Variants: Φ_{θ, L_z} (fix K , sweep L_z); $\Phi_{\theta, \delta m}$ (sweep mass perturbation, see below).

0.12.3 Ternary mass plot

Fix geometry via ν_0 . Map $(u, v) \in [0, 1]^2$ to the simplex interior: $x = u$, $y = (1 - u)v$, $m_1 = 1 - x - y$, $m_2 = x$, $m_3 = y$. Apply interior buffer $\varepsilon_m = 10^{-4}$ by shrinking toward barycentre: $\mathbf{m} \leftarrow (1 - 3\varepsilon_m)\mathbf{m} + \varepsilon_m(1, 1, 1)$.

$$\Phi_{\text{mass}}(u, v) = (\nu_0, (m_1, m_2, m_3), \text{rest}).$$

The Burrau point $(c, b, a)/(a + b + c)$ is an overlay marker.

0.12.4 Bifurcation strips

Publication-oriented: shape horizontal, one parameter vertical. For (θ, K) :

$$\Phi_{\text{strip}, K}(u, v) = (\nu(\theta(u)), \mathbf{m}_{\text{Burrau}}(\nu(\theta(u))), (L_z=0, K(v))).$$

For $(\theta, \delta m)$, let $\mathbf{m}(u, v) = (1 - v)\mathbf{m}_{\text{Burrau}}(\nu(\theta(u))) + v\mathbf{m}_{\text{target}}$ where $\mathbf{m}_{\text{target}} = (1/3, 1/3, 1/3)$ by default:

$$\Phi_{\text{strip}, \delta m}(u, v) = (\nu(\theta(u)), \mathbf{m}(u, v), \text{rest}).$$

0.13 Central hypothesis

Hypothesis. Fractal basin structures observed in the Burrau–Pythagorean family are cross-sections of codimension- k structures ($k < 6$) in the full 8D IC manifold, rather than artefacts of the constrained submanifold.

Test. Lock onto a basin boundary in a Burrau slice. Continuously rotate the slice basis into unconstrained dimensions. Observe whether the boundary persists, deforms, or dissolves.

Part III

Interaction Model

0.14 The projective microscope

Fix a point $\mathbf{z}_0 \in \mathbb{R}^8$. All existing controls—sliders, tilt, chart mode, zoom—orbit around it. Changing the slicing orientation $(\mathbf{q}_1, \mathbf{q}_2)$ rotates the 2D viewing plane through the fixed point.

0.14.1 Existing controls (unchanged)

Per-dimension sliders: set individual components of $\mathbf{z}_0$.

Tilt: continuously rotate individual basis vectors into chosen latent dimensions. See §0.14.2 below.

Zoom: scalar multiplier on $\mathbf{q}_1, \mathbf{q}_2$ (log-stepping recommended).

Chart mode switching: changes Φ while keeping $\mathbf{z}_0$ as centre.

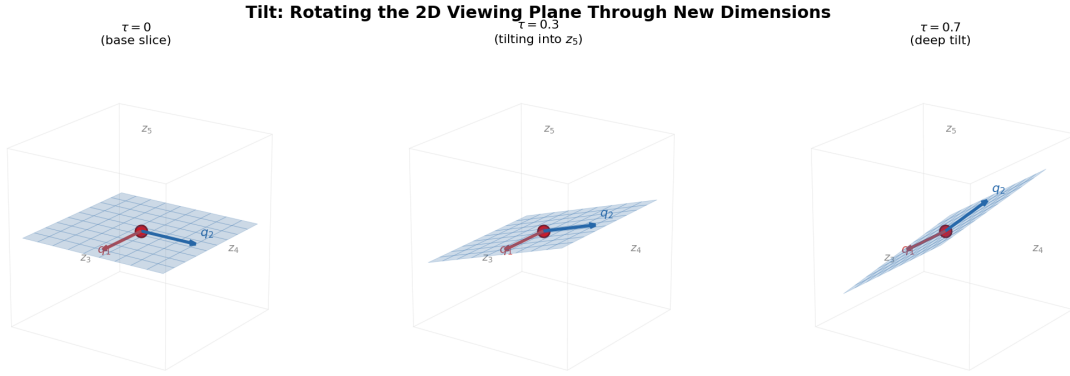


Figure 13: Tilt: rotating the 2D viewing plane through new dimensions. The lock point $\mathbf{z}_0$ (red) stays fixed while the tilt parameter τ continuously rotates the basis vector $\mathbf{q}_2$ out of its original direction (z_4) and into a new dimension (z_5). At $\tau = 0$ the slice shows the base view; increasing τ reveals cross-sections through previously hidden dimensions. This is the central tool for testing the persistence hypothesis (§0.13).

0.14.2 Tilt: Givens rotation into hidden dimensions

Two independent tilt controls, one per basis vector. Each is parameterised by a target latent dimension k and an angle $\tau \in [-90^\circ, +90^\circ]$.

Givens rotation. Given a base basis vector $\mathbf{q}$ (either $\mathbf{q}_1$ or $\mathbf{q}_2$) and a target dimension index k , let $\mathbf{e}_k \in \mathbb{R}^8$ be the k -th standard basis vector. The tilted vector is:

$$\mathbf{q}' = \cos \tau \, \mathbf{q} + \sin \tau \, \mathbf{e}_k.$$

At $\tau = 0$: no tilt, $\mathbf{q}' = \mathbf{q}$. At $\tau = \pm 90^\circ$: fully committed to the target dimension, $\mathbf{q}' = \pm \mathbf{e}_k$. The sign of τ matters—the IC manifold is not generally symmetric under sign flips in latent coordinates, so $+30^\circ$ and -30° reveal genuinely different cross-sections.

Target dimension selection. The 8D latent space has 6 dimensions orthogonal to the current viewing plane. The target selector for each tilt slider offers these 6 directions. The two sliders should default to different targets. Selecting which hidden dimension to tilt into is the core hypothesis-testing gesture: “does this fractal boundary persist when I rotate $\mathbf{q}_2$ toward the angular-momentum axis?”

Orthonormalisation. Apply Gram–Schmidt after every tilt change, *before* deriving tile dispatch uniforms. Anchor on $\mathbf{q}_1$ (the horizontal/primary axis stays exact):

1. Compute tilted vectors: $\mathbf{q}'_1$ from slider 1, $\mathbf{q}'_2$ from slider 2.
2. Normalise $\mathbf{q}'_1$: $\hat{\mathbf{q}}_1 = \mathbf{q}'_1 / \|\mathbf{q}'_1\|$.
3. Orthogonalise $\mathbf{q}'_2$ against $\hat{\mathbf{q}}_1$: $\mathbf{q}_2^\perp = \mathbf{q}'_2 - (\mathbf{q}'_2 \cdot \hat{\mathbf{q}}_1) \hat{\mathbf{q}}_1$.
4. Normalise: $\hat{\mathbf{q}}_2 = \mathbf{q}_2^\perp / \|\mathbf{q}_2^\perp\|$.

Cost: two dot products, two scalar multiplies, two normalisations on 8-vectors. Unmeasurable.

When is correction needed? If the two tilt targets are different dimensions $j \neq k$, the tilted vectors are already orthogonal: $\mathbf{q}'_1 \cdot \mathbf{q}'_2 = \cos \tau_1 \cos \tau_2 (\mathbf{q}_1 \cdot \mathbf{q}_2) + \sin \tau_1 \sin \tau_2 (\mathbf{e}_j \cdot \mathbf{e}_k) = 0$, since $\mathbf{q}_1 \perp \mathbf{q}_2$ and $\mathbf{e}_j \perp \mathbf{e}_k$. Gram–Schmidt is a no-op. If both sliders target the *same* dimension $j = k$, the cross-term $\sin \tau_1 \sin \tau_2 \neq 0$ and correction is required. Always running Gram–Schmidt handles both cases identically.

Degenerate case. If $\|\mathbf{q}_2^\perp\| < \varepsilon$ (the two tilted vectors have become nearly parallel), the second axis is ill-defined. This happens when both sliders target the same dimension at similar angles. Clamp τ_2 to prevent $\|\mathbf{q}_2^\perp\|$ from dropping below $\varepsilon = 10^{-6}$, or fall back to replacing $\mathbf{q}_2$ with the next available orthogonal complement direction.

UI conventions.

- Display in degrees, store internally in radians.
- Range $[-90^\circ, +90^\circ]$, centred at 0° .
- Tilt 1 ($\mathbf{q}_1$) is the primary slider; tilt 2 ($\mathbf{q}_2$) is secondary/collapsed by default.
- Label format: “Tilt $\mathbf{q}_1$ into z_5 : 23.4° ”.
- If both sliders target the same dimension, show a subtle overlap indicator.

0.15 Lock gesture

Select pixel at (s, t) ; snap slice centre to that IC.

Affine charts. $\mathbf{z}_{\text{locked}} = \mathbf{z}_0 + (2s - 1)\mathbf{q}_1 + (2t - 1)\mathbf{q}_2$. CPU-side arithmetic, no GPU readback.

Nonlinear charts. CPU-side replay of Φ and D , or GPU buffer readback.

Locked mode. Sliders frozen at $\mathbf{z}_{\text{locked}}$. Tilt, zoom, chart mode work unchanged but orbit around the locked point. On unlock, current view state preserved as new free-mode starting point.

0.16 Locked-pixel inspector

When a pixel is locked, the user can open an overlay (e.g. via Tab) displaying a full single-trajectory analysis of that IC. This is a fundamentally different compute path from the tiled GPU pipeline.

0.16.1 Compute model

The tiled pipeline gives you the *map*: millions of samples at **f32**, optimised for throughput. The inspector gives you the *microscope*: one sample at arbitrary fidelity, optimised for depth.

The locked pixel provides chart coordinates (u, v) . Decode the IC through the same $\Phi \rightarrow D \rightarrow C$ pipeline, then run a single high-fidelity integration—on the CPU, in **f64** or arbitrary precision if desired. Because it is a single trajectory, there is no parallelism requirement and no struct-size constraint. The output can be as rich as needed:

- full position/velocity history at every integration step (not just M checkpoints)
- real-space trajectories $\mathbf{r}_i(t)$ for animation
- shape-sphere trajectory $\mathbf{n}(t)$ at full time resolution
- energy conservation $\Delta E/E$ as a continuous function of t
- symbolic sequence / free-group word for the encounter history
- frequency analysis over the full trajectory
- all scalar diagnostics ($d_{\min}$, t_{end} , L_n , diffusion, FTLE if desired)
- full IC metadata (masses, positions, momenta, chart coordinates, latent $\mathbf{z}$)

0.16.2 Why CPU and why high precision

One trajectory, no parallelism needed, so **f64** is free. This matters for:

- chasing trajectories deep into near-collision singularities where **f32** loses significant digits
- extending the integration horizon far beyond what the GPU pipeline uses
- validating the GPU pipeline: lock an interesting pixel, recompute at **f64**, and check whether the **f32** tiled result was qualitatively correct or a numerical artefact

This is a built-in validation tool. Any time a user sees something unexpected on the map, they can lock it and verify independently.

0.16.3 Inspector overlay contents

The overlay should display at minimum:

- **Shape-sphere panel**: animated $\mathbf{n}(t)$ trajectory on S^2 with physics overlay (BC/Euler/Lagrange markers)
- **Real-space panel**: animated particle trajectories $\mathbf{r}_i(t)$ with trails
- **IC summary**: masses, positions, momenta, chart coordinates, outcome classification
- **Diagnostics**: energy drift, frequency diffusion, FTLE estimate, encounter sequence
- **Free-group word**: symbolic encoding of the encounter history (which body is closest to which, in what order)

0.16.4 Relationship to the tiled pipeline

The inspector does not read from the tile `SimResult` buffer or tile summaries. It is a completely independent recompute from the IC. This means:

- changing palette or render mode on the map does not affect the inspector
- the inspector can use a different integrator, time step, or horizon than the tiled pipeline
- the inspector result can be compared against the tiled `f32` result as a consistency check
- the map continues to render and refine while the inspector is open

0.17 Lookup: entering ICs directly

Exact IC (full lock). User provides (m, n) , a Pythagorean triple, raw $\mathbf{z}$, or physical coordinates $(m_i, \mathbf{r}_i, \mathbf{p}_i)$. System encodes into the latent chart via the inverse paths below, then locks.

Partial specification (constrained slice). User provides fewer coordinates than needed. System fills unspecified coordinates by canonical defaults: unspecified masses $\rightarrow$ equal masses (unless chart implies Burrau); unspecified geometry in Burrau charts $\rightarrow$ current locked ν or $\nu = 1/2$; unspecified momentum $\rightarrow$ rest start; unspecified latent coordinates $\rightarrow 0$. Then auto-configure the slice basis to span the unspecified degrees of freedom.

0.17.1 Inverse encode paths

Lookup requires a deterministic partial inverse of the decode map. Three cases arise:

1. **Exact inverse available:** latent chart, mass-simplex, Burrau geometry, free Jacobi momenta. The formulae below apply.
2. **Constrained inverse available:** invariant charts (L_z, E) , (L_z, K) where chart coordinates can be recovered but the global latent point is not unique. Choose the canonical representative with smallest latent-space norm.
3. **No canonical inverse:** charts requiring projection or optimisation (e.g. arbitrary physical coordinates that do not align with any active chart). Fall back to latent $\mathbf{z}$ via the general inversion below.

Mass inverse. Given (m_0, m_1, m_2) with $\sum m_i = 1$:

$$\mu_1 = \log \frac{m_1}{m_0}, \quad \mu_2 = \log \frac{m_2}{m_0}, \quad z_{\mu k} = \operatorname{artanh} \left(\frac{\mu_k}{\mu_{\max}} \right).$$

Clamp $|\mu_k|$ to $(1 - \varepsilon_\mu)\mu_{\max}$ before artanh if at saturation, with $\varepsilon_\mu = 10^{-6}$. For a Burrau shape ν : $\mu_1 = \log(b/c)$, $\mu_2 = \log(a/c)$.

Configuration inverse. From physical positions, translate to COM, reconstruct Jacobi vectors $\boldsymbol{\rho} = \mathbf{r}_1 - \mathbf{r}_0$, $\boldsymbol{\lambda} = \mathbf{r}_2 - (m_0\mathbf{r}_0 + m_1\mathbf{r}_1)/M_{01}$. Mass-weight, canonicalise (rotate $\tilde{\boldsymbol{\rho}}$ to $+x$, mirror so $\tilde{\lambda}_y \geq 0$), then:

$$\alpha = \operatorname{atan2}(\|\tilde{\boldsymbol{\lambda}}\|, \|\tilde{\boldsymbol{\rho}}\|), \quad \beta = \operatorname{atan2}(\tilde{\lambda}_y, \tilde{\lambda}_x) \in [0, \pi].$$

Invert the sigmoid decode:

$$s_\alpha = \frac{\alpha - \alpha_{\min}}{\pi/2 - 2\alpha_{\min}}, \quad s_\beta = \frac{\beta}{\pi}, \quad z_\alpha = \text{logit}(s_\alpha), \quad z_\beta = \text{logit}(s_\beta),$$

with s clamped into $[\varepsilon_z, 1 - \varepsilon_z]$, $\varepsilon_z = 10^{-6}$.

Free-momentum inverse. From COM-frame momenta, invert the Jacobi map: $\mathbf{p}_\lambda = \mathbf{p}_2$, $\mathbf{p}_\rho = \mathbf{p}_1 + (m_1/M_{01})\mathbf{p}_\lambda$. Pack components and invert sigmoid:

$$s_k = \frac{1}{2}(q_k/q_{\max} + 1), \quad z_{qk} = \text{logit}(\text{clamp}(s_k, \varepsilon_q, 1 - \varepsilon_q)).$$

If $|q_k| > q_{\max}$, clamp and set a `lookup_clamped` flag.

Canonical inverse policy. (1) Translate to COM. (2) Canonicalise by same rotation/mirror as forward decode. (3) Invert into current chart where possible, otherwise into latent $\mathbf{z}$. (4) If multiple valid inverses remain, choose smallest Euclidean norm in latent space. (5) Surface any clamping in the UI.

0.17.2 Chart-aware validation

Not every point in the 8D latent hypercube $[0, 1]^8$ is valid in every chart, and not every user-entered IC maps to a point inside the hypercube at all. Lookup and lock must validate against the *active chart's* constraints before committing.

Validation layers (applied in order).

1. **Hypercube bounds.** After inverse encoding, check $z_k \in [0, 1]$ for all k . If any z_k lies outside, either clamp (setting `lookup_clamped`) or reject. This catches physical coordinates that exceed the decoder's representable range (e.g. momenta larger than $q_{\max}$, extreme mass ratios beyond $\mu_{\max}$).
2. **Chart feasibility.** Each chart may impose additional constraints beyond the hypercube:
 - (L_z, E) : the point must lie inside the feasibility parabola $|L_z| \leq \sqrt{2I(E - U)}$. A user-entered (L_z, E) pair outside this region has no physical realisation at the current configuration.
 - (L_z, K) : $K \geq 0$ and $K \geq L_z^2/2I$.
 - Mass simplex: all $m_i > 0$ and $\sum m_i = 1$; the interior buffer ε_m further restricts the valid region.
 - Shape sphere: $\theta \in [\varepsilon, \pi - \varepsilon]$ (pole exclusion), $\varphi \in [0, 2\pi)$.
 - Burrau charts: $\nu \in (0, 1)$; specific ν values may correspond to degenerate triangles as $\nu \rightarrow 0$ or $\nu \rightarrow 1$.
 - Mixed-axis charts: inherit constraints from both axes.
3. **Decode sanity.** After decoding, verify: all masses positive; no two bodies coincident ($r_{ij} > r_{\text{coll}}$); COM is at origin (within tolerance); total momentum is zero (within tolerance). These catch edge cases where individually valid coordinates combine to produce a degenerate configuration.

Response to validation failure. Three options, in order of preference:

- **Project:** find the nearest valid point in the chart's feasible region (e.g. project onto the feasibility parabola for (L_z, E)). Lock to the projected point and notify the user.

- **Clamp:** snap each out-of-range coordinate to the nearest boundary, set `lookup_clamped`, and proceed. The UI should indicate that clamping occurred and which coordinates were affected.
- **Reject:** refuse the lookup with a specific error message identifying which constraint failed. Use this only when projection/clamping would produce a qualitatively different IC from what the user intended.

Lock validation. When locking by clicking a pixel, the chart coordinates are by construction valid (the pixel was rendered from a decoded IC). But tilt can move the locked point's neighbourhood into an infeasible region of the chart. The system must detect this and either re-centre to the nearest feasible point or switch to a chart where the current lock is valid.

Implementation note. Each chart type registers a `validate(u,v) -> ValidationResult` function that the lookup/lock system calls. This is a CPU-side check; the GPU shader never sees invalid ICs because they are caught before dispatch. The validation function is also used by the quadtree to skip tiles that fall entirely outside the feasible region (e.g. tiles above the feasibility parabola in the (L_z, E) chart).

0.18 Named compound directions

Pre-defined **q** vectors for physically meaningful slice orientations, composing with the existing tilt mechanism:

Mass perturbation away from Burrau: direction in mass subspace from Burrau logits toward equal masses.

Energy increase at fixed L_z : configuration-dependent direction in momentum subspace.

Burrau-to-unconstrained morph: tilt one basis vector from a configuration direction into a mass or momentum direction, directly testing the hypothesis of §0.13.

0.19 Implementation summary

Component	Status	New work
Affine slice uniforms	Existing	None
Per-dimension sliders	Existing	Freeze/unfreeze in locked mode
Tilt mechanism	Existing	None
Chart mode switching	Existing	None
Zoom	Existing	None
Lock gesture	New	Pixel $\rightarrow$ z readback, pin z ₀
Lookup (exact IC)	New	Input parsing, encode paths
Lookup (partial spec)	New	Encode + auto-configure basis
Named direction library	New	Precomputed q vectors
GPU shader	Existing	No changes

Part IV

Simulation and Analysis

Core architectural principle: one canonical trajectory object. For each sample, the primary geometric object is the checkpointed shape-sphere path $Z_n = (\mathbf{n}(t_1), \dots, \mathbf{n}(t_M))$, $\mathbf{n}(t_m) \in S^2 \subset \mathbb{R}^3$. This is the shared representation for: stability analysis, adaptive refinement, tile summarisation, animation playback, representative-tile rendering, caching and invalidation, and similarity/coherence comparisons across nearby ICs. No separate refinement-only feature pass and no separate rendering-only trajectory representation should exist. All angle-like or phase-like observables (frequency diffusion phase, render-time hue, arc length) are *derived* from $\mathbf{n}(t)$ or from auxiliary scalars computed during integration, never stored as primary trajectory state.

0.20 Equations of motion and integrators

$$\dot{\mathbf{r}}_i = \frac{\mathbf{p}_i}{m_i}, \quad \dot{\mathbf{p}}_i = \sum_{j \neq i} G m_i m_j \frac{\mathbf{r}_j - \mathbf{r}_i}{\|\mathbf{r}_j - \mathbf{r}_i\|^3}.$$

0.20.1 KDK leapfrog (default, 2nd order)

For a time step Δt , with $\mathbf{F}_i(\mathbf{r}) = \sum_{j \neq i} G m_i m_j (\mathbf{r}_j - \mathbf{r}_i) / \|\mathbf{r}_j - \mathbf{r}_i\|^3$:

1. Kick half-step: $\mathbf{p}_i \leftarrow \mathbf{p}_i + \frac{\Delta t}{2} \mathbf{F}_i(\mathbf{r})$.
2. Drift full-step: $\mathbf{r}_i \leftarrow \mathbf{r}_i + \Delta t \mathbf{p}_i / m_i$.
3. Kick half-step: $\mathbf{p}_i \leftarrow \mathbf{p}_i + \frac{\Delta t}{2} \mathbf{F}_i(\mathbf{r})$.

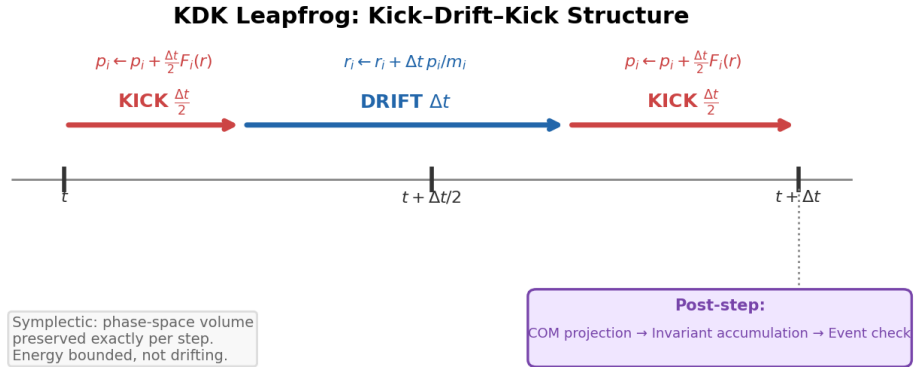


Figure 14: KDK leapfrog structure. Two half-kicks bracket a full drift. The COM projection and invariant accumulation run as post-step operations after the second kick completes. The method is symplectic: phase-space volume is preserved exactly per step.

Adaptive substepping. $\Delta t_{\text{sub}} = \Delta t_{\text{macro}} / N_{\text{sub}}$ where

$$N_{\text{sub}} = \min \left(N_{\text{max}}, \max \left(1, \left\lceil \left(\frac{r_{\text{sub}}}{r_{\text{min}}} \right)^{\gamma_{\text{sub}}} \right\rceil \right) \right), \quad r_{\text{min}} = \min_{i < j} \|\mathbf{r}_i - \mathbf{r}_j\|.$$

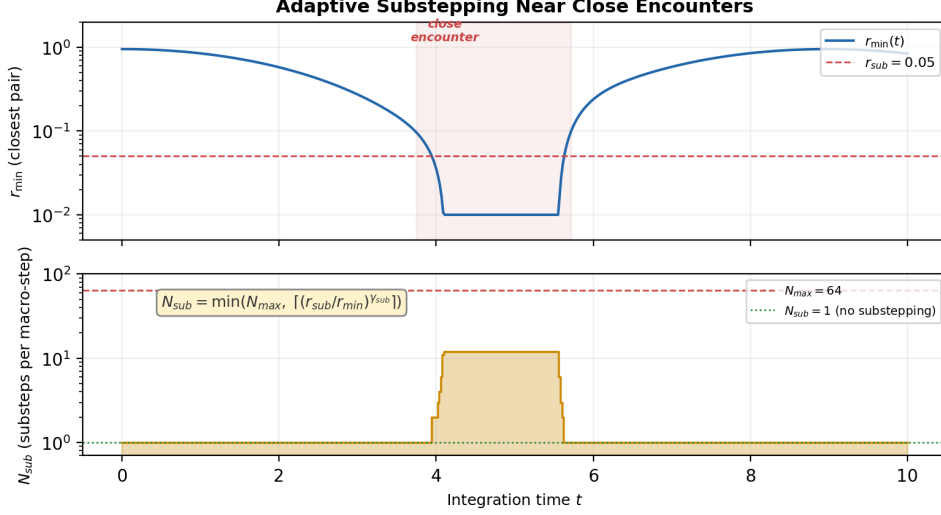


Figure 15: Adaptive substepping near a close encounter. Top: the minimum pairwise separation $r_{\min}(t)$ dips sharply during a close approach. Bottom: the substep count N_{sub} ramps up in response, capped at N_{max} . Away from encounters, $N_{\text{sub}} = 1$ and the integrator runs at full macro-step size.

0.20.2 Yoshida composition (4th and 6th order)

Yoshida’s method composes multiple KDK steps with specific weights to cancel lower-order error terms. The code path is identical to KDK—just called multiple times per macro-step with different sub-step sizes. Symplecticity is preserved.

Yoshida 4th order (3 stages). One macro-step of size h is replaced by three KDK steps with sizes w_1h , w_2h , w_3h where:

$$w_1 = w_3 = \frac{1}{2 - 2^{1/3}} \approx 1.3512, \quad w_2 = -\frac{2^{1/3}}{2 - 2^{1/3}} \approx -1.7024.$$

Note $w_2 < 0$: the middle step goes backward in time. This is a mathematical requirement for explicit high-order symplectic methods. Cost: $3 \times$ force evaluations per step. Recommended default for Balanced/Research tiers.

Yoshida 6th order (7 stages). Seven KDK steps per macro-step, using three independent coefficients $w_1, \dots, w_7$ derived from the same composition principle applied twice. The coefficients are:

$$\begin{aligned} w_1 &= w_7 = 0.78451361047755726382, \\ w_2 &= w_6 = 0.23557321335935813368, \\ w_3 &= w_5 = -1.17767998417887100695, \\ w_4 &= 1.31518632068391121888. \end{aligned}$$

Cost: $7 \times$ force evaluations. Research tier only.

GPU performance note. Higher-order methods add uniform work per thread (more stages per step). On a GPU running millions of trajectories, this is *good* for utilisation: threads in

smooth regions that would otherwise idle while waiting for close-encounter threads to finish their substeps are now doing useful higher-order work. Yoshida 4th is nearly free in wall-clock terms on GPU for this reason.

Integrator selection by quality tier.

Tier	Integrator	Force evals/step	When
Preview	KDK (2nd)	$1\times$	dragging, zooming, first paint
Balanced	Yoshida 4th	$3\times$	default interactive
Research	Yoshida 4th or 6th	$3\times$ or $7\times$	publication-grade, deep zoom
Validation	RK4	$4\times$	short-horizon cross-check

0.20.3 RK4 (validation reference)

Classical 4th-order Runge–Kutta. Higher local accuracy than KDK but not symplectic: energy drifts over long horizons. Use for short-horizon validation against KDK/Yoshida on selected tiles or in the locked-pixel inspector (§0.16). Not for production rendering.

0.20.4 Future direction: KS regularisation

Close encounters dominate the error budget because the $1/r$ gravitational singularity produces arbitrarily large accelerations. Adaptive substepping handles this by brute force (more steps near singularities), but the fundamental issue remains: the equations of motion are singular in Cartesian coordinates.

Kustaanheimo–Stiefel (KS) regularisation transforms the integration variable so the $1/r$ singularity is removed analytically. This allows fixed-step integration through close encounters with bounded error. However, KS regularisation changes the state representation from Cartesian to regularised coordinates, which would ripple through the decoder, canonicaliser, and observable extraction pipeline. It is therefore a v2 architectural change, not a drop-in integrator swap. Named here so the pipeline design does not accidentally preclude it.

0.20.5 Failure handling

NaN/Inf $\rightarrow$ SIM_FAILED. Persistent $N_{\text{sub}} = N_{\text{max}} \rightarrow$ TIMEOUT/MAX_SUBSTEPS.

0.20.6 COM projection and invariant monitoring

Motivation. The atlas decodes trajectories into a COM-consistent representation. Runtime COM projection is numerical housekeeping that keeps floating-point drift from reintroducing the redundant modes the atlas removed. Without projection, coordinates drift as $r_i = r_i^{\text{internal}} + R_{\text{drift}}$, wasting f32 dynamic range on physically irrelevant bulk motion.

Projection rule. After each complete integration step (after the final half-kick in KDK, or after all Yoshida stages), compute

$$R_{\text{com}} = \frac{1}{M} \sum_{i=1}^3 m_i r_i, \quad V_{\text{com}} = \frac{1}{M} \sum_{i=1}^3 m_i v_i,$$

and project:

$$r_i \leftarrow r_i - R_{\text{com}}, \quad v_i \leftarrow v_i - V_{\text{com}}.$$

M_{total} is constant; compute once and cache. Cost: ~ 20 FLOPs per step, negligible relative to the force evaluation.

Why COM is special. COM position and total linear momentum are linear constraints generated by exact translation symmetry, dynamically decoupled from internal motion, and removable by a canonical projection with no ambiguity. This is not true for energy or angular momentum—restoring those after drift requires choosing a non-unique state modification. The simulator shall *not* explicitly project energy or angular momentum.

Invariant monitoring. Energy and angular momentum should be tracked as diagnostics. Before the integration loop begins, evaluate and store:

$$E_0 = \sum_i \frac{\|\mathbf{p}_i\|^2}{2m_i} - G \sum_{i<j} \frac{m_i m_j}{\|r_i - r_j\|}, \quad L_{z,0} = \sum_i m_i (x_i v_{y,i} - y_i v_{x,i}).$$

After each step (post-projection), evaluate $E(t)$ and $L_z(t)$ and update running accumulators:

$$\Delta E_{\text{max}} \leftarrow \max(\Delta E_{\text{max}}, |E(t) - E_0|), \quad \Delta L_{z,\text{max}} \leftarrow \max(\Delta L_{z,\text{max}}, |L_z(t) - L_{z,0}|).$$

The energy evaluation reuses cached pairwise distances from the force computation; the marginal cost is the kinetic term (three dot products) plus two comparisons per step.

Per-step vs start-and-end. Accumulating the transient worst-case drift per step (not just evaluating at termination) catches close-encounter spikes where energy conservation temporarily degrades but recovers by the time the escaper is ejected. This is the regime where the Burrau dynamics are most interesting.

0.21 Event detection

0.21.1 Collision

$\min_{i<j} \|\mathbf{r}_i - \mathbf{r}_j\| < r_{\text{coll}}$. Record pair.

0.21.2 Escape

For each candidate escaping body k relative to the other pair (ij) , define the outer Jacobi separation and velocity:

$$\lambda_{(ij),k} = \mathbf{r}_k - \frac{m_i \mathbf{r}_i + m_j \mathbf{r}_j}{m_i + m_j}, \quad \mathbf{v}_\lambda := \dot{\lambda}_{(ij),k}.$$

Define the outer Jacobi reduced mass and momentum:

$$\mu_{\text{out}} = \frac{m_k(m_i + m_j)}{m_i + m_j + m_k} = m_k(m_i + m_j) \quad (\text{since } M = 1), \quad \boxed{\mathbf{p}_{\text{out}} = \mu_{\text{out}} \mathbf{v}_\lambda},$$

and the effective two-body outer energy:

$$E_{\text{out}} = \frac{\|\mathbf{p}_{\text{out}}\|^2}{2\mu_{\text{out}}} - \frac{G m_k(m_i + m_j)}{\|\boldsymbol{\lambda}\|}.$$

Require three gates persistently:

1. Distance gate: $\|\boldsymbol{\lambda}\| > R_{\text{esc}}$.
2. Outward gate: $\boldsymbol{\lambda} \cdot \mathbf{v}_\lambda > 0$.
3. Energy gate: $E_{\text{out}} > 0$.

Maintain a counter c_{esc} per sample. When all three gates hold: $c_{\text{esc}} \leftarrow \min(c_{\text{esc}} + 1, k_{\text{esc}})$. Otherwise: $c_{\text{esc}} \leftarrow \max(c_{\text{esc}} - 1, 0)$. Declare escape when $c_{\text{esc}} \geq k_{\text{esc}}$. Record which body escaped. This linear persistence/decay suppresses single-step glitches without making the detector sticky over long resonant excursions.

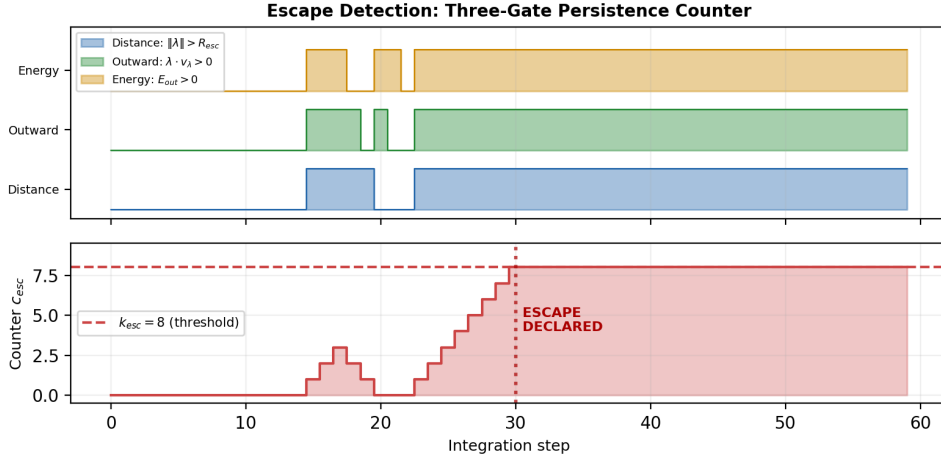


Figure 16: Escape detection persistence counter. Top: the three gates (distance, outward velocity, outer energy) shown as binary signals. Bottom: the counter c_{esc} increments when all three gates hold and decrements when any fails. Escape is declared when the counter reaches k_{esc} .

Diffusion window validity. Frequency diffusion is meaningful mainly for the BOUNDED class. If a sample terminates before the second window, set diffusion to NaN or a sentinel and exclude from diffusion-based coherence statistics.

0.22 Stability metrics

0.22.1 Shape-sphere observable

From Jacobi vectors (mass-weighted for unequal masses), define

$$\mathbf{n}(t) = \frac{1}{\|\tilde{\boldsymbol{\rho}}\|^2 + \|\tilde{\boldsymbol{\lambda}}\|^2} \begin{pmatrix} 2\tilde{\boldsymbol{\rho}} \cdot \tilde{\boldsymbol{\lambda}} \\ 2(\tilde{\boldsymbol{\rho}} \times \tilde{\boldsymbol{\lambda}})_z \\ \|\tilde{\boldsymbol{\rho}}\|^2 - \|\tilde{\boldsymbol{\lambda}}\|^2 \end{pmatrix} \in S^2.$$

Canonical geometric representation. The canonical stored geometric state is the Cartesian unit vector $\mathbf{n}(t) \in S^2$; angle-like quantities (unwrapped phase, frequency diffusion, render-time hue) are *derived observables*, not the primary representation. Storing $\mathbf{n}(t)$ rather than (θ, φ) provides: stable interpolation and averaging with no singular behaviour at the poles; no angle-wrap ambiguity at the antimeridian; cheap comparison via Euclidean distance or dot products ($\|\mathbf{n}_i - \mathbf{n}_j\|^2 = 2(1 - \mathbf{n}_i \cdot \mathbf{n}_j)$); and direct reuse for both adaptive refinement and animated rendering without coordinate conversion.

Given fixed checkpoints $t_1, \dots, t_M$, the canonical sampled path descriptor is $Z_n = (\mathbf{n}(t_1), \dots, \mathbf{n}(t_M)) \in \mathbb{R}^{3M}$.

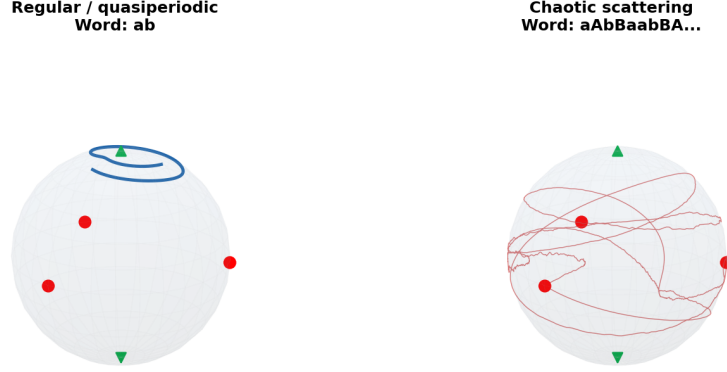


Figure 17: Shape-sphere trajectories for two contrasting initial conditions. Left: a regular/quasiperiodic orbit traces a smooth loop on S^2 (short free-group word). Right: a chaotic scattering trajectory tangles around the collision singularities (long word, many encounters). Red dots: BC collision points. Green triangles: Lagrange poles.

0.22.2 Frequency diffusion

Phase. $\theta(t) = \text{atan2}(n_y, n_x) \in (-\pi, \pi]$.

Unwrapping. Given sampled phases $\theta_k = \theta(t_k)$:

$$\Delta = \theta_k - \theta_{k-1}, \quad \Delta \leftarrow \Delta - 2\pi \text{ if } \Delta > \pi, \quad \Delta \leftarrow \Delta + 2\pi \text{ if } \Delta < -\pi, \quad \boxed{\tilde{\theta}_k = \tilde{\theta}_{k-1} + \Delta.}$$

Windowed frequency estimate. Over window W , fit $\tilde{\theta}(t) \approx \omega t + \theta_0$ by least squares:

$$\boxed{\omega = \frac{\sum_{k \in W} (t_k - \bar{t})(\tilde{\theta}_k - \bar{\tilde{\theta}})}{\sum_{k \in W} (t_k - \bar{t})^2}.}$$

Diffusion. Two windows W_1, W_2 give ω_1, ω_2 :

$$\boxed{D = |\omega_2 - \omega_1|, \quad D_{\text{rel}} = \frac{|\omega_2 - \omega_1|}{|\omega_1| + \varepsilon_\omega}.}$$

Secondary observable. The complex Jacobi ratio $\zeta(t) = (\lambda_x + i\lambda_y)/(\rho_x + i\rho_y)$ provides an independent phase proxy $\theta_\zeta = \arg \zeta$. Large diffusion in one but not the other indicates numerical/branch artefacts rather than genuine chaos.

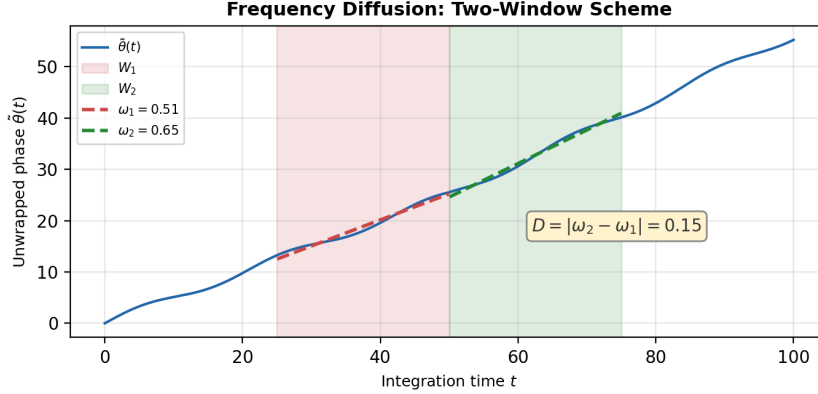


Figure 18: Frequency diffusion two-window scheme. The unwrapped phase $\tilde{\theta}(t)$ is fit by least-squares linear regression in two consecutive windows W_1 and W_2 , yielding frequency estimates ω_1 and ω_2 . The diffusion $D = |\omega_2 - \omega_1|$ measures how much the dominant frequency changed between windows—large D indicates chaotic or transient dynamics.

0.22.3 Finite-time Lyapunov exponent (FTLE)

What FTLE answers. “If I perturb this trajectory by an infinitesimal amount, how fast does that perturbation grow over a finite time?” This is a property of the flow near that trajectory, not of the rendering resolution. It does not depend on screen zoom. It is the dynamical-systems quantity.

Definition. Let $x(t)$ be the base trajectory from x_0 , and let $\delta x(0)$ be an infinitesimal perturbation. The finite-time Lyapunov exponent over horizon T is

$$\lambda_T(x_0) = \frac{1}{T} \log \frac{\|\delta x(T)\|}{\|\delta x(0)\|}.$$

Mass-weighted phase-space norm.

$$\delta^2 = \sum_i m_i \|\mathbf{r}'_i - \mathbf{r}_i\|^2 + \alpha \sum_i \frac{1}{m_i} \|\mathbf{p}'_i - \mathbf{p}_i\|^2,$$

where α is a nondimensionalisation constant (default $\alpha = 1$ when $G = M = I = 1$). The norm choice must be documented; different norms yield different FTLE values.

Benettin renormalisation loop. Without renormalisation, the shadow trajectory leaves the linear regime and you measure nonlinear final separation, not local stretching. The renormalisation loop keeps the perturbation tiny and in the tangent/linearised regime. Every M_{renorm} macro-steps (indexed $j = 1, \dots, J$):

1. Evolve base and shadow trajectories over interval Δt_j .
2. Measure current separation δ_j .
3. Accumulate: $S \leftarrow S + \log(\delta_j/\delta_0)$.

4. Rescale shadow back to distance δ_0 along the current perturbation direction.

The FTLE estimate is $\lambda_T \approx S/T$ where $T = \sum_j \Delta t_j$.

Perturbation hierarchy. For the three-body problem there are three levels of FTLE-like measurement, from most to least faithful:

1. **Full phase-space FTLE:** perturbation in positions *and* momenta. Measures the canonical FTLE of the physical flow.
2. **Reduced IC-manifold FTLE:** perturbation in the 2D chart coordinates only. Measures stretching of the parametrised flow map; useful but not the same dynamical object.
3. **Finite-cloud spread:** no renormalisation, finite neighbourhood. This is ensemble spread (§0.22.4), not FTLE.

All three are useful, but they are not the same object and should not be conflated.

Implementation caveats.

- The renormalisation interval M_{renorm} should be short enough that δ_j/δ_0 stays numerically comfortable (10^{-2} – 10^4) but long enough for the perturbation to align with the locally fastest-growing direction.
- Position-only perturbations are useful diagnostics but should not be labelled as canonical full-state FTLE.
- Nondimensionalisation is strongly recommended.

Compute shader integration. When FTLE is enabled (Tier C / Research mode), each invocation maintains a shadow state alongside the base trajectory: 4 extra position floats + 4 extra momentum floats = 32 bytes per sample in thread-local registers. The Benettin accumulator S and renormalisation counter are thread-local scalars. On completion, $\lambda_T = S/T$ is written into the `stretch` field of `SimResult`. When FTLE is disabled (Preview/Balanced), `stretch` holds a cheaper proxy (e.g. max local deformation or arc-length-based estimate). The quality tier is a dispatch-time uniform, not a per-sample decision.

Interpretation. Low FTLE: nearby trajectories stay close; locally regular or weakly stretching. High FTLE: strong local exponential separation; chaos, transient instability, or proximity to invariant manifolds / basin boundaries. FTLE can still depend on the finite time horizon T , the norm, and transient behaviour.

0.22.4 Finite-resolution ensemble spread

What ensemble spread answers. “If I take a finite patch of nearby ICs at the scale I care about visually, how mixed are their outcomes?” This is about the fate of a finite neighbourhood at the scale you are actually rendering. It is not about the infinitesimal dynamics of the flow in the formal chaos-theory sense.

Why this is useful for Principia. Principia is not only doing dynamics research—it is also rendering a map. For a rendered map, the key practical question is often not “what is the asymptotic local stretching exponent?” but “at this zoom level, is this region visually resolved or not?” A boundary may be mathematically sharp in the continuum, but if the current pixel

footprint spans several filaments, then visually that pixel is genuinely mixed. Ensemble spread tells you that directly.

Definition. For a base point (u, v) with lifted state $x_0 = \chi((u, v))$, sample a finite neighbourhood

$$x_0^{(k)} \in B_r(x_0), \quad k = 1, \dots, N,$$

where r is the local pixel/subpixel footprint radius in the rendered chart. Integrate each sample and compute an observable $F(x_0^{(k)})$.

Continuous spread.

$$\sigma_F^2(x_0) = \frac{1}{N} \sum_{k=1}^N \left\| F(x_0^{(k)}) - \bar{F}(x_0) \right\|^2, \quad \bar{F}(x_0) = \frac{1}{N} \sum_{k=1}^N F(x_0^{(k)}).$$

Categorical impurity. If the observable is a discrete outcome class with empirical probabilities p_i :

$$H(x_0) = - \sum_i p_i \log p_i \quad (\text{entropy}), \quad D(x_0) = 1 - \max_i p_i \quad (\text{majority disagreement}).$$

For basin rendering, outcome entropy is often especially useful.

Why resolution-dependent. The neighbourhood probed has a finite size. Zoom in and the effective perturbation radius shrinks; a region that looked mixed before may separate cleanly into distinct subregions. This is not a bug—it is the whole point. The quantity answers “how stable is this picture at this scale?” rather than “what is the intrinsic local stretching rate?”

Interpretation. Low ensemble spread: the neighbourhood behaves coherently, the pixel is visually stable, probably inside a regular basin interior. High ensemble spread: nearby ICs diverge into distinct outcomes, the pixel straddles a boundary or chaotic mixing structure, the rendered colour is not trustworthy from one sample alone.

Dispatch model. When ensemble is enabled, dispatch E jittered copies per grid position, each producing an identical `SimResult`. Jitter offsets are deterministic (stratified or quasi-random within the pixel footprint). The reduction pass computes spread from all $N^2 \times E$ samples into one `TileReduction`. The `SimResult` struct does not change—you just have more of them. Ensemble size depends on quality tier: $E = 0$ (preview), $E = 2\text{--}4$ on suspicious tiles (balanced), $E = 4\text{--}16$ broadly (research).

0.22.5 Why FTLE and ensemble spread correlate but are not equivalent

Both become large near sensitive regions. At a fractal basin boundary, infinitesimal perturbations separate fast *and* finite neighbourhoods contain multiple outcomes. So visually they often highlight similar structures, which is why they are easy to conflate.

But they can differ:

Ensemble Spread: Finite Pixel Footprint

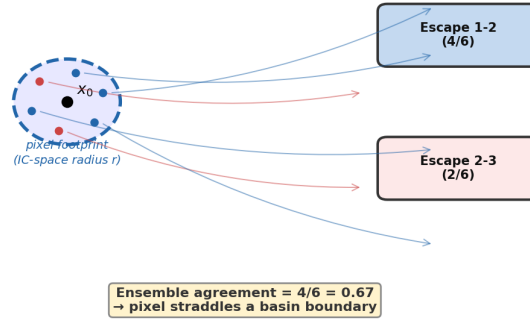


Figure 19: Ensemble spread schematic. The base point x_0 and E jittered copies are scattered within the pixel’s IC-space footprint. Their trajectories diverge to different outcomes (blue: escape 1–2; red: escape 2–3). The ensemble agreement (4/6 here) measures whether the pixel straddles a basin boundary at the current rendering resolution.

- A region may have modest local stretching yet look visually mixed because the pixel footprint crosses many thin filaments.
- A region may have strong local stretching yet not produce mixed final outcomes over the chosen finite integration/classification rule.

If Principia exposes both, users can distinguish:

- “This looks mixed because the rendering footprint crosses multiple basins” (ensemble).
- “This is dynamically unstable in the Lyapunov sense” (FTLE).

Those are different kinds of meaning.

	FTLE	Ensemble spread
Perturbation scale	infinitesimal	finite (pixel footprint)
Renormalisation	required	not used
Resolution-dependent	no	yes (intentionally)
Depends on zoom	no	yes
Primary use	dynamical interpretation	rendering & adaptive sampling
Measures	local exponential stretching rate	fate-mixing of a finite neighbourhood

Pixel-footprint ensembles are simultaneously an anti-aliasing device, an uncertainty estimator, and a refinement signal. This triple role is one of the strongest conceptual unifications in the design.

0.22.6 Trajectory coherence on the shape sphere

The mode-independent refinement basis. Store checkpointed paths $\gamma_i = \{\mathbf{n}_i(t_m)\}_{m=1}^M$. Average chordal mismatch:

$$d_n(\gamma_i, \gamma_j) = \frac{1}{M} \sum_{m=1}^M \|\mathbf{n}_i(t_m) - \mathbf{n}_j(t_m)\|^2 = \frac{2}{M} \sum_{m=1}^M (1 - \mathbf{n}_i(t_m) \cdot \mathbf{n}_j(t_m)).$$

Tile spread: $S_n(C) = \frac{2}{N(N-1)} \sum_{i < j} d_n(\gamma_i, \gamma_j)$.

Compressed descriptor. For GPU efficiency, compare a compact per-sample descriptor:

$$Z_k = (\mathbf{n}(t_1), \dots, \mathbf{n}(t_M), L_n, t_{\text{end}}, d_{\text{min}}, s_{\text{stretch}}),$$

where L_n is total path length on S^2 , t_{end} is termination/escape/collision time, d_{min} is minimum pair separation, and s_{stretch} is an optional FTLE-like accumulator. Tile spread via weighted variance:

$$S_Z(C) = \frac{1}{N} \sum_{k=1}^N \|Z_k - \bar{Z}_C\|_W^2, \quad \bar{Z}_C = \frac{1}{N} \sum_{k=1}^N Z_k,$$

where $\|v\|_W^2 = v^\top W v$ with W diagonal or block-diagonal.

Interpretation. A tile may remain coarse only if nearby initial conditions produce sufficiently similar shape-sphere histories and event statistics that one summary can stand in for the whole tile.

0.22.7 Free-group word of the encounter sequence

What the free-group word is. The planar three-body configuration space, after removing the three binary-collision singularities from the shape sphere S^2 , has fundamental group isomorphic to the *free group on two generators* $F_2 = \langle a, b \rangle$. As the shape-sphere trajectory $\mathbf{n}(t)$ evolves, it traces a path in $S^2 \setminus \{\hat{\mathbf{b}}_1, \hat{\mathbf{b}}_2, \hat{\mathbf{b}}_3\}$. The homotopy class of this path is a word in F_2 : a compressed topological fingerprint of the encounter history.

Two trajectories with the same free-group word have the same encounter topology regardless of their metric details. The word records *which collision singularities the trajectory wound around, in what order, and in which direction*.

Generators. Choose two of the three binary collision points as generators. For equal masses with $\hat{\mathbf{b}}_1 = (1, 0, 0)$, $\hat{\mathbf{b}}_2 = (-\frac{1}{2}, \frac{\sqrt{3}}{2}, 0)$, $\hat{\mathbf{b}}_3 = (-\frac{1}{2}, -\frac{\sqrt{3}}{2}, 0)$:

Symbol	Code	Meaning
a	0b00	Positive loop around $\hat{\mathbf{b}}_1$ (pair 01 close encounter, anticlockwise)
$A = a^{-1}$	0b01	Negative loop around $\hat{\mathbf{b}}_1$ (pair 01, clockwise)
b	0b10	Positive loop around $\hat{\mathbf{b}}_2$ (pair 02 close encounter, anticlockwise)
$B = b^{-1}$	0b11	Negative loop around $\hat{\mathbf{b}}_2$ (pair 02, clockwise)

The third collision point $\hat{\mathbf{b}}_3$ does not need a separate generator: in the free group, a loop around $\hat{\mathbf{b}}_3$ is expressible in terms of a and b via the punctured-sphere relation. This is the topological reason why two generators suffice for three punctures.

Branch-cut construction. Place two branch cuts on the shape sphere:

- Cut C_a : a great-circle arc from $\hat{\mathbf{b}}_1$ to the north pole $\hat{\mathbf{l}}^+ = (0, 0, 1)$.
- Cut C_b : a great-circle arc from $\hat{\mathbf{b}}_2$ to $\hat{\mathbf{l}}^+$.

At each integration step, check whether $\mathbf{n}(t)$ crossed either cut. Crossing detection: for a cut defined by two endpoints $\hat{\mathbf{p}}$ and $\hat{\mathbf{q}}$, compute the signed distance of $\mathbf{n}$ from the great-circle plane containing $\hat{\mathbf{p}}$ and $\hat{\mathbf{q}}$:

$$d(t) = \mathbf{n}(t) \cdot (\hat{\mathbf{p}} \times \hat{\mathbf{q}}).$$

A sign change in d between consecutive steps indicates a crossing. The sign of d at the previous step determines direction (positive $\rightarrow$ negative = symbol a or b ; negative $\rightarrow$ positive = symbol A or B).

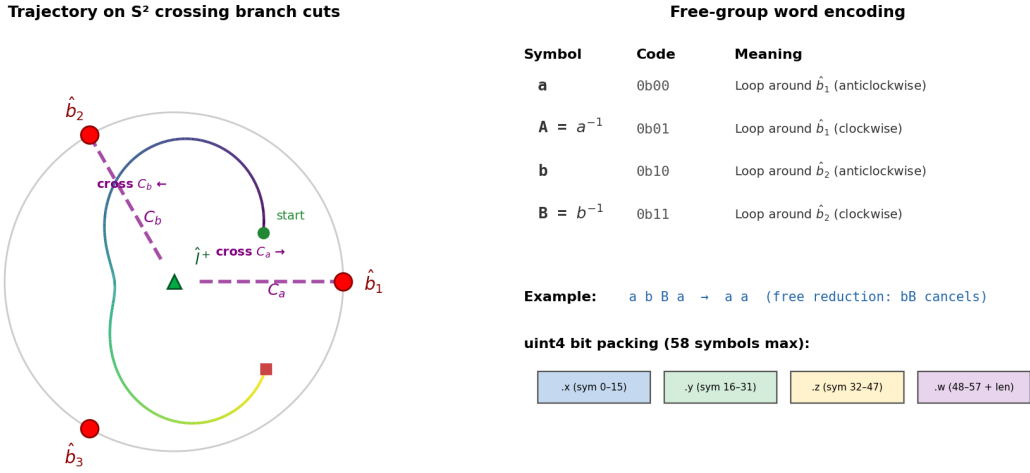


Figure 20: Free-group word encoding. Left: a trajectory on S^2 crossing the branch cuts C_a (from $\hat{b}_1$ to $\hat{l}^+$) and C_b (from $\hat{b}_2$ to $\hat{l}^+$). Each crossing appends a 2-bit symbol. Right: the symbol table, free reduction rule, and `uint4` bit-packing layout (58 symbol slots).

Free reduction. After appending a symbol, check for cancellation with the previous symbol: if the last two symbols are aA , Aa , bB , or Bb , delete both (free reduction). This keeps the stored word in reduced form throughout integration at negligible cost (one comparison per append).

Unequal masses. For unequal masses, the collision points $\hat{\mathbf{b}}_j$ shift from their equal-mass positions on the equator. The branch cuts must be adjusted accordingly: connect each generator's collision point to a reference point (e.g. the Lagrange pole $\hat{\mathbf{l}}^+$) via a great-circle arc that avoids the other collision points. The encoding scheme and crossing detection are otherwise identical.

Compute shader implementation. During integration, maintain a bit-packed word in a `uint4` register (thread-local). Two bits per symbol, packed from the LSB of `.x` upward through `.y`, `.z`, `.w`: this gives 64 symbol slots. Store the current word length ℓ in the top 6 bits of `.w` (leaving 58 usable symbol slots in practice). At each macro-step:

1. Evaluate $d_a(t)$ and $d_b(t)$ for both branch cuts.
2. If sign changed from previous step: determine symbol from cut identity and crossing direction.

3. Append 2-bit code at position 2ℓ in the `uint4`.
4. Free-reduce against previous symbol if cancellation occurs (decrement ℓ).
5. Increment ℓ (clamped to max capacity; set overflow flag if exceeded).

Cost. One dot product per cut per macro-step (two dot products total), one comparison, and one conditional bit-write. Negligible relative to the integration itself.

Bit-packing layout.

```
uint4 free_word;
// .x: symbols 0-15 (bits 0-31, 2 bits each)
// .y: symbols 16-31 (bits 0-31, 2 bits each)
// .z: symbols 32-47 (bits 0-31, 2 bits each)
// .w: symbols 48-57 (bits 0-19), length in bits 26-31
//      bits 20-25 reserved
```

Why this matters. The free-group word is a topological invariant of the encounter sequence. It enables:

- **Symbolic classification:** two trajectories with the same word have the same encounter topology, regardless of metric details.
- **Render mode:** colour pixels by word length, first symbol, periodic structure, or full lexicographic hash.
- **Boundary detection:** word changes between adjacent pixels indicate topological basin boundaries (sharper than metric-based spread).
- **ML applications:** the (word, outcome) pair is the most compressed representation of “what happened and how it ended.” Training on these pairs learns the relationship between encounter topology and final fate—the central scientific question of the three-body problem in its most compact form.
- **Search and clustering:** find all ICs producing a given word; cluster by edit distance on words; identify periodic orbits (words that are powers of a shorter word).
- **Conjugacy and symmetry:** words related by cyclic permutation or generator relabelling correspond to symmetry-related orbits.

0.22.8 Alternative indicators

SALI: tracks alignment of two deviation vectors; chaotic motion drives $\text{SALI} \rightarrow 0$ exponentially.

0–1 test: from observable $\phi(t)$, compute $p_{k+1} = p_k + \phi_k \cos(ck)$, $q_{k+1} = q_k + \phi_k \sin(ck)$; linear growth of MSD indicates chaos.

Future indicators (not yet implemented): event-time sensitivity $S_{t,\nabla} \approx \|\nabla t_{\text{end}}\|$ (bifurcation detector); mean resultant length $R_m = \|(1/N) \sum_k \mathbf{n}_k(t_m)\|$ at each checkpoint (spread on S^2); recurrence compactness $R_\tau = \min_{t'} \|\mathbf{n}(t') - \mathbf{n}(t' + \tau)\|$ (periodic-island detection); time-occupancy entropy on a coarse spherical partition (morphology proxy).

0.22.9 Stability metric hierarchy

Implementation should be guided by cost tier, not by which metric is “best.”

Tier A — default / cheap / always-on. Available in every render mode and tile summary.

Metric	Formula	Use
Shape-path arc length	$L_n = \sum_{m=1}^{M-1} \arccos(\text{clamp}(\mathbf{n}(t_m) \cdot \mathbf{n}(t_{m+1}), -1, 1))$	Activity proxy, tile spread
Min pair separation	$d_{\min}$	Singularity proximity
Event time	t_{end}	Boundary signal
Frequency diffusion	D or D_{rel}	Cheap chaos scalar
Energy drift	$\epsilon_E = E(t_{\text{end}}) - E(0) / (E(0) + \epsilon)$	Numerical reliability

Tier B — medium-cost / selective. Use in balanced mode or for suspicious tiles: tile-local trajectory coherence (main refinement signal); outcome impurity / local entropy; event-time spread $S_t = \text{Var}(t_{\text{end}})$; arc-length spread $S_L = \text{Var}(L_n)$; local direction-spread of $\mathbf{n}(t_m)$ (mean resultant length, worst-checkpoint angular dispersion).

Tier C — expensive / research / spot-check. FTLE with Benettin renormalisation; pixel-footprint ensemble sampling (off in preview, adaptive in balanced, broadly on in research); SALI; 0–1 test; long-horizon recurrence diagnostics.

Key conceptual point. Ensemble spread is not a compromise because FTLE is hard. It measures something genuinely different and more relevant to the renderer: anti-aliasing need, finite-resolution uncertainty, boundary width at the current scale, and whether a tile summary can stand in for its footprint. Pixel-footprint ensembles are simultaneously an anti-aliasing device, an uncertainty estimator, and a refinement signal.

Implementation consequence. FTLE is not used as the primary adaptive-refinement signal in Preview or Balanced modes. Trajectory coherence and ensemble spread drive refinement; FTLE is a Research-tier scientific overlay.

0.22.10 Canonical per-sample summary

The GPU emits one payload per sample serving both rendering and refinement:

Field	Type	Notes
$\mathbf{n}(t_1), \dots, \mathbf{n}(t_M)$	float4×M	.xyz = $\mathbf{n}(t_m)$, .w = $\tilde{\theta}(t_m)$ (unwrapped phase)
free-group word	uint4	2 bits/symbol, 58 slots, length in top 6 bits of .w
arc length L_n on S^2	float	accumulated during integration
t_{end}	float	
$d_{\min}$ (min pair separation)	float	
stretch proxy	float	
energy drift ϵ_E	float	
outcome class	uint	broad category (bounded/escape/collision/...)
event code	uint	sub-detail (which body/pair, degenerate reason)

Why .w = unwrapped phase. The .xyz channels carry the canonical geometric descriptor $\mathbf{n}(t_m) \in S^2$; this is the primary representation and is never compromised. The .w channel occupies the fourth lane already present in the aligned `float4` checkpoint record. The unwrapped phase $\tilde{\theta}(t_m) = \text{atan2}(n_y, n_x)$ (accumulated during integration) enables the fragment shader to compute frequency diffusion via a cheap windowed linear regression without recomputing `atan2` + unwrapping per pixel per frame. Zero additional memory cost. **Contract:** any downstream routine that needs only geometry must ignore .w.

0.22.11 Time-coherent shape representation and animation policy

For animation without re-integration, two representations:

Knot samples (preferred for chaotic trajectories). Store M samples $\mathbf{n}(t_j)$; interpolate and renormalise during rendering. Use when: trajectory is chaotic, event times are sharp, phase is nonuniform or branchy, representative playback must preserve abrupt turns.

Truncated Fourier. $n_\alpha(t) \approx a_0^\alpha + \sum_{k=1}^K a_k^\alpha e^{ik\omega t}$. Store $e^{i\theta}$ not θ to avoid branch cuts. Use when: motion is quasiperiodic/smooth, compact preview is more important than exact faithfulness, tile is being shown as a coarse representative.

0.22.12 Representative sample choice for coarse tiles

When a tile is rendered at coarse resolution, the displayed trajectory must be chosen carefully.

Mean path. Good when the tile is genuinely coherent. Arithmetic mean of $\mathbf{n}$ checkpoints, renormalised back to S^2 .

Medoid path. Choose the sample with minimum average weighted distance to all others:

$$k^* = \arg \min_k \sum_j \|Z_k - Z_j\|_W^2.$$

Better when the tile contains mild multimodality but is still being kept coarse. Avoids nonsense “average trajectories” near actual boundaries.

Dominant-outcome representative. If the tile is mixed, use a medoid restricted to the dominant class for classification rendering, and expose impurity separately.

Playback guard. If tile outcome impurity or checkpoint angular dispersion exceeds threshold, representative playback should be disabled or visually marked as non-representative. Showing one sample’s trajectory in a genuinely mixed tile is misleading.

Part V

Rendering Pipeline

0.23 Render graph architecture

The renderer is structured as a fixed four-stage pipeline rather than a monolithic mode switch. Each stage has a clear contract. Palette changes and overlay toggles never trigger recomputation of the simulation.

Pipeline.

`SimResult` $\xrightarrow[\text{brightness node}]{\text{colour node}}$ `vec3 RGB` $\xrightarrow[\text{float}]{\text{float}}$ `combiner` $\rightarrow$ `postprocess` $\rightarrow$ `screen`

Stage contracts.

Stage	Input	Output	Role
Colour	<code>SimResult</code> , <code>ICDescriptor</code> , <code>FragmentInfo</code>	<code>vec3f</code> (linear RGB)	What hue/colour
Brightness	<code>SimResult</code> , <code>ICDescriptor</code> , <code>FragmentInfo</code>	<code>f32</code> $\in [0, 1]$	How light/dark
Combiner	<code>vec3f</code> + <code>f32</code>	<code>vec3f</code> (linear RGB)	Merge colour + brightness
Post	<code>vec3f</code> + debug overlay	<code>vec3f</code> (final)	CVD, overlays, screen

Fragment context. The fragment shader receives spatial and temporal context alongside the simulation data:

```
struct FragmentInfo {
    tile_uv:    vec2f,    // [0,1]^2 within the tile
    world_uv:   vec2f,    // position in IC-space
    screen_uv:  vec2f,    // position on screen
    time:       f32,      // wall-clock for animations
    playback_tau: f32,    // simulation time for playback
};
```

The sample data comes from a storage buffer read indexed by the tile-local pixel coordinate. No indirection layer or wrapper struct.

Combiner. The combiner is a fixed stage with a mode parameter, not a swappable shader node. Three modes:

1. **Replace lightness** (default): convert colour to OKLAB, replace L with brightness scalar, convert back. Principled and perceptually correct.
2. **Modulate lightness**: $L \leftarrow L \cdot b$.
3. **Multiply RGB**: $\text{rgb} \leftarrow \text{rgb} \cdot b$. Simple but perceptually nonuniform.

Two shader types. Sim shaders read `SimResult` (and optionally `ICDescriptor`) per pixel, producing the actual visualisation. Tile debug shaders read tile-level data (`TileReduction` fields forwarded via a small CPU→GPU buffer) and produce debug overlays composited in the postprocess stage. Both use the same compilation pipeline and shader helper library.

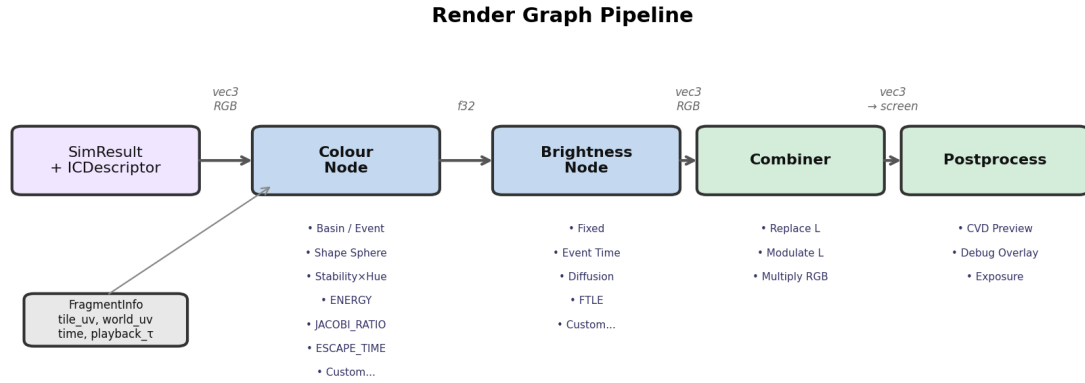


Figure 21: The four-stage render pipeline. `SimResult` and `ICDescriptor` feed into swappable colour and brightness nodes; the combiner merges them in OKLAB space; postprocess handles CVD simulation and debug overlays. Changing palette or render mode never triggers recomputation.

0.24 Shader helper library

A shared WGSL library imported by both built-in and custom shaders.

Colour-space helpers. `srgb_to_linear`, `linear_to_srgb`, `linear_rgb_to_oklab`, `oklab_to_linear_rgb`, `oklab_to_oklch`, `oklch_to_oklab`.

Scalar and transfer-function helpers. `remap`, `smooth_remap`, `log_remap`, `symlog` (linear near zero, logarithmic outside), `percentile_curve`, `transfer_viridis`, `transfer_cubehelix`.

Shape-sphere helpers. `n_at(sim, i)`, `phase_at(sim, i)`, `final_n(sim)`, `binary_collision_proximity(n)`, `lagrange_proximity(n)`.

VMF colour helpers. `vmf_oklab_6pole`, `vmf_okabe_ito`, `lut_sphere`, `direction_cosines_rgb`.

Namespace convention. All library functions are prefixed `principia_` to avoid collisions with custom shader code. The shader compiler detects name collisions before they reach the WGSL compiler.

0.25 Physics visualisation modes

Twenty colour modes organised into four groups, each mapping a per-sample scalar from `SimResult` or `ICDescriptor` to colour via a 1D palette lookup. These slot into the colour node list alongside the existing event classification, shape sphere, and stability modes.

0.25.1 Group 1: Conserved quantities and invariants

Mode	Source	Palette	Notes
ENERGY	E_0	Diverging (centred $E = 0$)	Symmetric log scale. $E < 0$ bound, $E > 0$ unbound.
ANG_MOMENTUM	Lz_0	Diverging (centred $L_z = 0$)	Sign distinguishes prograde/retrograde.
KINETIC	K_0	Sequential	Always ≥ 0 .
POTENTIAL	V_0	Sequential (inverted)	Always ≤ 0 . More negative = deeper well.
VIRIAL	virial_ratio	Diverging (centred at 1)	$= 1$ at virial equilibrium.

Energy colour scale. The energy range across a typical Burrau-family slice spans several orders of magnitude. A symmetric log scale (**symlog**) is recommended: linear within $[-\varepsilon, +\varepsilon]$, logarithmic outside. The zero-energy contour should be visually distinct as it separates bound and unbound configurations.

0.25.2 Group 2: Mass and configuration geometry

Most informative on charts that vary mass or geometry across the slice.

Mode	Source	Palette	Notes
MASS_RATIO_12	m_1/m_2	Diverging (centred 1)	Equal mass pair at centre.
MASS_RATIO_13	m_1/m_3	Diverging (centred 1)	
MASS_FRACTION	q_mass	Sequential	0 = test particle, 1/3 = equal.
JACOBI_RHO1	rho1_mag	Sequential	Inner pair separation.
JACOBI_RHO2	rho2_mag	Sequential	Outer body distance.
JACOBI_RATIO	rho_ratio	Log diverging (centred 1)	> 1 : hierarchical. < 1 : compact.
JACOBI_ANGLE	rho_angle	Cyclic	$[-\pi, \pi]$. Uses cyclic OKLAB palette.
MIN_PAIR_DIST	r_min_pair_0	Sequential (log)	Small = near-collision IC.

Jacobi ratio. The ratio $|\rho_2|/|\rho_1|$ is a scale-free measure of hierarchical structure. Values $\gg 1$ indicate a tight inner pair with a distant third body (perturbation theory regime). Values ~ 1 indicate a compact configuration where all three bodies interact comparably (chaotic regime). This is arguably the most physically informative scalar extractable from the geometry.

Cyclic palette. Since **rho_angle** wraps at $\pm\pi$, the palette must be cyclic. Standard cyclic palettes (twilight, HSL hue) work; a cyclic VMF-OKLAB mode is the natural implementation.

0.25.3 Group 3: Trajectory-derived quantities

Mode	Source	Palette	Notes
ESCAPE_TIME	t_end	Sequential (log)	Long-lived = fractal boundary.
CLOSE_ENCOUNTERS	encounter count	Sequential (integer)	High = chaotic scattering.
MIN_APPROACH	d_min	Sequential (log)	Closest approach over trajectory.

Escape time and fractal dimension. Pixels with long escape times cluster on basin boundaries—the orbits that take longest to “decide” their outcome. Rendering escape time in log scale with a perceptually uniform palette produces images where the fractal structure is visible as luminance variation.

0.25.4 Group 4: Numerical diagnostics

Mode	Source	Palette	Notes
ENERGY_DRIFT_ABS	<code>delta_E_max_abs</code>	Sequential (log)	Worst transient energy drift.
ENERGY_DRIFT_REL	<code>energy_drift</code>	Sequential (log)	Final relative energy drift.
LZ_DRIFT_ABS	<code>delta_Lz_max_abs</code>	Sequential (log)	Worst transient L_z drift.
LZ_DRIFT_REL	<code>Lz_drift</code>	Sequential (log)	Final relative L_z drift.

Diagnostic colour scale. Floor at $\sim 10^{-7}$ (f32 machine epsilon regime). Ceiling at $\sim 10^0$. Regions with drift $> 10^{-2}$ should be visually distinct (warm/hot colours) as numerically suspect trajectories.

0.25.5 Compositing model

All physics visualisation modes operate in two regimes:

Primary mode. The mode replaces the basin colouring entirely. Useful when the physics quantity itself is the object of study.

Overlay mode. The mode is composited over the existing primary render as a semi-transparent layer in the postprocess stage. Alpha derived from scalar magnitude. Compositing uses premultiplied alpha in linear OKLAB space.

The “Diagnostics” group (Group 4) can overlay any primary mode simultaneously. Groups 1–3 are mutually exclusive with the existing primary modes (basin, shape sphere, angular velocity).

0.25.6 GUI colour mode selector

The mode selector organises all colour modes into named groups:

Group	Modes
Basin (existing)	Event classification, Shape sphere, Stability×hue
Invariants	ENERGY, ANG_MOMENTUM, KINETIC, POTENTIAL, VIRIAL
Geometry	MASS_RATIO_*, MASS_FRACTION, JACOBI_*, MIN_PAIR_DIST
Trajectory	ESCAPE_TIME, CLOSE_ENCOUNTERS, MIN_APPROACH
Diagnostics	ENERGY_DRIFT_*, LZ_DRIFT_*

Each group is collapsible in the UI. The “Diagnostics” group is independently toggleable as an overlay.

0.26 Spherical colour map system

The goal is to assign a colour to every point on S^2 satisfying: continuity (no seams), perceptual uniformity, reserved lightness dimension (for stability scalar), and full 2D encoding.

0.26.1 Colour spaces

All work in OKLAB/OKLCH. Conversion from OKLAB (L, a, b) to linear sRGB via the standard M_1, M_2 matrices with cube-root nonlinearity. Polar form: $a = C \cos \theta_h$, $b = C \sin \theta_h$.

0.26.2 Von Mises–Fisher blending engine

The vMF distribution $f(\hat{\mathbf{n}}; \hat{\mathbf{p}}, \kappa) \propto \exp(\kappa \hat{\mathbf{n}} \cdot \hat{\mathbf{p}})$. Place $N = 6$ colour poles at axis intersections $\{\pm\hat{x}, \pm\hat{y}, \pm\hat{z}\}$ with complementary hues. The (a, b) coordinates at query point $\hat{\mathbf{n}}$ are a vMF-weighted mean in OKLAB:

$$a(\hat{\mathbf{n}}) = C \cdot \frac{\sum_i w_i \cos \theta_i}{\sum_i w_i}, \quad b(\hat{\mathbf{n}}) = C \cdot \frac{\sum_i w_i \sin \theta_i}{\sum_i w_i}, \quad w_i = \exp(\kappa \hat{\mathbf{n}} \cdot \hat{\mathbf{p}}_i).$$

Key properties: seamless (no atan2); opposing poles cancel to neutral grey; linear in OKLAB; parameterised by L, C, κ .

Two hue schemes: full OKLAB (red/cyan/green/magenta/blue/yellow) and Okabe–Ito CB-safe (blue/orange/vermillion/sky-blue/bluish-green/reddish-purple).

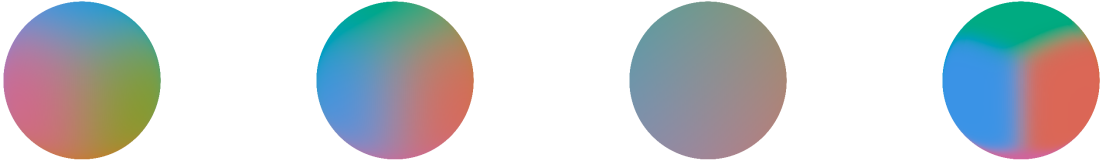


Figure 22: VMF colour maps. Left to right: full OKLAB ($\kappa = 3$); Okabe–Ito ($\kappa = 3$); Okabe–Ito low κ (wide blobs); Okabe–Ito high κ (crisp poles). Opposing poles cancel to neutral grey at midpoints.

0.26.3 Seamless LUT sphere

$N_e = 16$ LUT samples placed as equatorial vMF poles; endpoints at poles. Supports Viridis, Cividis, Plasma, Magma, Inferno, Twilight (cyclic), Cool-warm, Principia palette, Cubehelix.

0.26.4 CVD simulation

Post-process in linear sRGB. Matrix M_{cud} multiplies the linear (R_ℓ, G_ℓ, B_ℓ) triplet:

$$M_{\text{deutan}} = \begin{pmatrix} 0.625 & 0.375 & 0 \\ 0.700 & 0.300 & 0 \\ 0 & 0.300 & 0.700 \end{pmatrix}, \quad M_{\text{protan}} = \begin{pmatrix} 0.567 & 0.433 & 0 \\ 0.558 & 0.442 & 0 \\ 0 & 0.242 & 0.758 \end{pmatrix},$$

$$M_{\text{tritan}} = \begin{pmatrix} 0.950 & 0.050 & 0 \\ 0 & 0.433 & 0.567 \\ 0 & 0.475 & 0.525 \end{pmatrix}, \quad M_{\text{achrom}} = \begin{pmatrix} 0.299 & 0.587 & 0.114 \\ 0.299 & 0.587 & 0.114 \\ 0.299 & 0.587 & 0.114 \end{pmatrix}.$$

Pipeline order: pixel function $\rightarrow$ physics overlay $\rightarrow$ CVD simulation $\rightarrow$ canvas write.

Pole hue angle table. The 6-pole vMF colour map uses the following hue angles θ_i (OKLCH):

Pole	Direction	Full OKLAB θ_i	Okabe–Ito θ_i
$\hat{\mathbf{p}}_1$	$+\hat{x}$	0° (Red)	250° (Blue)
$\hat{\mathbf{p}}_2$	$-\hat{x}$	180° (Cyan)	70° (Orange)
$\hat{\mathbf{p}}_3$	$+\hat{y}$	120° (Green)	30° (Vermillion)
$\hat{\mathbf{p}}_4$	$-\hat{y}$	300° (Magenta)	210° (Sky Blue)
$\hat{\mathbf{p}}_5$	$+\hat{z}$	240° (Blue)	170° (Bluish Green)
$\hat{\mathbf{p}}_6$	$-\hat{z}$	60° (Yellow)	350° (Reddish Purple)

Cubehelix analytical generation. For $t \in [0, 1]$ with start $s = 0.5$, rotations $\lambda = 1.5$, hue $h = 1$:

$$\begin{aligned} \phi &= 2\pi(s/3 - \lambda t), & a &= h \cdot t(1 - t)/2, \\ R &= t + a(-0.14861 \cos \phi + 1.78277 \sin \phi), & G &= t + a(-0.29227 \cos \phi - 0.90649 \sin \phi), \\ B &= t + a(1.97294 \cos \phi). \end{aligned}$$

Seam-free guarantee. The following modes are continuous on S^2 (no seams): all vMF modes ($\exp(\kappa \hat{\mathbf{n}} \cdot \hat{\mathbf{p}})$ is smooth everywhere); seamless LUT sphere; latitude stripes ($\cos(f \arccos n_z)$ is smooth); longitude stripes ($\sin(f \operatorname{atan2}(n_y, n_x))$ is continuous for integer f); Perlin noise (3D Cartesian, no seam by construction). Intentionally discontinuous: octant, Voronoi-6, hemispheres, icosahedral, Fibonacci (hard), checkerboard, Truchet.

0.27 Render modes

0.27.1 State taxonomy

Every pixel has a well-defined state (never dropped): `DEGENERATE` | `COLLISION_TO` | `SIM_FAILED` | `TIMEOUT` | `COLLISION(pair,t)` | `ESCAPE(body,t)` | `BOUNDED(T)`.

0.27.2 Event classification mode

Colour by outcome class (pair/body identity), brightness by time-to-event: $b = \text{clamp}(1 - t_{\text{event}}/T, 0, 1)$.

0.27.3 Stability evolution mode (OKLab/OKLCH)

Hue from shape-sphere position $\mathbf{n}$ via vMF blending with pole-safe colouring (poles $\rightarrow$ dedicated hues, equatorial palette via 1D LUT over longitude φ). Lightness modulated by diffusion:

$L = L_{\min} + (L_{\max} - L_{\min})s(D)$ with $s(D) = \text{clamp}(\log(1 + D/D_0)/\log(1 + D_{\max}/D_0), 0, 1)$, chroma $C = C_0$ (hue-stable). Gamut handling via precomputed in-gamut LUT with chroma bisection.

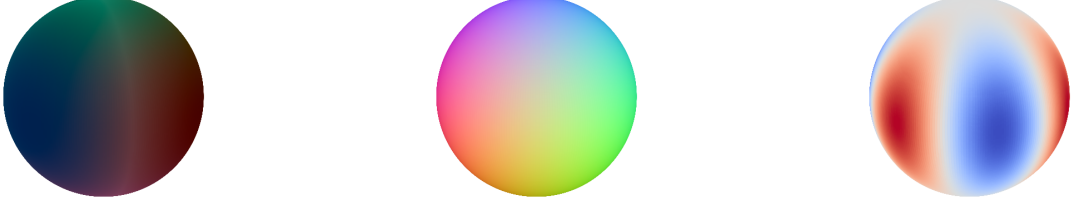


Figure 23: Special colour modes. Left: stability×hue (the principal Principia encoding—hue identifies basin identity, darkness marks proximity to binary collision singularities). Centre: direction cosines (each Cartesian component mapped to an RGB channel). Right: spherical harmonic Y_3^3 (3-fold symmetry of the equal-mass problem).

0.27.4 Shape sphere physics overlay

Special configurations (equal-mass). Three classes of distinguished points on S^2 :

Symbol	Configuration	Location	Count
$\text{BC}_{12}, \text{BC}_{23}, \text{BC}_{31}$	Binary collisions	Equator, 120° apart	3
$\text{E}_1, \text{E}_2, \text{E}_3$	Euler collinear	Equator, between BCs	3
L^+, L^-	Lagrange equilateral	North & south poles	2

Explicit coordinates (equal-mass conventions):

$$\hat{\mathbf{b}}_1 = (1, 0, 0), \quad \hat{\mathbf{b}}_2 = (-\tfrac{1}{2}, \tfrac{\sqrt{3}}{2}, 0), \quad \hat{\mathbf{b}}_3 = (-\tfrac{1}{2}, -\tfrac{\sqrt{3}}{2}, 0),$$

$$\hat{\mathbf{e}}_j = -\hat{\mathbf{b}}_j, \quad \hat{\mathbf{l}}^\pm = (0, 0, \pm 1).$$

Stability × hue (principal Principia encoding). Hue encodes shape-sphere position; lightness encodes proximity to the binary collision singularities:

$$L = 0.25 + 0.55 \cdot \tfrac{1}{2} \left(1 - \max_j \hat{\mathbf{n}} \cdot \hat{\mathbf{b}}_j \right).$$

Points near a collision are dark; antipodal regions are light. This is the principal rendering mode for the Principia fractal: hue identifies basin identity while darkness marks the singularity structure.

Gaussian blob overlay. Each special configuration blends its colour into the pixel via a vMF blob:

$$\mathbf{c}_{\text{out}} = \mathbf{c}_{\text{base}} + \sum_j w_j (\mathbf{c}_j - \mathbf{c}_{\text{base}}), \quad w_j = s \cdot 4 \cdot \max(0, \exp(\kappa_j (\hat{\mathbf{n}} \cdot \hat{\mathbf{p}}_j - 1) + 0.01)),$$

where $s \in [0, 1]$ is user-controlled strength, $\kappa_j = 11$ for BCs and $\kappa_j = 9$ for Euler/Lagrange configurations.

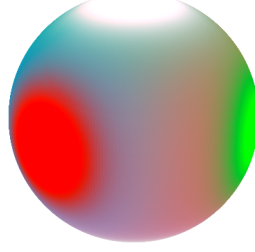


Figure 24: Shape sphere physics overlay on the Okabe–Ito vMF base map. Binary collision points BC_{12} , BC_{23} , BC_{31} visible as coloured blobs at the equator at 120° intervals; Lagrange equilateral configurations mark the poles.

Unequal-mass interpretation. For unequal masses, either retain the equal-mass overlay as a pedagogical frame, or provide an optional transformed overlay consistent with the chosen mass-weighted conventions. Otherwise users may overinterpret equal-mass landmarks in unequal-mass slices.

Physics-aware rendering defaults. Scientific defaults should prioritise: seam-free sphere mapping; monotonic lightness for scalar overlays; colour-blind-safe palette by default; brightness reserved for an independent scalar; stable interpretation of darkness/lightness across zoom levels; no palette change should alter the underlying diagnostic semantics.

0.27.5 Pattern and special modes

Over 30 modes. Key formulae for implementation:

Fibonacci lattice. N near-uniformly distributed points on S^2 . For $i = 0, \dots, N - 1$:

$$n_{z,i} = 1 - \frac{2i}{N-1}, \quad r_i = \sqrt{1 - n_{z,i}^2}, \quad \phi_i = \pi(\sqrt{5} - 1) \cdot i.$$

Golden-angle palette spacing: $\Delta\theta = 2\pi/\varphi_g^{-1}$ where $\varphi_g = (\sqrt{5} - 1)/2$, ensuring adjacent cells always have highly contrasting hues. The dot lattice variant renders points as coloured dots (angular radius $\rho = \cos(1.4/\sqrt{N})$) on a dark background.

Turing-like standing waves. Three plane waves at 120° with the 3-fold symmetry of the equal-mass three-body problem:

$$v = \frac{1}{3} \left[\sin(fn_x) + \sin\left(f\left(\frac{1}{2}n_x + \frac{\sqrt{3}}{2}n_y\right)\right) + \sin\left(f\left(\frac{1}{2}n_x - \frac{\sqrt{3}}{2}n_y\right)\right) \right].$$

Custom N -pole vMF. N poles on a tilted great circle at elevation $\psi = \text{tilt} \cdot \pi/2$:

$$\hat{\mathbf{p}}_i = (\cos(\phi_0 + 2\pi i/N) \cos \psi, \sin(\phi_0 + 2\pi i/N) \cos \psi, \sin \psi).$$

Set $N = 3$, $\psi = 0$ to approximate three-body attractor geometry.

Additional modes. Voronoi-type (octant, Voronoi-6, hemispheres, icosahedral, soft Voronoi), stripe patterns (checkerboard with $\text{even} = \lfloor f\theta/\pi \rfloor + \lfloor f\phi/(2\pi) \rfloor \bmod 2$, latitude stripes $\cos(f \arccos n_z) > 0$, longitude stripes $\sin(f \operatorname{atan2}(n_y, n_x)) > 0$, Truchet mosaic), overlays (grid, iso-hue contours, gradient magnitude, Perlin noise), basin blend ($\text{col} = t \text{pal}[i_1] + (1 - t) \text{pal}[i_2]$ with $t = \sigma(k_s(d_1 - d_2 - 0.04))$), and spherical harmonics Y_ℓ^m .



Figure 25: Pattern modes. Left to right: octant (8 regions by sign); Voronoi-6 (nearest axis pole); checkerboard ($f = 4$, seam-free); vMF OKLAB continuous base map for comparison. All use the Okabe–Ito CB-safe palette where applicable.

Part VI

WebGPU Systems Layer

Top-level architecture requirements.

- R1.** The same simulated data must support both rendering and adaptive refinement.
- R2.** Time-evolution modes must be supported without requiring a second feature pass.
- R3.** The quadtree must remain usable even if the refinement heuristic is imperfect.
- R4.** FTLE, finite-scale ensemble spread, and tile coherence must be kept conceptually separate.
- R5.** User-facing complexity must remain low; the main quality knobs are `SAMPLES_PER_TILE_AXIS` and `MAX_DEPTH`.

0.28 Layered build-up

The quadtree system is built in three layers. Each layer produces a working, testable system. Later layers add capability without changing the contracts established by earlier layers. This is the implementation order, not just a conceptual aid.

0.28.1 Layer 0: flat grid, no cache

The minimum viable system. The viewport covers a region of $[0, 1]^2$ IC space, divided into a fixed grid of tiles. Each tile is $T \times T$ simulations. All tiles are dispatched, each producing T^2 `SimResult` records in a GPU storage buffer. The fragment shader reads the buffer and colours pixels. Panning or zooming recomputes everything.

Data flow.

CPU → GPU: tile uniforms derived from grid position + `SimUniforms`
GPU → GPU: `SimResult[T2]` (fragment shader reads directly)
GPU → CPU: nothing

Three contracts must be defined at this layer: `SimResult` (what the compute shader writes), the decoder (how tile-local UV becomes an IC), and the fragment shader (how `SimResult` becomes colour).

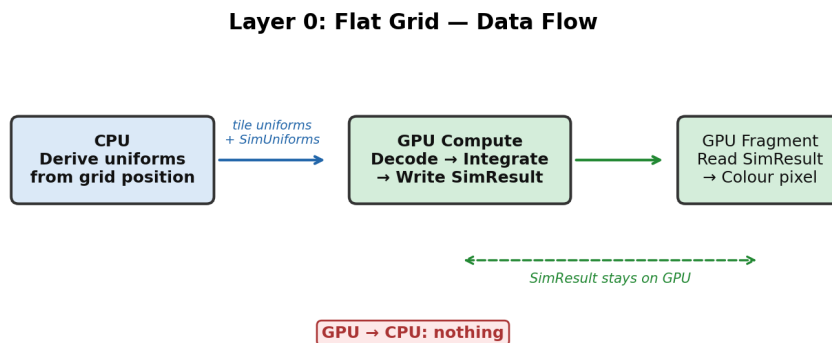


Figure 26: Layer 0 data flow. Tile uniforms flow CPU→GPU; `SimResult` stays entirely on the GPU. No data crosses the GPU→CPU bus.

0.28.2 Layer 1: tile cache and ancestor fallback

The system maintains a cache: a map from (z, t_x, t_y) to a GPU buffer of `SimResults`. When the camera moves, compute which tiles at the target zoom level are visible. For each: if cached, draw it; if not, walk up the tree to the nearest cached ancestor, draw that ancestor stretched over the missing region, and queue the missing tile for compute.

Slippy-map behaviour. Panning never blanks the screen—there is always a cached ancestor to show. Zooming in shows blurry ancestors that sharpen as children finish computing. This is the Google Maps interaction model.

Data flow. Identical to Layer 0. The CPU tracks cache membership (a boolean per tile identity), but no simulation data flows GPU→CPU. The queue is simple FIFO—no priority, no adaptive refinement.

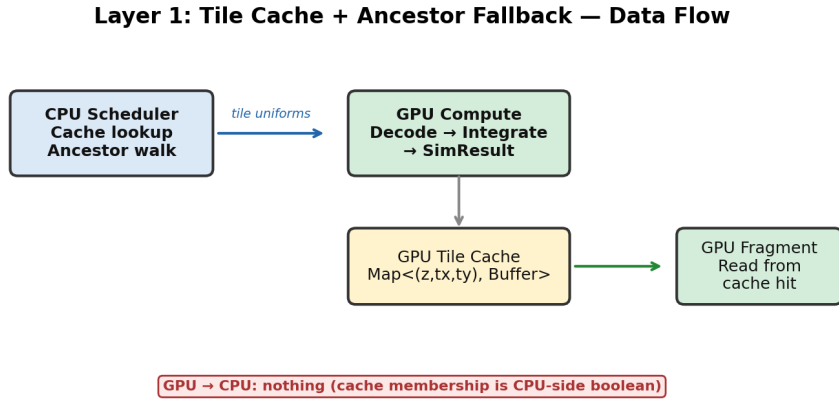


Figure 27: Layer 1 data flow. The tile cache stores `SimResult` buffers keyed by (z, t_x, t_y) . The ancestor fallback walks up the tree to find the nearest cached tile. Still no GPU→CPU data.

0.28.3 Layer 2: adaptive refinement

After a tile’s simulations finish, a GPU reduction pass aggregates the T^2 `SimResults` into a `TileReduction`: outcome histogram, coherence, suspect fraction, worst energy drift, escape time range. The CPU reads this back (~ 80 bytes per tile) and uses it to make split/priority decisions.

Data flow.

CPU → GPU: tile uniforms + optionally linearised decoder uniforms
GPU → GPU: `SimResult` $[T^2]$ + `ICDescriptor` $[T^2]$
GPU → CPU: `TileReduction` (~ 80 bytes per tile)

This is the only layer where data flows GPU→CPU during normal operation. The CPU-side tile object gains state:

```

interface Tile {
  z: number; tx: number; ty: number; // identity
  simBuffer: GPUBuffer | null;        // T×T SimResults
  reduction: TileReduction | null;    // from GPU reduction
  lifecycle: 'unseen'|'queued'|'computing'|'ready'|'readyRefinable';
  priority: number;
  cacheAge: number;
  computeCostMs: number;
  children: [Tile, Tile, Tile, Tile] | null;
  parent: Tile | null;
}

```

World UVs are never stored—derived from (z, t_x, t_y) by arithmetic. The renderer walks the tree to collect visible tiles (with ancestor fallback). The scheduler walks the same tree to decide what to compute next, ordered by priority.

Layer 2: Adaptive Refinement — Data Flow

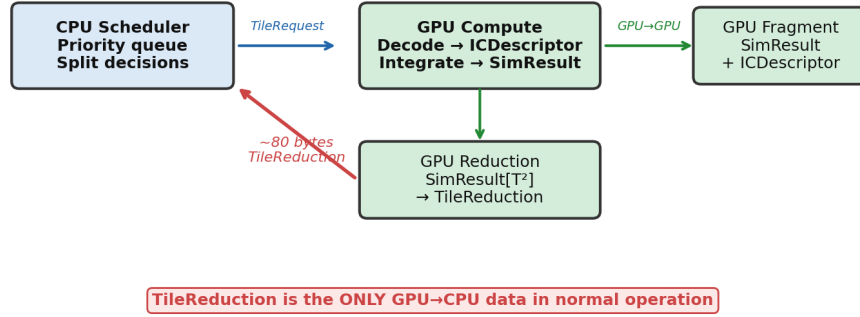


Figure 28: Layer 2 data flow. The GPU reduction pass aggregates `SimResult[T2]` into a `TileReduction` (~80 bytes), which is the *only* data crossing the GPU→CPU bus. The CPU scheduler reads it to make split/priority decisions. The rendering path (GPU→GPU) is unchanged from Layers 0–1.

Tile debug overlays. The scheduler’s internal state (lifecycle, priority, ancestor gap) is useful for debugging. When the debug overlay is active, the CPU packs per-visible-tile debug data into a small buffer (~16 KB for a typical viewport) and the tile debug shader reads it in the postprocess stage. Zero cost when debug overlay is off.

0.29 Tiled quadtree architecture

0.29.1 Tile pyramid

Fix tile resolution $T \times T$ (e.g. $T = 512$). Zoom levels $z = 0, \dots, Z_{\max}$ with $2^z \times 2^z$ tiles at level z . Tile key (z, t_x, t_y) covers:

$$u \in \left[\frac{t_x}{2^z}, \frac{t_x + 1}{2^z} \right], \quad v \in \left[\frac{t_y}{2^z}, \frac{t_y + 1}{2^z} \right].$$

0.29.2 Camera-to-zoom mapping

$$z_{\text{base}} = \text{clamp}(\lfloor \log_2(W/(T\Delta u_{\text{view}})) \rfloor, 0, Z_{\text{max}}).$$

0.29.3 Parent fallback

If tile not ready, render nearest ancestor ($z - 1, \lfloor t_x/2 \rfloor, \lfloor t_y/2 \rfloor$) upscaled. Immediate display with progressive sharpening.

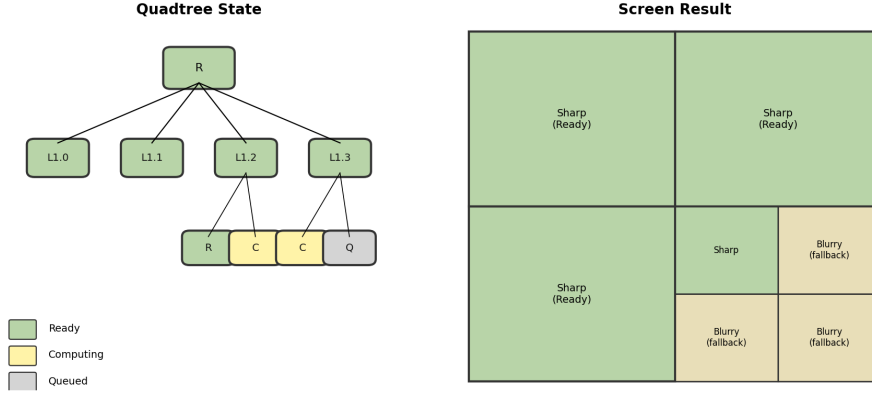


Figure 29: Ancestor fallback in action. Left: quadtree state showing ready (green), computing (yellow), and queued (grey) tiles. Right: the screen result—ready tiles render sharp; missing children are filled by their nearest ready ancestor drawn upscaled (blurry). No region is ever blank.

0.29.4 The quadtree as adaptive scientific sampling

Uniform rendering wastes effort because basin interiors are often visually and dynamically smooth while boundaries are filamentary and information-dense. The quadtree is not just an LOD trick—it is a way to allocate simulation budget according to local dynamical complexity.

Consequences: smooth tiles may remain coarse even at large screen area; boundary tiles may need refinement even when visually small; “resolution” means both screen resolution and dynamical complexity resolution.

Important distinction. This is not a physical spatial tree over bodies in the simulation. It is a *parameter-domain adaptive dispatch tree*. Tiles are patches of IC space, not patches of physical space.

Why CPU-managed. *Reason 1:* hierarchy logic is irregular—tile existence, priority, cache eviction, promotion, and parent-child fallback are branchy, pointer-like, and global. *Reason 2:* the GPU should do dense arithmetic (decode, integrate, reduce), not orchestration. *Reason 3:* screen-space dispatch, memory budgeting, visibility tests, dependency tracking, and cache invalidation are all CPU-natural operations.

0.30 Two-stage tile pipeline

Diagnostics stage (expensive): decode + canonicalise + integrate + events + stability + time-coherence fit. Output: packed diagnostic payload per pixel.

Render stage (cheap): convert diagnostics to RGBA for chosen render mode.

Decouples palette/compactification changes from integration cost.

0.30.1 Per-pixel compute workflow

For each pixel (IC) within a tile, the diagnostics compute shader executes:

1. **Decode IC** from chart coordinates via the current (Φ, D, C) pipeline.
2. **Integrate trajectory** (macro-step loop with adaptive substepping).
3. **Update observables** at each macro-step: shape-sphere vector $\mathbf{n}(t)$, pairwise distances, energy.
4. **Update stability accumulators:** unwrapped phase $\tilde{\theta}_k$ and windowed frequency estimates; optionally FTLE shadow trajectory with Benettin renormalisation.
5. **Detect terminal events:** collision ($r_{\min} < r_{\text{coll}}$), escape (three gates persistent), timeout, NaN/Inf.
6. **Write packed diagnostic payload:** event code + time, diffusion scalar, shape-sphere checkpoints $\mathbf{n}(t_m)$, scalar summaries.

0.30.2 Ensemble sampling (optional)

If ensemble stability is enabled for a tile:

1. Generate N jittered ICs inside each pixel footprint.
2. Integrate each independently through the same workflow.
3. Accumulate outcome histogram and continuous spread per pixel.

Ensemble size may depend on quality tier (preview vs refine).

0.30.3 Rendering from the same canonical data

The same tile summary drives all rendering modes:

Mode	Fields consumed
Static colour / classification	dominant outcome, mean stretch, selected checkpoints
Shape animation	mean or representative $\mathbf{n}(t_m)$ checkpoints
Stability overlay	mean and spread of stretch or scalar metric
Uncertainty overlay	S_{tile} , impurity, or coherence score
Path-morphology view	sampled $\mathbf{n}(t_m)$ plus path length and event metrics

This reuse of a single canonical dataset is the central efficiency win: no separate refinement-only feature pass is required.

0.30.4 Quality tiers

	Preview	Balanced	Research
When	dragging/zooming/first paint	default interactive	publication-grade
Integrator	KDK (2nd)	Yoshida 4th	Yoshida 4th or 6th
Horizon T	reduced	normal	long
Checkpoints M	small (4–8)	standard (8–16)	large (16–32)
Stability	diffusion only	diffusion + coherence	+ FTLE
Ensemble	none or tiny	on suspicious tiles	broadly on
Max depth	lower	medium	deeper
Split threshold	loose	moderate	strict
Payload	minimal	standard	rich
Validation	none	none	optional RK4 cross-check

0.30.5 Full-retention mode

An optional mode that skips the reduction step entirely, keeping every `SimResult` persistently in GPU memory. The quadtree still manages dispatch scheduling, progressive fill, visibility culling, and priority ordering—it just writes full per-sample results to a persistent region of a large storage buffer instead of reducing to `TileReduction`.

Memory cost. For $M = 8$: 176 bytes per sample. At one sample per pixel over a 1024×1024 viewport: ≈ 176 MB. At $M = 16$: ≈ 304 MB. Expensive but feasible on any discrete GPU for a single static frame.

What you gain. Every pixel is backed by its own full checkpointed trajectory. Palette changes, render-mode swaps, and overlay toggles are instant because the fragment shader reads directly from the retained buffer—no recomputation. This is the correct mode for publication-quality export.

Tiled dispatch. Full retention does not mean flat-grid dispatch. Tiles remain the dispatch unit because you cannot submit a million-sample compute job in one pass without hitting WebGPU timeout limits. Each tile (i, j) at level ℓ owns a persistent slice of the output buffer at a known offset. The fragment shader indexes directly into this buffer by screen position.

Adaptive benefit preserved. The quadtree can still split a tile and recompute its children at higher spatial resolution, writing denser samples into the buffer. You keep all of them rather than crushing them into one summary.

Hybrid workflow. The expected usage pattern is: explore interactively using the standard summarised pipeline (≈ 2 MB persistent state, 60 fps), then when you find the view you want, fire a full-retention pass over the current viewport for export. Same compute shader, same fragment shader, same `SimResult` struct. The only difference is whether the reduction pass runs.

	Summarised (default)	Full retention
Persistent state	≈ 2 MB (tile summaries)	≈ 176 – 304 MB (all samples)
Framerate	interactive (60 fps)	compute-once, render-many
Palette swap cost	free (render-only cache)	free (read from buffer)
Quadtree role	dispatch + reduce + render	dispatch + render (no reduce)
Use case	exploration	publication / export

0.30.6 Export modes

Non-interactive batch modes for publication output, video generation, and data export. All configuration is done in the interactive mode first; the export mode executes a batch job with no user interaction during computation. No quadtree, no adaptive refinement, no LOD variation.

Three export modes.

	Static frame	Animated sweep	Live simulation
Purpose	paper plots	videos, GIFs	time-evolution vis
Integration	full trajectory	full trajectory per frame	one step per frame
Output	<code>SimResult</code> [$W \times H$]	sequence of frames	<code>float4</code> buffer
Parameters	fixed	one or more swept	fixed
Interactive	no (batch)	no (batch)	no (batch, play only)

Static frame export

The user specifies: resolution ($W \times H$, e.g. 4096×4096), chart, frozen coordinates, quality tier, integrator, and horizon. The viewport (chart, pan, zoom) is inherited from the interactive mode’s current view. The system partitions the viewport into a regular grid of dispatch chunks and processes them sequentially. Every pixel is integrated at identical settings: same Δt , same horizon, same integrator order, same energy drift threshold. The output is a flat `SimResult` [$W \times H$] buffer.

Guarantees. No tile boundary artefacts. No LOD variation. No coherence-driven refinement bias. Identical integration quality everywhere. A reviewer can verify that every pixel was treated equally. This matters for quantitative figures, statistical claims, and reproducibility.

Workflow. Explore interactively (quadtree mode) $\rightarrow$ find the view $\rightarrow$ set resolution and quality $\rightarrow$ fire export $\rightarrow$ wait (progress bar, may take minutes at 4096^2 Research quality) $\rightarrow$ output as PNG, raw data, or both.

Animated sweep export

Renders a sequence of static frames while varying one or more parameters between frames. Each frame is a complete flat render at the specified resolution. The output is a numbered frame sequence (PNG or raw), convertible to video/GIF.

Sweepable parameters. Any parameter that can be set in the interactive mode:

- **Slice offset:** sweep any Z_k through a range. Animates how basin structure deforms as a single latent dimension varies. The core 8D exploration animation.
- **Tilt:** sweep tilt angle τ from 0 to $\tau_{\max}$. Animates the persistence hypothesis—watch basin boundaries persist or dissolve as the slice rotates into a new dimension.
- **Burrau ν :** sweep triangle shape across the Pythagorean family. The bifurcation animation.
- **Energy / angular momentum:** sweep K or L_z . Watch basins nucleate as momentum increases from the rest-start case.
- **Time horizon T :** sweep integration time. Watch fine structure emerge as trajectories are integrated longer.
- **Render mode:** cycle through render modes at a fixed view. Comparison animation.
- **Zoom:** progressive zoom into a basin boundary. The Mandelbrot-style deep zoom video.

Sweep specification. A sweep is defined by:

```
{
  parameter: "z[3]",           // which parameter to vary
  start: -1.0,                 // starting value
  end: 1.0,                    // ending value
  frames: 120,                 // number of frames
  easing: "linear",            // linear, ease-in-out, logarithmic
  resolution: [2048, 2048],    // per-frame resolution
  quality: "balanced",         // quality tier
  output: "frames/z3_sweep"    // output path prefix
}
```

Multiple parameters can be swept simultaneously (e.g. ν and K together). The sweep executor iterates: for each frame, set all parameters to their interpolated values, run a flat static render, write the frame, advance to the next.

Live simulation export

Each frame dispatches one compute pass that advances every pixel by one integration step Δt . The persistent state is a `float4` buffer (current $\mathbf{n}(t)$ plus phase) and a second buffer for positions and momenta. Trajectories that have terminated hold their final state and stop advancing.

Output. Frame sequence showing a million ICs evolving simultaneously. Close encounters flare as $\mathbf{n}(t)$ swings rapidly. Escape events cause pixels to freeze while neighbours keep moving. Forward-only; the output is a video, not an interactive session.

Data export API

For ML training, statistical analysis, and external tooling, the export modes expose the raw `SimResult` data in structured formats. The API is a set of export functions, not a network service—it runs in the browser and produces downloadable files.

Export formats.

Format	Contents	Use case
PNG	rendered image at current palette/mode	figures, visual inspection
JSON	per-sample: (u, v) , decoded IC, outcome, word, all float fields, all packed fields decoded	small datasets, debugging
Binary (ArrayBuffer)	raw <code>SimResult</code> $[W \times H]$	large datasets, ML pipelines
CSV	per-sample: selected fields as columns	spreadsheets, quick analysis
NPZ	NumPy-compatible arrays	Python/ML workflows

Binary format layout. Header (magic number, version, W , H , M , chart type, frozen Z_0 values, quality tier, integrator, horizon) followed by $W \times H$ contiguous `SimResult` records in row-major order. The header contains all metadata needed to interpret the records and reproduce the computation.

Field decoding. The packed uint fields (`sample_descriptor`, `trajectory_stats`) are exported in two modes:

- **Raw:** uint32 values as-is. Consumer is responsible for bit masking. Compact, fast, no information loss.
- **Decoded:** each bit field extracted to its own named column/field. `outcome_class`, `escape_body`, `collision_pair`, `suspect_energy`, `encounter_count`, `substep_log2`, `benettin_count`, `dominant_pair`, `t_dmin_frac`, `total_steps_log2`, `orbit_count`, `dmin_pair`, `retrograde`. Human-readable, directly loadable into pandas/NumPy.

Batch export for ML. For generating large training corpora (the (x_{IC}, w, y, s) tuples described in the ML programme), the workflow is:

1. Configure chart, quality tier, resolution in the interactive mode.
2. Optionally define a parameter sweep (e.g. sweep ν across 100 Pythagorean triples).
3. Fire batch export. Each frame produces a binary `SimResult` file.
4. Post-processing script (Python) reads the binary files, extracts the training tuple fields, and writes to the ML pipeline's expected format (HDF5, TFRecord, etc.).

At 1024×1024 resolution, one frame produces ~ 176 MB of raw data (~ 1 M samples). A 100-frame sweep produces ~ 100 M training samples in ~ 17 GB. Generation is GPU-bound, not I/O-bound.

Reproducibility metadata. Every export includes the full `ViewState` (see §0.30.6) as a JSON sidecar, plus a SHA-256 hash of the binary output data. This is sufficient to reproduce the exact computation.

Unified view state

The GUI's reactive store, the URL sharing system, the export pipeline, the animated sweep executor, and the reproducibility sidecar all share one serialisation format: the `ViewState`. This is the single source of truth for every parameter that affects computation or rendering.

Schema.

```
ViewState {
  // Chart
  chart_type:      string      // "latent", "lz_e", "lz_k",
                                // "shape_sphere", "mass_simplex",
                                // "burrau", "mixed_axis"
  chart_params:    object      // chart-specific (see below)

  // 8D position
  z0:              float64[8]   // latent offset vector

  // Slice basis
  h_axis:          int          // which Z_k is horizontal axis
  v_axis:          int          // which Z_k is vertical axis
  q1:              float64[8]   // primary basis vector
  q2:              float64[8]   // secondary basis vector
  mag:             float64      // +/- half-width in latent space

  // Orientation
  rotation:        float64      // global slice rotation
  tilt1:           float64      // tilt into dimension d1
  tilt2:           float64      // tilt into dimension d2
  tilt1_target:    int          // which latent dim
  tilt2_target:    int          // which latent dim

  // Viewport
  zoom:            float64
  pan:             float64[2]   // centre offset in UV space

  // Integration
  integrator:      string      // "kdk", "yoshida4", "yoshida6", "rk4"
  horizon:         float64     // T_max
  dt:              float64     // base timestep
  max_steps:       int
  checkpoints:     int         // M

  // Quality
  quality_tier:    string      // "preview", "balanced", "research"
  samples_per_axis: int        // N
  max_depth:       int         // quadtree depth limit
  ensemble_count:  int         // E

  // Rendering
  render_mode:     string      // "diffusion", "outcome", etc.
  colour_palette:  string      // "vmf_oklab", "okabe_ito", etc.
  cvd_mode:        string      // "none", "protanopia", etc.
  physics_overlay: bool

  // Resolution (for export)
  resolution:      int[2]      // [W, H]
```

```

// Metadata
principia_version: string
timestamp:         string      // ISO 8601
}

```

Chart-specific parameters. The `chart_params` object varies by `chart_type`:

- **Shape sphere:** `projection` (equirectangular/equal-area), `pole_buffer` (ε), `hover_streamline` (bool).
- **Burrau:** `nu` ($\nu = n/m$), or equivalently `m`, `n` (integers).
- (L_z, E) : `k_max`, `gamma_k`.
- (L_z, K) : `k_max`.
- **Mixed axis:** `axis1_source`, `axis2_source`, `axis1_warp`, `axis2_warp`.
- **Latent slice:** (no extra parameters).
- **Mass simplex:** (no extra parameters).

Four consumers, one schema.

Consumer	Input	What it does
URL sharing	<code>encode(view_state)</code>	compress to URL, reconstruct on load
Single-frame export	<code>view_state</code>	dispatch one flat render
Animated sweep	<code>view_state + overrides[]</code>	interpolate fields per frame
Reproducibility sidecar	<code>view_state + sha256</code>	verify exact reproduction

Animation timeline. An animated export is a `ViewState` plus a `Timeline`: a set of parameter tracks, each with keyframes that control how that parameter evolves across the output frame sequence.

```

Timeline {
  base:         ViewState      // default state for all fields
  duration_frames: int        // total output frames
  framerate:    int           // fps for video encoding (default 30)
  tracks: [
    {
      path:      string        // ViewState field path
      keyframes: [
        {
          frame:  int          // frame index (0-based)
          value:  any           // value at this keyframe
          easing: string       // interpolation TO the next keyframe
        }
      ]
    }
  ]
}

]

output: {
  format:      string          // "binary", "png", "both"
  path:        string          // output directory
}

```

```

    include_sidecar: bool
  }
}

```

Track evaluation. For each frame f and each track: binary-search the keyframe array for the bracketing pair (k_i, k_{i+1}) where $k_i.\text{frame} \leq f < k_{i+1}.\text{frame}$. Compute the local parameter $t = (f - k_i.\text{frame}) / (k_{i+1}.\text{frame} - k_i.\text{frame})$, apply $k_i.\text{easing}$ to t , and interpolate between $k_i.\text{value}$ and $k_{i+1}.\text{value}$. Before the first keyframe, use the base **ViewState** value. After the last keyframe, hold the last keyframe's value.

Easing functions.

Name	Behaviour
<code>linear</code>	$t' = t$
<code>ease_in</code>	$t' = t^2$
<code>ease_out</code>	$t' = 1 - (1 - t)^2$
<code>ease_in_out</code>	smooth hermite: $t' = 3t^2 - 2t^3$
<code>hold</code>	$t' = 0$ (value stays at k_i until k_{i+1})
<code>log</code>	$t' = \log(1 + 9t) / \log(10)$ (slow start, fast end)
<code>step</code>	$t' = 0$ for $t < 1$, $t' = 1$ at $t = 1$ (instant jump)

What this enables.

- **Simultaneous sweeps with different easings.** Each track has its own keyframes and easing. Z_3 can ease-in while ν is linear while zoom is ease-in-out.
- **Holds.** Two keyframes at the same value with frames apart = parameter stays constant. Linger at a basin boundary.
- **Sequential actions.** Sweep Z_3 over frames 0–60, hold, then sweep ν over frames 80–140, then zoom in over frames 140–200. Tracks overlap in time but their active intervals can be staggered.
- **Variable pacing.** Keyframes close together in value but far apart in frame count = slow, detailed exploration. The opposite = fast transit through uninteresting regions.
- **Discrete transitions.** Render mode switch: keyframe at frame 100 with ‘‘outcome’’, easing `step`, keyframe at frame 101 with ‘‘diffusion’’. Instant cut.
- **Compound animations.** Slice through Z_3 , pause, rotate the tilt angle, pause, zoom into a boundary, all in one export. A full video storyboard in one JSON file.

Backwards compatibility. A simple two-value linear sweep is a degenerate timeline: one track, two keyframes (frame 0 and frame N), linear easing. The old sweep API can be syntactic sugar:

```

// Simple sweep sugar
principia.exportSweep("z0[3]", -1.0, 1.0, 100, "linear")

// ...is equivalent to:
principia.exportTimeline({
  base: currentViewState,
  duration_frames: 100,

```

```

tracks: [{ path: "z0[3]", keyframes: [
  { frame: 0,   value: -1.0, easing: "linear" },
  { frame: 100, value:  1.0 }
]]
})

```

Executor. The same dispatch loop as before, now with keyframe evaluation:

1. Clone the base `ViewState`.
2. For each track, evaluate at the current frame index and write the interpolated value to the cloned state.
3. Run chart validation on the resulting state (§0.17.2).
4. Dispatch the flat render.
5. Write the output frame (PNG, binary, or both).
6. Advance to the next frame.

The per-frame overhead of keyframe evaluation is a handful of binary searches and lerps—negligible compared to the GPU dispatch.

Any `ViewState` field is animatable. Because the timeline system operates on field paths, it automatically supports: latent dimension sweeps (`z0[k]`), tilt animation (`tilt1`), chart parameter variation (`chart_params.nu`), zoom animation (`zoom`), pan animation (`pan[0]`), horizon sweep (`horizon`), render mode cycling (`render_mode`), and any combination thereof. No special-case code for each parameter type. Numeric fields interpolate; string fields use `step` easing (instant switch at keyframe); boolean fields use `step` easing (toggle at keyframe).

Interactive store. The GUI’s reactive store *is* a live `ViewState`. Every slider writes to it. Every dropdown writes to it. The URL export serialises it. The timeline executor iterates over it. Import deserialises into it. One object, one schema, one source of truth.

Implementation. All export modes share the decoder, integrator, and chart map with the interactive pipeline. The difference is orchestration only.

Dispatch structure. “Flat” means flat hierarchy, not no dispatch structure. A 4096×4096 render cannot be submitted as a single compute dispatch (WebGPU timeout limits). Instead, the viewport is partitioned into a regular grid of *dispatch chunks*—fixed-size sub-regions (e.g. 256×256 pixels each) dispatched sequentially in raster order. Each chunk is a self-contained compute pass identical to one tile in the interactive pipeline, but with no parent/child relationship, no adaptive splitting, no coherence scoring. Every chunk is the same size, same level, same settings. The final image is assembled by writing each chunk’s results to the correct offset in the output buffer.

Zoom precision in export. The tile-local precision scheme (§0.31.2) and linearised decoder (§0.31.3) apply *identically* to dispatch chunks. The CPU computes (c_u, h_u, c_v, h_v) for each chunk in `f64`. At deep zoom, chunks are small in UV space and the linearised decoder kicks in per-chunk—each chunk gets its own CPU-precomputed reference IC $\mathbf{x}_0$ and Jacobian J_D . At extreme zoom the entire viewport may fit in a single chunk, in which case one Jacobian covers

the whole image. There is no precision disadvantage compared to the interactive quadtree mode; the precision machinery is per-dispatch-unit, not per-quadtree-node.

What the export modes do *not* have:

- No `TileReduction` computed (no reduction pass).
- No coherence score, no split/merge/keep decisions.
- No priority queue, no progressive refinement.
- No user interaction during computation (progress bar only).
- No LOD variation (every chunk computed at identical settings).

Comparison of all pipeline modes.

	Interactive	Full retention	Export (static/sweep)	Export (live)
Quadtree	yes	yes (no reduce)	no	no
Adaptive	yes	yes	no	no
LOD variation	yes	yes	no (uniform)	no (uniform)
Output	<code>TileReduction</code>	<code>SimResult[]</code>	<code>SimResult[]</code>	<code>float4[]</code>
Interaction	full	full	none (batch)	none (play)
ViewState	live (reactive)	live (reactive)	frozen (per-frame)	frozen
Use case	exploration	analysis	publication / ML	video

0.31 WebGPU execution model

0.31.1 Dispatch

One compute invocation per simulated sample. Do not make threads cooperate on a single trajectory in v1. For a tile with `SAMPLES_PER_TILE_AXIS = N`, dispatch a 2D grid of $N \times N$ invocations. Each invocation maps its local sample coordinate to chart-space UV, decodes the IC, integrates the trajectory, computes `SimResult`, and writes one record to a storage buffer. A second reduction pass per tile produces `TileReduction`.

0.31.2 Deep-zoom precision: tile-local coordinates

At extreme quadtree depth, tiles cover a tiny fraction of the $[0, 1]^2$ UV range. Naïvely computing global UV on the GPU as $u = u_{\min} + (u_{\max} - u_{\min}) \cdot t$ in `f32` loses precision: at depth 30 the tile width is $\sim 10^{-9}$, burning most of `f32`'s 7 significant digits on the offset from the origin.

The fix: CPU-computed tile centre and half-width. The CPU (in `f64`) computes per-tile uniforms:

$$c_u = \frac{u_{\min} + u_{\max}}{2}, \quad h_u = \frac{u_{\max} - u_{\min}}{2},$$

and likewise for v . These are passed to the GPU as `f32` uniforms. The shader computes sample positions as:

$$u(t) = c_u + h_u \cdot (2t - 1), \quad t \in [0, 1],$$

where t is the tile-local sample coordinate (e.g. $(i + 0.5)/N$ for sample i). The shader only ever does arithmetic relative to the tile centre, so it has full **f32** precision across the tile width regardless of absolute depth.

Why this works. h_u is the quantity that matters for *within-tile* resolution. At depth ℓ , $h_u = 2^{-(\ell+1)}$, which is an exact power of two and therefore exactly representable in **f32** at any depth up to the exponent limit ($\sim 2^{-126}$). The absolute position c_u has finite **f32** precision, meaning the tile is slightly misplaced in global coordinates—by less than one tile width—which is invisible in the rendered image.

Precision floor. The ultimate resolution limit is not the coordinate system but the *decoder*. At extreme zoom, two tile-local samples that are adjacent in t may decode to ICs differing by less than **f32** machine epsilon in physical coordinates (masses, positions, momenta). At that point the energy drift diagnostic will show all samples as numerically identical or suspect. The quadtree should stop splitting when this floor is reached. A practical guard: if all N^2 samples in a tile produce bitwise-identical **SimResult** records, the tile is at the **f32** resolution floor and should not be split further regardless of coherence score.

Summary of precision responsibilities.

Component	Precision	Responsibility
CPU (quadtree manager)	f64	tile bounds, centre/half-width computation, scheduling
GPU (per-tile uniforms)	f32	c_u, h_u, c_v, h_v ; exact for half-widths, slight offset for centres
GPU (shader)	f32	tile-local sample positions, IC decode, integration
Locked-pixel inspector	f64 (CPU)	single-trajectory validation at arbitrary precision

0.31.3 Linearised decoder for deep zoom

At shallow zoom the GPU runs the full nonlinear decoder $D(u, v)$ per sample: sigmoid, softmax, trig, canonicalisation. At deep zoom this decoder hits its **f32** precision floor (adjacent samples decode to indistinguishable ICs). The linearised decoder extends the usable zoom range *and* reduces per-sample cost.

Principle. Within a deep-zoom tile, the decoder is smooth and the tile width h is tiny. Taylor-expand around the tile centre:

$$D(c + \delta) \approx D(c) + J_D(c) \cdot \delta, \quad \delta = \begin{pmatrix} h_u(2t_u - 1) \\ h_v(2t_v - 1) \end{pmatrix},$$

with approximation error $O(h^2)$. At depth 30, $h \sim 10^{-9}$, so the error is $\sim 10^{-18}$ —far below any representable precision.

CPU precomputation (f64). For each deep tile:

1. Evaluate the full nonlinear decoder at the tile centre: $\mathbf{x}_0 = D(c_u, c_v)$ in **f64**. This produces the reference IC (masses, positions, momenta) at **f64** precision.

2. Compute the Jacobian $J_D = \partial D / \partial(u, v)$ at the tile centre, either analytically or by central differences in **f64**. This is a matrix mapping UV offsets to physical-coordinate offsets.
3. Pass $\mathbf{x}_0$ (cast to **f32**) and J_D (cast to **f32**) as per-tile uniforms.

$\mathbf{x}_0$ is the best possible **f32** representation of the true centre IC because it was computed in **f64**. J_D represents *rates of change*, which are $O(1)$ and lose no precision in **f32**.

GPU shader (f32). Each sample computes:

$$\mathbf{x}(t_u, t_v) = \mathbf{x}_0 + J_D \cdot \delta.$$

One matrix-vector multiply plus one vector addition. The entire sigmoid/softmax/trig/canonicalisation pipeline is replaced by a few fused multiply-adds. The GPU never calls the nonlinear decoder.

Precision gain. The nonlinear decoder on GPU hits its floor at depth ~ 20 – 23 (**f32** machine epsilon in the decode chain). The linearised path pushes this to depth ~ 50 because the nonlinear precision lives in $\mathbf{x}_0$ (computed at **f64**) and the per-sample correction $J_D \cdot \delta$ is small and well within **f32** range.

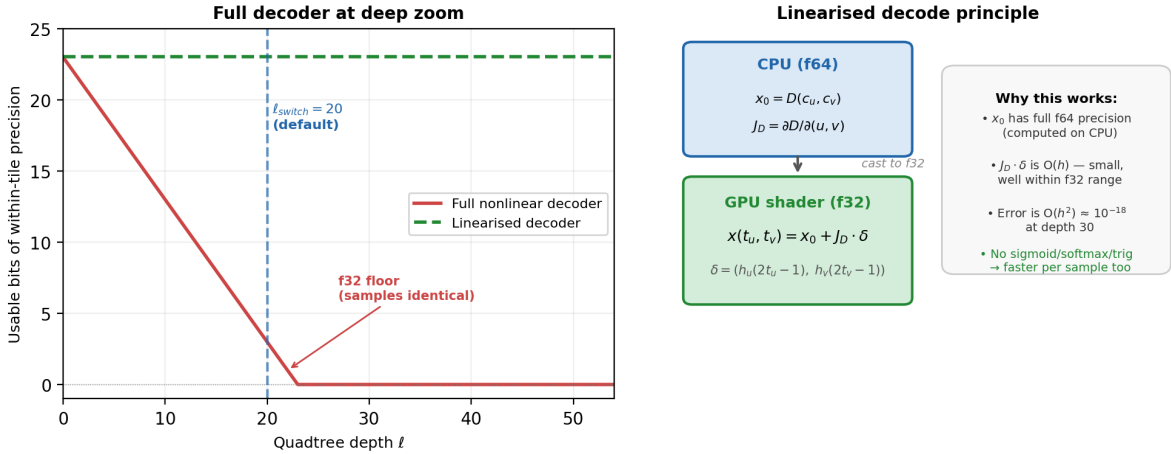


Figure 30: Linearised decoder for deep zoom. Left: usable within-tile precision (in bits) versus quadtree depth. The full nonlinear decoder (red) loses precision as depth increases, hitting the **f32** floor around depth 23. The linearised decoder (green) maintains full **f32** precision at any depth because the reference IC x_0 is computed in **f64** on the CPU and the per-sample correction $J_D \cdot \delta$ is small. Right: the computation split between CPU (**f64**) and GPU (**f32**).

Performance gain. The linearised decode is also *faster* than the full decoder. At deep zoom the quadtree has the most tiles, the most samples, and the highest compute pressure—exactly where the optimisation has the largest effect. Interior tiles that are already uniform-colour are coarsened early and never reach deep zoom; the linearised path targets the expensive boundary tiles.

Switchover. The CPU scheduler selects the decode mode per tile:

- **Depth** $\leq \ell_{\text{switch}}$: full nonlinear decoder on GPU. The tile is wide enough that the linear approximation may not be accurate and the full decoder has adequate precision.

- **Depth** $> \ell_{\text{switch}}$: linearised decoder. The CPU precomputes $\mathbf{x}_0$ and J_D and sets a flag in the `TileRequest`.

Default $\ell_{\text{switch}} = 20$. The transition is seamless: same `SimResult` output, same reduction pass, same rendering. A `DECODE_MODE` flag in `TileRequest.flags` selects the shader path. Alternatively, the switchover can be triggered when the energy drift diagnostic detects adjacent samples producing identical results (adaptive rather than depth-threshold).

Jacobian cost. Two decoder evaluations per tile (central differences in each axis), in `f64` on the CPU. Microseconds per tile. Negligible compared to the GPU integration cost, and amortised over N^2 samples.

0.31.4 Workgroup shape

Recommended: `@workgroup.size(8, 8, 1)`. One workgroup covers an 8×8 patch of samples. For $N = 16$: 4 workgroups per tile. For $N = 32$: 16 workgroups per tile. The 8×8 size behaves well across WebGPU implementations, keeps shared-memory pressure low, and maps cleanly to typical tile sample sizes.

0.31.5 Reduction strategy

Pass 1 writes `SimResult[N2]`. Pass 2 reduces into `TileReduction` using one workgroup per tile if N^2 is modest, otherwise a two-stage block reduction into temporary partials then final reduction.

0.31.6 WGSL constraints

Use `f32` for integration and reductions in `v1`; `f16` is optional and only for compact payload storage. Storage-buffer structs must obey 16-byte alignment. The `float4` layout for checkpoints is mandatory for alignment and simultaneously carries the unwrapped phase in `.w` at zero additional memory cost:

```
// Each checkpoint: .xyz = n(t_m) on S^2, .w = unwrapped phase
// No separate PackedVec3 needed; float4 is the natural unit
```

Avoid assuming large workgroup shared memory. Keep bind-group counts modest by packing per-tile constants into one uniform/storage record. Treat `shader-f16` as a capability upgrade, not a requirement.

0.32 Tile diagnostic payload

Required fields. `event_code` (u8), `event_t` (f16, normalised), `diffusion` (f16).

Optional time-coherence. Truncated Fourier for $u(t) = e^{i\theta(t)}$: store $a_k \in \mathbb{C}$ as pairs of f16, ω as f16; typical $K \in [3, 8]$.

Packing recommendations. Use f16 for event time, diffusion, lightweight scalar overlays. Use f32 for accumulators and reductions. Quantise $\mathbf{n}(t_m)$ only if visual tests show no noticeable damage. Bit-pack status flags: degenerate, collision-at- $t=0$, sim-failed, timeout, cache-valid, contains-ensemble, contains-FTLE, numerically-suspect.

Payload versioning. Every cached payload should carry a compact compatibility signature covering: chart/decode version, integrator config, horizon, checkpoint schedule, enabled metrics, event thresholds, ensemble mode, and payload schema version. This signature is separate from (and stricter than) the tile cache key, because it governs whether a payload can be reused without recomputation.

0.33 Tile-level summary and coherence score

GPU reduces per-sample summaries to a tile payload. Recommended logical structure:

Per-sample summary (GPU output).

```
struct SimResult {
    float4 n_checkpoints[M];    // .xyz =  $\mathbf{n}(t_m)$ , .w = unwrapped phase
    uint4 free_group_word;      // 2 bits/symbol, length in .w bits 26-31
    float arc_length_n;         // total path length on  $S^2$ 
    float t_end;                 // termination time
    float d_min;                 // minimum pair separation
    float ftle;                  // Benettin FTLE when enabled, else 0
    float energy_drift;          //  $|dE|/(|E|+eps)$ , final relative
    float diffusion;             //  $|\omega_2 - \omega_1|$ 
    // --- Invariant monitoring (new, see COM projection) ---
    float delta_E_max_abs;       //  $\max_{\tau} |E(\tau) - E(0)|$ 
    float Lz_drift;              //  $|dL_z|/(|L_z|+eps)$ , final relative
    float delta_Lz_max_abs;      //  $\max_{\tau} |L_z(\tau) - L_z(0)|$ 
    float E_0;                   // initial total energy
    float Lz_0;                  // initial angular momentum
    // --- end new fields ---
    uint sample_descriptor;      // outcome + flags + encounter stats
    uint trajectory_stats;       // timing + cost + winding + pair + dir
};
```

sample_descriptor bit layout.

Bit(s)	Name	Meaning
0–2	<code>class</code>	outcome class: escape (0), bounded (1), collision (2), degenerate (3), timeout (4). 3 bits, 5 values.
3–4	<code>detail</code>	union field keyed by class. Escape: which body (0/1/2). Collision: which pair (01/02/12). Bounded/other: 0.
5	<code>SUSPECT_ENERGY</code>	$ \epsilon_E > \tau_E$.
6	<code>SUSPECT_LZ</code>	$ L_z \text{ drift} > \tau_L$.
7	<code>FTLE_VALID</code>	<code>ftle</code> field contains true Benettin FTLE.
8	<code>WORD_TRUNCATED</code>	free-group word hit 58-symbol limit.
9–15	<code>encounter_count</code>	total close encounters ($r_{ij} < r_{\text{threshold}}$), 0–127, clamped.
16–22	<code>substep_log2</code>	$\lfloor \log_2(\text{max substep count}) \rfloor$. GPU cost proxy.
23–29	<code>benettin_count</code>	Benettin renormalisations performed, 0–127. FTLE confidence proxy.
30–31	<code>dominant_pair</code>	which body pair had the most encounters (01=0, 02=1, 12=2).

`trajectory_stats` **bit layout.**

Bit(s)	Name	Meaning
0–15	<code>t_dmin_frac</code>	$\lfloor 65535 \cdot t_{d_{\min}} / t_{\text{end}} \rfloor$. <i>When</i> the closest approach occurred, as a fraction of integration time. One extra conditional assignment during the argmin tracking already done for $d_{\min}$.
16–22	<code>total_steps_log2</code>	$\lfloor \log_2(\text{total substeps}) \rfloor$. Total integration cost for this sample.
23–28	<code>orbit_count</code>	$\lfloor \tilde{\theta}_M / 2\pi \rfloor$. Number of complete phase windings, 0–63. Derived from unwrapped phase already in <code>.w</code> .
29–30	<code>dmin_pair</code>	which pair achieved $d_{\min}$ (01=0, 02=1, 12=2). Different from <code>dominant_pair</code> : <code>dominant</code> is who had the most encounters, <code>dmin</code> is who had the single closest call.
31	<code>retrograde</code>	sign of $\tilde{\theta}_M$. 0 = prograde (anticlockwise), 1 = retrograde.

The `ftle` field. When `FTLE_VALID` is set in `sample_descriptor` (Research tier, `FTLE_ENABLED` in dispatch flags), this contains the true Benettin FTLE $\lambda_T = S/T$ from the shadow-state integration (§0.22.3). When `FTLE_VALID` is clear, the field is 0.0. No proxies, no conditional interpretation. Render modes that read `ftle` check the bit first.

The diffusion field. $D = |\omega_2 - \omega_1|$ computed from the unwrapped phase values in `n_checkpoints[m].w` using the two-window frequency fit (§0.22.2). When the trajectory terminates before enough checkpoints for two windows, set to NaN. For **ESCAPE** outcomes, diffusion is typically meaningless and should be excluded from tile coherence statistics.

Memory cost. For $M = 8$: 128 (checkpoints) + 16 (free-group word) + 44 (11 floats) + 8 (2 uints) = 196 bytes, padded to 208 for 16-byte alignment. A 16×16 tile: $256 \times 208 = 52$ KB.

SimResult Memory Layout (M=8, 208 bytes)

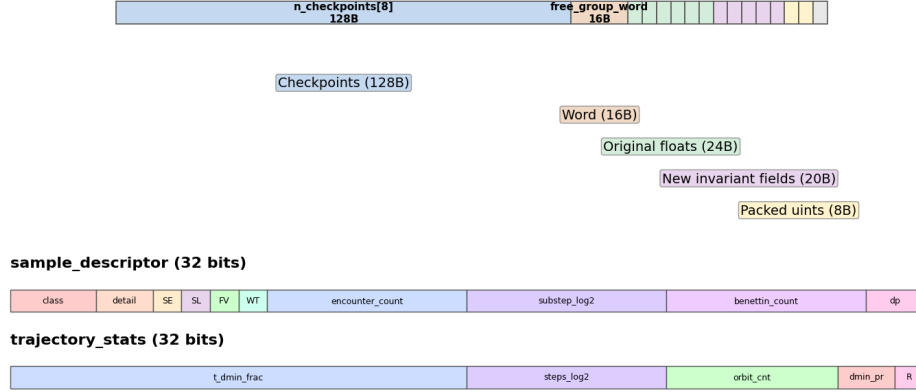


Figure 31: `SimResult` memory layout (208 bytes at $M = 8$). Top: field blocks to scale. The checkpoint array dominates. New invariant monitoring fields (purple) add 20 bytes. Bottom: bit-packing layouts for `sample_descriptor` (32 bits) and `trajectory_stats` (32 bits).

All fields are either free or near-free to compute. Every field in `SimResult` is derived from data the integration loop already holds in registers. The float fields require at most one comparison or accumulation per substep. The packed uint fields require at most one conditional assignment per substep on values already being tracked. No extra memory reads, no extra force evaluations, no branching beyond what the integrator already performs.

IC descriptor (decode-time physics quantities). The `SimResult` struct captures integration output—what the trajectory *did*. Properties of the initial condition itself (masses, geometry, energetics) are upstream of integration. These are stored in a separate companion struct written once per sample after decode, before integration begins, into a second storage buffer alongside the `SimResult` buffer. The fragment shader binds both.

```
struct ICDescriptor {
    float  m1, m2, m3;           // masses
    float  q_mass;               // m_min / M_total
    float  rho1_mag;             // |rho_1| (inner Jacobi)
    float  rho2_mag;             // |rho_2| (outer Jacobi)
    float  rho_ratio;            // |rho_2| / |rho_1|
    float  rho_angle;            // signed angle between rho_1, rho_2
    float  K_0;                  // kinetic energy at t=0
    float  V_0;                  // gravitational potential at t=0
    float  virial_ratio;         // 2*K_0 / |V_0|
    float  r_min_pair_0;         // min_{i<j} |r_i - r_j| at t=0
};
```

52 bytes per sample, padded to 64 for 16-byte alignment. At $M = 8$, the `ICDescriptor` buffer adds ~ 16 KB per 16×16 tile. Written once (not updated during integration), so zero per-step cost.

Jacobi vector convention. The Jacobi decomposition pairs bodies 1–2 as the inner pair and body 3 as the outer, consistent with the atlas decoder. The Jacobi vectors are reconstructed from Cartesian positions: $\rho_1 = r_1 - r_0$, $\text{com}_{01} = (m_0 r_0 + m_1 r_1)/(m_0 + m_1)$, $\rho_2 = r_2 - \text{com}_{01}$. The virial ratio uses $\max(10^{-30}, |V_0|)$ in the denominator to handle rest configurations where $K_0 = 0$.

Global simulation uniforms. Parameters that are constant across all tile dispatches within a frame. Bound once per frame (or per quality-tier change), not per tile.

```
struct SimUniforms {
    float  m[3];                // masses (from decoder)
    float  M_total;             // sum of masses
    float  G;                   // gravitational constant (1.0)
    float  dt_macro;            // base timestep
    uint   N_max;               // max substeps
    float  r_sub, gamma_sub;     // substep trigger
    float  T_horizon;           // integration horizon
    float  r_coll;              // collision radius
    float  R_esc;               // escape radius
    uint   k_esc;               // escape persistence
    float  eps_E;               // energy drift stabilising floor (1e-6)
    float  eps_L;               // Lz drift stabilising floor (1e-6)
    float  r_close;             // close encounter threshold (0.01)
    uint   quality_tier;        // preview/balanced/research
    uint   checkpoint_count;    // M
};
```

The `eps_E` and `eps_L` fields prevent division by near-zero when computing relative drift for parabolic energy or zero angular momentum rest configurations. `r_close` is the threshold for close-encounter counting in `sample_descriptor` bits 9–15.

Data flow architecture. Three data paths, with clear directions of travel:

Path	Direction	Content
Tile dispatch	CPU $\rightarrow$ GPU	per-tile uniforms from $(z, t_x, t_y) + \text{SimUniforms}$
Simulation output	GPU $\rightarrow$ GPU	<code>SimResult</code> [T^2] + <code>ICDescriptor</code> [T^2]
Tile reduction	GPU $\rightarrow$ CPU	<code>TileReduction</code> (~ 80 bytes per tile)

`SimResult` and `ICDescriptor` never leave the GPU during normal operation. The fragment shader reads them directly from storage buffers. `TileReduction` is the *only* data that crosses the GPU $\rightarrow$ CPU bus during normal operation, and only in the adaptive refinement layer. At Layer 0 (flat grid) and Layer 1 (tile cache + ancestor fallback), no data flows GPU $\rightarrow$ CPU at all.

Tile-level reduction (GPU → CPU).

```
struct TileReduction {
    TileID id;
    int    level;

    // Mean checkpoint trajectory (for representative rendering)
    float4 mean_n_checkpoints[M];

    // Per-float-field means
    float  mean_arc_length_n;
    float  mean_t_end;
    float  mean_d_min;
    float  mean_ftle;
    float  mean_energy_drift;
    float  mean_diffusion;

    // Per-float-field spreads (std dev or max-min)
    float  spread_n;           // max angular deviation from mean
    float  spread_arc_length_n;
    float  spread_t_end;
    float  spread_d_min;
    float  spread_ftle;
    float  spread_energy_drift;
    float  spread_diffusion;

    // Classification (reduced from sample_descriptor bits)
    float  outcome_impurity;    // 1 - max(class_fraction)
    uint   dominant_outcome;    // most common class|detail
    float  suspect_fraction;    // fraction with SUSPECT_ENERGY
    float  suspect_lz_fraction; // fraction with SUSPECT_LZ

    // Invariant drift (reduced from SimResult fields)
    float  energy_drift_worst;  // max |delta_E_max_abs| over tile
    float  lz_drift_worst;      // max |delta_Lz_max_abs| over tile

    // Free-group word (reduced)
    float  mean_word_length;
    float  spread_word_length;
    float  word_agreement;      // fraction sharing dominant word
    uint   dominant_word_hash;

    // Ensemble (valid when ensemble_count > 0)
    float  ensemble_outcome_agreement;
    int    ensemble_count;

    // Reduced from trajectory_stats bits
    float  mean_orbit_count;
    float  retrograde_fraction;

    // Composite scores
```

```

float  coherence_score;
float  priority_score;

// Metadata
int     sample_count;
uint    status_flags;
};

```

Tile-level status_flags.

Bit(s)	Name	Meaning
0	HAS_ENSEMBLE	ensemble dispatch was used
1	DECODE_LINEAR	linearised decoder was used (§0.31.3)
2	AT_F32_FLOOR	all samples bitwise-identical (§0.31.2)
3	SUSPECT_MAJORITY	>50% of samples have SUSPECT ENERGY
4	FTLE_VALID	all <code>ftle</code> values are true Benettin FTLE
5	PLAYBACK_VALID	representative trajectory playback is meaningful
6–7	reserved	

Reduction of float fields. The reduction pass iterates over N^2 samples (or $N^2 \times E$ when ensemble is enabled) to compute means and spreads. Adding `diffusion` and `ftle` to the existing mean/spread loop is two extra additions and two extra max-deviation checks per sample. Negligible cost.

Reduction of packed uint fields. `suspect_fraction`: mask bit 5 of each `sample_descriptor`, count, divide by N^2 . `retrograde_fraction`: mask bit 31 of each `trajectory_stats`, count, divide. `mean_orbit_count`: extract bits 23–28 of each `trajectory_stats`, sum, divide. All are mask-and-increment operations with no branching.

Ensemble reduction. When ensemble is enabled ($E > 1$), the dispatch produces $N^2 \times E$ `SimResult` records. The reduction pass groups them by grid position (each group of E records shares the same nominal IC, differing only by sub-pixel jitter). Per group: compute outcome agreement (do all E copies agree on outcome class? — mask bits 0–4 of `sample_descriptor`). These per-group agreement fractions are then averaged across the N^2 grid positions to produce the tile-level `ensemble_outcome_agreement`.

Composite coherence score. Concrete first-pass weights:

$$S_{\text{tile}} = 1.0 S_n + 0.20 S_L + 0.25 S_t + 0.15 S_d + 0.20 S_f + 0.25 S_D + 0.35 S_o + 0.30 S_E + 0.15 S_{L_z} + 0.25 S_{\text{ens}},$$

S_n : checkpoint spread. S_L : arc-length spread. S_t : event-time spread. S_d : $d_{\min}$ spread. S_f : FTLE spread (zero when FTLE_VALID is not set). S_D : diffusion spread (zero for tiles where most samples are escapes). S_o : outcome impurity. S_E : numerical unreliability (`suspect_fraction`). S_{L_z} : angular-momentum unreliability (`suspect_lz_fraction`). S_{ens} : ensemble disagreement ($1 - \text{ensemble_outcome_agreement}$; zero when ensemble is disabled). Shape trajectory remains dominant. These weights are starting values, not final constants; expose them in research mode.

Initial split thresholds. Split if $S_{\text{tile}} > \tau(\ell)$ with $\tau(\ell) = 0.20 + 0.02 \ell$. Force split if outcome impurity > 0.10 . Force split if `suspect_fraction` > 0.05 , unless at `MAX_DEPTH`. Do not split if `AT_F32_FLOOR` is set in `status_flags` (overrides all other criteria).

Design principle. The same tile summary used to decide whether more detail is needed is also sufficient to render the tile at the current level of detail. This eliminates duplicated feature computation and keeps the architecture mode-decoupled.

0.33.1 Explicit split / keep / merge criteria

Split a tile if any of: outcome impurity $> \tau_{\text{imp}}$; trajectory spread $S_n > \tau_n$; event-time spread $S_t > \tau_t$; arc-length spread $S_L > \tau_L$; FTLE spread $S_f > \tau_f$ (when `FTLE_VALID`); diffusion spread $S_D > \tau_D$; `suspect_fraction` $> \tau_E$ and tile is scientifically important; `ensemble_outcome_agreement` $< \tau_{\text{ens}}$ (sub-pixel sensitivity); tile is visible, below target screen-space resolution, and parent summary is insufficient; parent-child disagreement persists across refreshes.

Priority tiebreakers. When multiple tiles qualify for splitting, prefer: tiles with `retrograde_fraction` near 0.5 (prograde/retrograde boundary); tiles with high `mean_orbit_count` spread (dynamically rich region); tiles near the current locked pixel.

Stop splitting if: `AT_F32_FLOOR` is set (all samples identical); tile is at `MAX_DEPTH`; tile is offscreen.

Keep a tile coarse if all of: dominant outcome purity high; all spreads below thresholds; representative summary visually stable under current render mode; tile already finer than screen demand; no special high-interest override.

Merge or deprioritise if: offscreen or nearly offscreen; smoothly varying and overresolved; visually indistinguishable from ancestor summary; cache pressure exceeds budget.

0.34 Scheduling and progressive refinement

0.34.1 Why CPU-managed

The natural division of labour: GPU performs tile-local simulation, per-sample feature extraction, and local reduction to tile summaries; CPU owns the hierarchy, priority queue, caching, visibility logic, and dispatch scheduling. This is preferred because the hierarchy is orchestration logic—determining which tiles should exist, which should be refreshed, and which should be dispatched next under a finite budget.

0.34.2 Tile request

```
struct TileRequest {
    TileID id;
    int    level;
```

Split / Keep / Merge Decision Logic

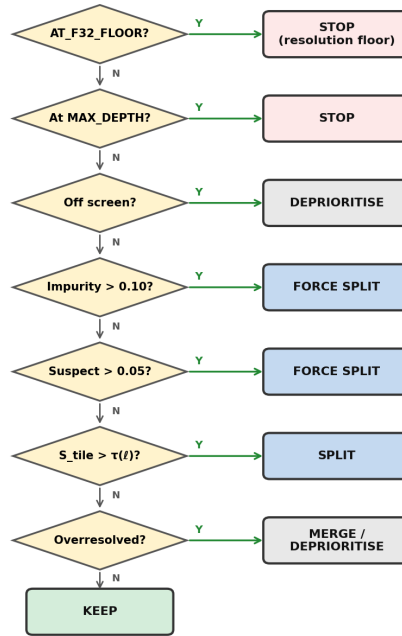


Figure 32: Split/keep/merge decision logic as a flowchart. Guard conditions (AT_F32_FLOOR, MAX_DEPTH, off-screen) are checked first. Then force-split conditions (impurity, suspect fraction). Then the composite score threshold. Finally, overresolved tiles are merged. The default is keep.

```

float2 uv_min, uv_max;          // legacy bounds (CPU-side reference)
float2 uv_centre, uv_half;      // precomputed for deep-zoom precision
int    sample_pattern_id;
int    time_checkpoint_count;
int    max_steps;
int    ensemble_count;          // E (0 or 1 = no ensemble)
float  t_max;
uint   flags;
};

```

flags field. Bit-packed dispatch options:

Bit(s)	Name	Meaning
0	DECODE_LINEAR	use linearised decoder (§0.31.3)
1	ENSEMBLE_ENABLED	dispatch E jittered copies per grid position
2	FTLE_ENABLED	compute full Benettin FTLE (Research tier)
3	PREVIEW_MODE	reduced horizon, coarse integration
4	FULL_RETENTION	skip reduction, keep all SimResult (§0.30.5)
5	COMPUTE_IC_DESCRIPTOR	write ICDescriptor buffer alongside SimResult
6–7	reserved	

Linearised decoder uniforms. When DECODE_LINEAR is set, the GPU needs the CPU-precomputed reference IC $\mathbf{x}_0$ and Jacobian J_D (§0.31.3). These are large ($\sim 15 + 30$ floats)

Tile Lifecycle State Machine

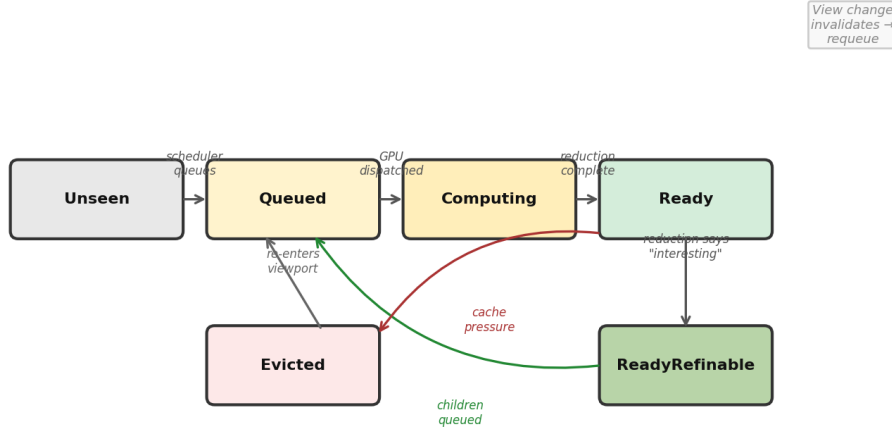


Figure 33: Tile lifecycle state machine. Transitions are labelled by their triggers. The **ReadyRefinable** state distinguishes “done enough to display” from “done forever”—the scheduler can revisit refinable tiles if budget allows. Eviction is weighted by recomputation cost (expensive tiles resist eviction).

and unused at shallow zoom, so they are passed in a **separate uniform buffer** bound only when `DECODE.LINEAR` is set, not embedded in `TileRequest`. This avoids bloating every tile request at shallow depths.

The `uv_centre` and `uv_half` fields are computed by the CPU in `f64` and passed as `f32` uniforms. The GPU shader computes sample positions as $u = \text{uv_centre.x} + \text{uv_half.x} \cdot (2t - 1)$ (see §0.31.2), never from `uv_min/uv_max` directly. The legacy bounds are retained for CPU-side scheduling, visibility culling, and debugging.

0.34.3 Priority

$$P_{\text{tile}} = w_v P_{\text{visible}} + w_z P_{\text{zoom}} + w_c P_{\text{complexity}} + w_f P_{\text{focus}}.$$

Term definitions.

Term	Definition	Rationale
P_{visible}	1 if tile intersects viewport, 0 otherwise	Never compute off-screen.
P_{zoom}	$\text{clamp}(1 - z - z_{\text{base}} /z_{\text{base}}, 0, 1)$	Tiles near desired zoom score highest.
$P_{\text{complexity}}$	$1 - \text{coherence from TileReduction}$	Incoherent tiles need splitting.
P_{focus}	inverse distance from tile centre to viewport centre	Prioritise where user is looking.

Default weights. $w_v = 10$, $w_z = 2$, $w_c = 3$, $w_f = 1$. Visibility dominates (never compute off-screen); complexity drives adaptive refinement; zoom match is a tiebreaker; focus is subtle. Expose all weights in research mode.

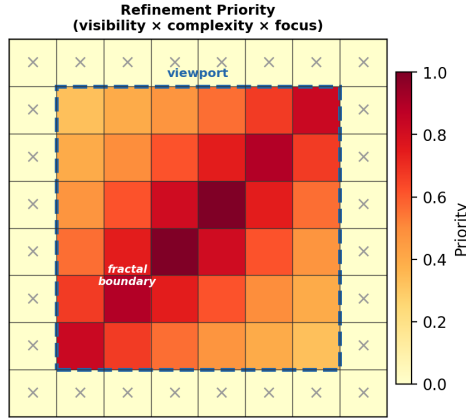


Figure 34: Refinement priority over a toy 8×8 tile grid. Hot colours: high priority (visible, near viewport centre, on a fractal boundary). Grey crosses: off-screen tiles (priority zero). The dashed rectangle is the viewport boundary. Priority concentrates compute budget at the intersection of visibility, focus, and dynamical complexity.

Preview tiles during interaction (reduced horizon). **Refine tiles** on idle (full quality). Limit in-flight jobs (2–4); epoch counter discards stale results.

0.34.4 Memory strategy and eviction

Use full per-sample `SimResult` buffers persistently in GPU memory for cached tiles. Reduce to `TileReduction` for CPU readback. Evict tiles under memory pressure using LRU weighted by recomputation cost: a tile that took a long time to compute (high `computeCostMs`) is expensive to evict because regenerating it hurts. Conversely, a tile with high coherence (uniform basin interior) is cheap to recompute and safe to evict first.

Cancellation. When the user pans away, tiles that are `computing` but no longer visible should stop. WebGPU dispatches cannot be cancelled mid-flight, but if a tile requires multiple dispatch passes (multi-pass integration, progressive refinement), the scheduler should skip subsequent passes for tiles that are no longer visible.

0.34.5 Per-frame update loop

```
while (running) {
    update_camera_and_view();
    visible_tiles = quadtree.collect_visible_frontier(view);
    candidate_tiles = ensure_baseline_tiles(visible_tiles, min_lod);

    for (tile in visible_tiles) {
        if (tile.is_leaf && tile.has_summary)
            candidate_tiles.push(tile);
    }
    for (tile in candidate_tiles)
        tile.priority_score = compute_priority(tile, view);

    jobs = top_k(candidate_tiles, frame_budget);
```

```

dispatch_gpu_tile_jobs(jobs);

completed = collect_completed_gpu_jobs();
for (result in completed) {
    tile = quadtree.get_node(result.id);
    tile.summary = result.summary;
    tile.has_summary = true;
    tile.is_leaf = true;
    if (should_split(tile)) {
        quadtree.split(tile);
        request_children(tile.children);
    }
}
render_tiles(quadtree.current_leaf_cover(view));
}

```

0.34.6 Split policy

$$\text{split}(C) \iff S_{\text{tile}} > \tau(\ell) \wedge \ell < \text{MAX_DEPTH}.$$

Interpreted as “make children eligible and high priority” rather than a rigid binary. $\tau(\ell)$ may vary weakly with level.

0.34.7 Controls

Three conceptually distinct quantities:

SAMPLES_PER_TILE_AXIS: number of simulated IC samples per tile axis, so total samples per tile $N_{\text{tile}} = \text{SAMPLES_PER_TILE_AXIS}^2$.

MAX_DEPTH: maximum quadtree subdivision depth.

TILE_PIXEL_RES (advanced): raster resolution of each tile texture (e.g. 256 or 512). A tile may be rendered at TILE_PIXEL_RES^2 pixels while only simulating N_{tile} samples, with interpolation/parent-fallback bridging the gap.

Interpretation. **SAMPLES_PER_TILE_AXIS** controls estimator quality and the reliability of tile summaries. A low value risks misclassifying a tile as coherent because it undersampled itself. **MAX_DEPTH** controls the smallest representable spatial scale in chart space. **TILE_PIXEL_RES** controls final visual sharpness. These are coupled in practice but conceptually distinct.

Suggested defaults.

Setting	Interactive	Balanced	Offline
SAMPLES_PER_TILE_AXIS	8	16	32
MAX_DEPTH	8	10	12–14
TILE_PIXEL_RES	256	256–512	512
Checkpoints M	8	16	32

0.35 Validation and failure modes

Integrator validation. Bounded-orbit energy drift histograms. KDK vs RK4 comparison on selected short horizons. Substep saturation statistics (how often $N_{\text{sub}} = N_{\text{max}}$).

Event validation. Collision detection against known near-collision cases. Escape-gate persistence sanity checks. Sensitivity of classification to R_{esc} and k_{esc} .

Metric validation. FTLE reproducibility against perturbation size and renormalisation interval. Diffusion robustness to checkpoint schedule. Ensemble spread stability under different jitter patterns.

Refinement validation. Convergence under increasing tree depth. Convergence under increasing ensemble size. Ancestor-fallback visual continuity. Split-threshold sensitivity.

Architectural validation checklist.

1. Verify $\mathbf{n}(t)$ storage and interpolation are stable near poles and wrap regions.
2. Verify tiles marked coherent remain visually and dynamically coherent when inspected one level deeper.
3. Verify coarse baseline coverage remains available while children are pending.
4. Verify changing render mode does not require changing the compute payload.
5. Verify FTLE-labelled outputs are only produced when Benettin renormalisation is actually performed.
6. Verify energy drift $|\Delta E/E|$ stays below a threshold for bounded trajectories.

Common failure modes. Confusing finite ensemble spread with FTLE. Coupling the compute shader to the active render mode. Using the heuristic as an absolute split oracle rather than a priority signal. Storing only angular coordinates and inheriting branch-cut artefacts. Invalidating parents before children are ready (causing visible holes or flicker).

Numerical failure regions are expected. Especially near: binary-collision approaches, near-triple-collision configurations, decoder seam boundaries, feasibility boundaries in invariant-momentum charts, near-zero-mass-simplex corners, and persistent maximal substepping regions. These are not “dropped pixels”—they are valid labelled outputs of a total pipeline. Separate physical terminal states (collision, escape), decode-time terminal states (degenerate geometry), and numerical terminal states (NaN, timeout) to aid analysis and debugging.

0.36 Cache and invalidation

A tile cache key must include enough information that reusing a cached tile is always correct. At minimum:

```
struct TileCacheKey {  
    // Tile identity
```

```

int z, tx, ty;                // tile coordinates
int T;                        // tile resolution

// Chart/view
ChartID chart_id;
float z0[8], q1[8], q2[8];    // latent slice params
// (or invariant-chart knobs for non-affine charts)

// Decode knobs
float mu_max, alpha_min, q_max;
float r_coll, delta_lambda;

// Integrator
IntegratorID integrator;      // KDK or RK4
float dt_macro;
int N_max;
float r_sub, gamma_sub, T_horizon;

// Event logic
float R_esc;
int k_esc;

// Payload format
uint payload_version;
};

```

If rendering is fused (no stored diagnostics), the key must additionally include render-mode parameters (palette LUT id, compactification mode, etc.).

Invalidation matrix.

Change	Invalidates
Palette / hue scheme / brightness mapping	Render cache only
Overlay toggles / physics blob strength	Render cache only
Integrator choice / Δt_{macro} / N_{max}	Diagnostics + render
Event thresholds (r_{coll} , R_{esc} , k_{esc})	Diagnostics + render
Chart parameters ($\mathbf{z}_0$, $\mathbf{q}_a$)	Diagnostics + render
Decode knobs (μ_{max} , α_{min})	Diagnostics + render
Horizon T	Diagnostics + render
Enabled metrics set	Diagnostics + render
Quality tier change	Diagnostics + render

Important separation: changing the palette or hue scheme should not invalidate diagnostics. Changing the integrator or event thresholds must invalidate diagnostics. This separation is one of the central wins of the two-stage pipeline.

.1 Conventions and numerical defaults

Units. $M = \sum m_i = 1$, $G = 1$, $I = 1$ (scale gauge).

Sigmoid. $\sigma(x) = 1/(1 + e^{-x})$.

Numerical defaults (starting points). Mass logit saturation $\mu_{\max} = 5$; Jacobi guard $\varepsilon = 10^{-6}$; hyperspherical buffer $\alpha_{\min} = 0.05$ rad; momentum cap $q_{\max} = 2$, $p^{\max} = 5$; invariant charts $K_{\max} = 2$, $\gamma_K = 2$; escape persistence $k_{\text{esc}} = 8$; mirror tie $\delta_\lambda = 10^{-12}$; substep trigger $r_{\text{sub}} = 0.05$, $\gamma_{\text{sub}} = 1.5$; macro-step $\Delta t_{\text{macro}} = 10^{-3}$, $N_{\max} = 64$, horizon $T \in [50, 200]$.

Diffusion windows. $W_1 : t \in [T/4, T/2]$, $W_2 : t \in [T/2, 3T/4]$.

Shape-sphere checkpoints. $M \in [8, 16]$ for first implementation.

Coherence weights (first-pass concrete values). $w_n = 1.0$, $w_L = 0.20$, $w_t = 0.25$, $w_d = 0.15$, $w_s = 0.20$, $w_o = 0.35$, $w_E = 0.30$ (shape trajectory dominates; see §0.33 for the full formula).

Acceptance targets. *Energy drift:* good $\epsilon_E < 10^{-5}$; acceptable interactive $\epsilon_E < 10^{-4}$; numerically suspect $\epsilon_E \geq 10^{-4}$. Any tile with a large fraction of suspect samples should refine, lower time step, or be visibly marked.

Collision and escape: collision radius $r_{\text{coll}} = 10^{-4}$ (exposed in research mode); escape radius $R_{\text{esc}} = 10$; escape persistence $k_{\text{esc}} = 8$.

Lookup/inverse tolerances: mass/simplex boundary clamp 10^{-6} ; latent logit/sigmoid clamp $\varepsilon_z = 10^{-6}$; invariant momentum seed norm threshold $\varepsilon_w = 10^{-10}$.

All acceptance constants belong in the cache signature.

Split and refinement thresholds (first-pass defaults).

Threshold	Symbol	Default
Outcome impurity force-split	τ_{imp}	0.10
Level-dependent split threshold	$\tau(\ell)$	$0.20 + 0.02\ell$
Suspect-sample force-split fraction	τ_E	0.05
Arc-length spread threshold	τ_L	0.5
Event-time spread threshold	τ_t	0.3
Direction dispersion threshold	τ_{disp}	0.4
Word-agreement minimum (keep coarse)	τ_w	0.80
FTLE research-mode enable	—	Tier C only

These are starting values; expose all in research mode for tuning.

.2 Cartesian Jacobi decode (reference only)

Map $z_k \rightarrow s_k = R_{\max}(2\sigma(z_k) - 1)$, set $\boldsymbol{\rho} = (s_{\rho x}, s_{\rho y})$, $\boldsymbol{\lambda} = (s_{\lambda x}, s_{\lambda y})$. Not used in main pipeline due to rotation-gauge ill-conditioning when $\|\boldsymbol{\rho}\| \approx 0$, uncontrolled pre-gauge scale, and wasted resolution on near-degenerate geometry.

Incompatibility with the 8D latent chart. This decode path requires 4 configuration latent coordinates $(z_{\rho x}, z_{\rho y}, z_{\lambda x}, z_{\lambda y})$ because it works in unreduced Cartesian Jacobi coordinates. The canonical-frame decode (§0.6.2) fixes rotation and scale, reducing configuration to 2 DOF (α, β) and eliminating the $z_{\lambda x}, z_{\lambda y}$ directions entirely. The Cartesian path is retained here only as a historical reference for validation against the canonical path.

.3 Colour map quick reference

Mode	Encoding	Perceptually uniform	CB safety
VMF OKLAB	Full 2D	✓	Deutan issues
VMF Okabe–Ito	Full 2D	✓	Safe (all types)
Viridis, Cividis	Full (LUT)	✓	Safe / Optimised
Stability×hue	Full + physics	✓	Safe
Cubehelix	Full (LUT)	Monotone L	Tolerant

Key parameters. $L \in [0.35, 0.90]$; $C \in [0.05, 0.22]$; $\kappa \in [0.5, 12]$; stripe frequency $f \in [2, 14]$ (integer for seam-free); Fibonacci $N \in [12, 96]$.